Technische Hochschule Ingolstadt

Bachelor's Thesis

Aircraft and Vehicle Informatics
Faculty of Computer Science

in Cooperation with
Airbus Defence and Space GmbH

Generic aspects of porting a Linux Ethernet driver to the L4Re microkernel

Name and Surname	:	**Paul Richter**

Issued on	:	24.10.2025
Submitted on	:	15.01.2026

First Examiner	:	Prof. Dr. Ulrich Margull
Second Examiner	:	Prof. Dr. Robert Gold
Company Advisor	:	Laurence Hupp

Declaration

in Accordance with §30 Abs. 4 Nr. 7 APO THI

I hereby declare that this thesis is my own work, that I have not presented it elsewhere for examination purposes and that I have not used any sources or aids other than those stated. I have marked verbatim and indirect quotations as such.

Ingolstadt, 15.01.2026

Paul Richter

Abstract

In safety-critical domains such as the automotive and aircraft industry, critical software that can endanger human life must be strictly isolated from non-critical software with limited impact. A microkernel supports this separation; this thesis focuses on the L4Re microkernel, and, since microkernels are generally less widely used, there are no existing device drivers for some hardware, including modern Network Interface Cards (NICs).

This also applies to the Intel i225 NIC integrated into the Ecrin aviation computer, which cannot be used without a suitable device driver. The goal of this thesis is to implement an i225 NIC device driver for L4Re and to assess how additional NIC device drivers can be ported to the same framework.

To reduce effort, the initial plan was to port the Linux igc device driver. After comparing multiple candidate frameworks, ixl was identified as the most suitable choice, but it requires rewriting the device driver instead of directly porting the existing code, using the Linux igc device driver only as a conceptual basis. Ixl already supports three Intel NICs, and the underlying hardware—especially between the igb and igc device driver families—is largely compatible, so the existing ixl igb device driver was adapted, mainly by changing register and bit definitions, to implement the ixl igc device driver.

The ixl igc device driver implemented in this thesis currently uses only basic features and offers substantial potential for advancement, both by enabling further performance-oriented mechanisms of the i225 and by adding features such as VLAN and TSN that are particularly interesting for the Real-Time domain. The thesis also identifies the current limitations of ixl, namely that it only supports PCI NICs and a single Rx/Tx queue, and shows that, within these constraints, additional device drivers can be integrated: porting further Intel drivers is straightforward due to shared hardware design, while adapting non-Intel drivers remains feasible by mapping common NIC functionality onto the ixl interfaces using existing drivers and datasheets as references.

Abbreviations

EIAC Extended Auto Clear Register. 69

EIAM Extended Interrupt Auto Mask Enable Register. 68, 69

EICR Extended Interrupt Cause Register. 68, 69

EIMS Extended Interrupt Mask Set/Read. 69

EITR Extended Interrupt Throttle Register. 69

GPIE General Purpose Interrupt Enable. 68

ICU Interrupt Control Unit. 57

IVAR Interrupt Vector Allocation Registers. 68

LOC Lines of code. 38, 39

Mtu Maximum Transmission Unit. 77

RSS Receive Side Scaling. 70

RTOS Real Time Operating System. 70, 83

Rx Receive. 49, 52, 54, 56–63, 66–70, 75–79, 81, V

TCTL Transmit Control Register. 64

TSN Time-Sensitive Networking. 70

Tx Transmit. 49, 52–54, 56, 57, 63, 64, 68, 70, 76–79, 81, V

VM virtual machine. 15, 23, 28, 41, 44, 45

VNS virtual network switch. 22, 23, 31, 37–39, 41, 44–46, 54–56, 66, 67, 69, 73, 74, 76, 78, 80, 81

Glossary of Technical Terms

APIC The **Advanced Programmable Interrupt Controller**. A hardware component in x86 architectures that manages interrupt delivery to the CPU, crucial for handling I/O events in a multi-core environment [Url23, pp. 8–10]. 9

Capability In L4Re, an unforgeable token that grants a task a specific right to access or operate on a particular system object (e.g., memory, a device), central to the system's security model. For more details see subsection 2.4.2. 15–21

CPU The **Central Processing Unit**. The hardware component that executes instructions. Its execution modes (privileged and unprivileged) enforce system security. This thesis specifically focuses on the x86 architecture. 3–6, 8–11, 42, 43, 70, 77

DDE The **Device Driver Environment**. A specialized execution environment or compatibility layer integrated into an operating system that is designed to host and run device drivers which were originally developed for, and executed on, a different operating system. xliv, 26–30, 34–37, 82

Device driver The specialized software usually provided by a hardware manufacturer that abstracts the device's specific technical details, providing the operating system with a standardized interface to control the device. xliv, xlv, 1, 2, 6, 8, 9, 12, 13, 18, 21, 23–28, 30–33, 35, 36, 38, 39, 41, 43, 45, 47, 49–51, 53–57, 60, 66–69, 71, 72, 77–83, V

DMA The **Direct Memory Access** mechanism. A hardware feature that allows I/O devices (like a NIC) to transfer data directly to/from memory without continuous CPU involvement, boosting throughput. 6, 16, 24, 30, 43, 50, 51, 57, 59, 60, 62, 63, 79, 81

Ecrin A PC certified for military standards, specifically configured with an Intel i7-1185GRE processor and both i210 and i225 NICs. 2, 3, 6, 41, 42, 71–75, V

I/O server In the L4Re framework, the I/O server is the central L4Re Application that manages access to I/O devices through the vbus. For more information see subsection 2.4.7. 15, 19, 21, 22, 32, 33, 35, 37–39, 44, 71–73

IDT The **Interrupt Descriptor Table**. A data structure used to locate the appropriate handler for a given IRQ. 9

Igc device driver The device driver for Intel Gigabit Ethernet controllers. This thesis focuses on porting this Linux driver to the L4Re operating system using the ixl framework and testing its functionality on the Ecrin. xlv, 2, 25, 31–33, 36, 40–43, 45–50, 52–59, 63, 66–72, 74–78, 82, 83, V

IOMMU The **I/O Memory Management Unit**. A hardware component that translates I/O virtual addresses used by DMA devices into physical memory addresses, enforcing memory protection and device isolation. 6, 43

IPC The mechanism for **Inter-Process Communication**. It is the essential facility used by processes (or Micro-Apps in a microkernel operating system) to communicate and request services from one another by securely crossing the isolation boundary enforced between their distinct address spaces by virtual memory. This mechanism is used very extensively in microkernels. 11, 13, 14, 16–18, 21, 35, 37

IRQ The **Interrupt Request**. This is a crucial method to halt the normal execution flow of the CPU, forcing it to execute a specific block of code known as the ISR. It is heavily utilized by hardware devices to signal urgent events (like data being ready for processing), and the corresponding ISR is typically part of the device driver's code. 4, 8, 9, 16, 25, 48, 50, 51, 53

ISR The **Interrupt Service Routine**. The function executed by the CPU in response to an IRQ, before returning control to the interrupted program. 4, 9, 49, 50, 80, 81

Ixl The ixl framework is a user space device driver framework adapted from the ixy project specifically for the L4Re operating system. It provides useful abstractions that simplify the process of developing new device drivers for L4Re. xlv, 30, 31, 37–41, 43–47, 51–57, 59, 60, 67, 72, 73, 76–83, V

Kernel The Kernel is the core component of an operating system [Bel22, p. 18], defined as all software that is executed in privileged mode. Its role is to provide core functionalities and manage fundamental system resources, including memory management, scheduling, and IPC. 1, 10–19, 21, 24, 26–28, 30, 34, 36

Kernel space The protected region of virtual memory reserved for the kernel code and data, accessible only when the CPU is in privileged mode. 11, 18, 23

L4Re The **L4 R**untime **e**nvironment operating system framework. A modern, capability-based microkernel system designed for security and real-time use, providing a foundation for building operating systems with strong isolation. 2, 14, 16, 18, 19, 21, 23, 26, 32, 33, 35, 37–41, 43, 45, 46, 53, 71–74, 77, 82, V

L4Re Application An L4Re Application is an application that runs on top of the L4Re runtime environment. For a more detailed description see subsubsection 2.4.3.1 and subsubsection 2.4.6.1. 10, 16, 18–21, 33, 35, 37–39, 41, 44–46, 73, 74

L4Re runtime environment The L4Re runtime environment abstracts the basic microkernel interfaces for ease of use and is comprised of many modular user-space components for example Sigma0, Moe, Ned, I/O server. Together they simplify the development of L4Re Applications. 14, 15, 17, 18, 20, 35, 43, 73

Micro-App A Micro-App is a user developed L4Re application (e.g. Real-Time application or a device driver). 15, 18, 31, 41, 44, 45

Microkernel An operating system architecture that minimizes the code running in privileged mode by moving most services (e.g., device drivers, filesystems) to isolated user space processes. 1, 2, 10, 13–18, 20, 21, 23, 26–28, 30, 34, 35, V

MMIO The **Memory-Mapped I/O** technique. A method that allows the CPU to interact with hardware devices by treating their device registers as if they were specific, designated addresses in system memory. 5, 50, 51, 57, 72, 81

MMU The **Memory Management Unit**. Hardware responsible for translating virtual memory addresses into physical memory addresses [Bau23, p. 91], enabling memory protection and paging. 5, 6

Moe Moe is the L4Re resource manager. It includes abstraction for more flexible memory management, loads the init program Ned, and obtains the memory it needs from the Sigma0 resource manager. 15, 19, 20, 32, 33, 35, 39

Monolithic operating system An operating system architecture where the major part of the operating system's functionality—including system services and all device drivers—is executed as a single, large entity within the kernel's address space and runs entirely in privileged mode (e.g., Linux, Windows). 1, 12, 13

MSI The **Message Signaled Interrupt** mechanism. A PCI feature where a device signals an interrupt by performing a special memory write operation, replacing the older pin-based IRQ lines approach. 8, 9, 42, 43, 50, 69

MSI-X MSI-X is the improved version of MSI defined by the PCI revision 3.0. For more details see the end of subsubsection 2.1.6.1. 9, 42, 43, 49, 50, 57, 66–70

NAPI A Linux kernel mechanism used to reduce the overhead of interrupt storms by employing a polling strategy. NAPI is responsible for processing received packets and freeing transmitted packets.. 48–50

Ned Ned is the L4Re init program. It sets up the further system according to Lua Configuration script. For more details see subsection 2.4.6. 15, 19–22, 37, 39, 44, 74

NIC The **Network Interface Card**. The hardware device that provides the physical connection for a computer to a network (e.g. WLAN network card, or Ethernet network card). xliv, 24–26, 30, 31, 36, 37, 39, 41–44, 49–53, 55, 57–67, 69–73, 75–83, V

PCI The **Peripheral Component Interconnect** standard. A foundational computer bus architecture for attaching hardware devices. 3, 6–9, 21, 36, 43, 47, 50, 54, 55, 67, 78, 81

PCIe The **Peripheral Component Interconnect Express** standard. A high-speed bus that replaced PCI, serving as the primary interface for modern high-bandwidth peripherals. 3, 6, 7, 9, 36, 47, 54, 67

Privileged mode The highest CPU execution state that grants the running process full, unrestricted access to all hardware resources and instructions. 10, 11, 15

Sigma0 The Sigma0 process is the fundamental L4 resource manager that runs in user space. It is granted control over all physical memory (including I/O memory addresses) mapped as virtual memory. For more details see subsection 2.4.4. 15, 18–20, 32, 33, 35, 36, 39

Unprivileged mode The execution state of the CPU that grants the running process restricted access used for running application programs (on microkernel operating systems also system services). It restricts the code from performing privileged operations (like direct hardware access or memory management) to enforce system security and isolation. 10, 11

User space The memory and execution environment where application programs run in unprivileged mode isolated from other applications. In microkernel operating systems also operating system functionality runs in user space. 11, 13, 15, 17, 18, 21, 23, 30

Vbus The **Virtual Bus**. In the context of the L4Re framework, the vbus is a virtualized version of the physical system bus. This virtual bus can be logically separated into smaller segments, allowing individual segments to be securely granted or assigned to specific L4Re applications for direct device interaction. 21, 22, 37, 39, 44, 55, 71–73

Virtual memory A memory management technique that provides each process with an isolated address space, implemented by the MMU to ensure protection and multitasking functionality. 5, 6, 11, 13, 17

Contents

1 Introduction **1**

2 Current state of research **3**

2.1 PC Architecture . . . 3

2.1.1 CPU & Basic System Overview . . . 3

2.1.2 IRQ . . . 4

2.1.3 Virtual Memory . . . 5

2.1.4 Memory Mapped I/O . . . 5

2.1.5 Direct Memory Access (DMA) . . . 6

2.1.6 PCIe/PCI . . . 6

2.1.6.1 MSI . . . 8

2.2 Operating System . . . 10

2.2.1 Kernel & User Space . . . 11

2.2.2 Process Isolation & IPC . . . 11

2.2.2.1 Message Passing . . . 11

2.3 Microkernel OS vs. Monolithic OS . . . 12

2.3.1 Monolithic OS . . . 12

2.3.2 Microkernel OS . . . 13

2.4 L4Re . . . 14

2.4.1 System Overview . . . 14

2.4.2 Capabilities . . . 15

2.4.3 L4Re microkernel . . . 17

2.4.3.1 L4Re Applications . . . 18

2.4.3.2 Micro-App . . . 18

2.4.4 Sigma0 . . . 18

2.4.5 Moe . . . 19

2.4.6 Ned . . . 20

2.4.6.1 L4Re Application . . . 20

2.4.7 Io Server 21
2.4.8 Virtual Network Switch 22
2.4.9 lwIP 23
2.5 Linux Device Driver Interactions 23
2.5.1 Core Concepts of a Network Interface Card 24
2.5.2 Linux Ethernet Device Drivers 24
2.6 OSI Model Context 25
2.7 Device Driver Porting Frameworks 26
2.7.1 Porting framework for Linux Network Device Drivers to the L3 microkernel 26
2.7.2 Porting framework for Linux Drivers to the DROPS operating system 26
2.7.3 Genode & DDE 27
2.7.4 Ixl 30

3 Porting Framework Evaluation 32

3.1 Comparison Criteria 32
3.1.1 Compatibility 32
3.1.2 Integration Effort 33
3.1.3 Maintainability 33
3.2 Framework Evaluation 34
3.2.1 Porting framework for Linux Network Device Drivers to the L3 microkernel 34
3.2.2 Porting framework for Linux Drivers to the DROPS operating system 34
3.2.3 Genode & DDE 34
3.2.3.1 Evaluating Compatibility to L4Re for Genode & DDE (C1) 34
3.2.3.2 Evaluating the Integration of Genode & DDE in L4Re (C2) 35
3.2.3.3 Evaluating Maintainability of Genode & DDE within L4Re (C3) 36
3.2.4 Ixl 37
3.2.4.1 Evaluating Compatibility to L4Re for Ixl (C1) 37
3.2.4.2 Evaluating the Integration of Ixl in L4Re (C2) 38
3.2.4.3 Evaluating Maintainability of Ixl for L4Re (C3) 39
3.3 Framework Selection 40

4 Concept 41

4.1 Functional Requirements 41
4.2 Architecture 42
4.2.1 Hardware Architecture 42

4.2.2 Software Architecture 43
4.2.2.1 Component Initialization and Capabilities 44
4.2.2.2 Client Connection and Data Flow 44
4.3 Testing 45
4.3.1 Test 1: Ethernet Frame Reception 45
4.3.2 Test 2: Ethernet Frame Transmission 45
4.3.3 Test 3: Multiclient test 46
4.3.4 Test 4: Performance (Bandwidth) Assessment 46

5 Implementation 47
5.1 Analysis 47
5.1.1 Analysis of Linux igc device driver 47
5.1.2 Analysis of ixl abstract device driver class 51
5.1.3 Matching ixl & Linux kernel API 52
5.1.4 Evaluating ixl and linux function compatibility 52
5.2 Igc Driver Implementation in ixl 54
5.2.1 Driver Registration 54
5.2.2 Driver Initialization 56
5.2.2.1 Basic Initialization 56
5.2.2.2 Device reset & complete initialization 57
5.2.3 Implementing Receive (Rx) Functionality 58
5.2.3.1 init_rx() 58
5.2.3.2 start_rx_queue() 60
5.2.3.3 rx_batch() 61
5.2.4 Implementing Transmit (Tx) Functionality 63
5.2.4.1 init_tx() 63
5.2.4.2 start_tx_queue() 63
5.2.4.3 tx_batch() 64
5.2.5 Implementing Interrupt (IRQ) Handling 66
5.2.5.1 msix_init() 66
5.2.5.2 Interrupt Settings 68
5.2.5.3 enable_rx_interrupt() 68
5.2.5.4 check_recv_irq() & ack_recv_irq() 69
5.3 Feature Comparison between ixl igc driver and igc hardware functionality . 70

6 Testing **71**

6.1 Test 1: Ethernet Frame Reception 71

6.1.1 Execution of the Test 72

6.1.2 Results 72

6.2 Test 2: Ethernet Frame Transmission 72

6.2.1 Execution of the Test 73

6.2.2 Results 73

6.3 Test 3: Multiclient test 73

6.3.1 Execution of the Test 73

6.3.2 Results 74

6.3.3 Conclusion 74

6.4 Test 4: Performance (Bandwidth) Assessment 74

6.4.1 Execution of the Test 75

6.4.2 Results 75

7 Evaluation of porting efforts **76**

7.1 Evaluating igc porting efforts 76

7.1.1 Evaluation of Performance Results in Comparison to Linux igc Driver 77

7.2 Evaluating the generic aspects of porting additional Ethernet drivers 78

7.2.1 Evaluating the porting of Intel Ethernet drivers 78

7.2.2 Evaluating the porting of non-Intel Ethernet drivers 79

8 Summary & Future Work **82**

A Igc code examples **xv**

B Test Utilities and Results **xxxvii**

List of Figures **xliv**

List of Tables **xlv**

Chapter 1

Introduction

In the automotive and aircraft industries, where malfunctioning software can have critical impact, like harming human life, it's essential for the software to work correctly. To ensure the safety of software, it's developed according to safety standards which increases the development effort.

Therefore, it's common to separate software into safety-critical and non-safety-critical parts. This means the non-safety-critical parts do not have to be developed according to strict safety standards, thereby reducing complexity and development time.

However, when safety-critical and non-safety-critical software run on the same hardware, it's crucial to ensure that they cannot influence each other. For example, commonly used operating systems like Windows and Linux are monolithic [Nov25, p. 23]. This means if any process creates an error at kernel level, the entire system can be compromised. Hence, a non-critical piece of software can influence a critical part of software on a monolithic operating system, by crashing the whole system.

A microkernel operating system solves this problem by drastically reducing the amount of code that runs inside the kernel, thereby significantly decreasing its attack surface [TB15, pp. 65–66][L4R25c, p. 5]. Furthermore, in a microkernel architecture, most system services are moved out of the kernel and split into smaller components that run in isolated domains. If one of these components fails, the fault is contained, and it cannot take down the entire system [TB15, pp. 65–66]. This isolation and resulting increase in reliability make microkernels a perfect choice for embedded systems [Roc04, pp. 4, 5][VA11, p. 32].

A common challenge with using a microkernel operating system is that devices such as graphics cards, sensors, or network cards often do not work out of the box. This is due to the absence of a matching device driver for these devices on the microkernel operating system.

Device drivers are specialized software provided by manufacturers that know exactly how to communicate with the hardware they were made for. They are necessary because, even

among similar devices, the electronic design can vary significantly, meaning no single piece of software can control them all.

The problematic aspect is that these device drivers are always operating-system-specific [Hel01, p. 1]. This creates a problem because microkernel operating systems are currently niche products primarily used in the embedded sector [VA11, p. 32], and manufacturers therefore lack the incentive to develop and provide compatible device drivers. These missing, operating system-specific device drivers are the direct cause of the original problem: devices not working by default on microkernel systems.

To utilize every hardware with a microkernel system, one must either re-implement each driver for the operating system—an enormous task given the variety of different hardware—or adopt a more efficient approach: finding a way to convert existing device drivers to work with the microkernel system [Hel01, p. 1].

The optimal solution to this challenge would be to create a generic framework capable of porting any driver. However, this is problematic because every driver is operating-system-specific, and not all operating systems or vendors make their driver code public. For this reason, most research papers in this field use Linux device drivers, as their code is open-source.

Another point of difficulty lies in the fact that driver architecture differs significantly depending on the hardware device. The fundamental differences across various hardware devices make it challenging to create a single, unified framework. Consequently, there have been attempts to create specialized frameworks for only one class of device drivers, such as network cards [Sta96].

The primary goal of this thesis is to analyze different frameworks to find an efficient way to port the Intel igc device driver to the L4Re microkernel. This porting effort is based on the igc source code publicly available in the Linux kernel [Tor25]. The resulting implementation must ensure that the ported device driver can be used by multiple applications on the L4Re. The implementation will be limited to porting only one driver. It serves as a foundation for evaluating the porting of further device drivers.

The ported device driver will be tested on the nanoONYX, which, for simplicity, will be referred to as “Ecrin” after its manufacturer. To verify the successful implementation of the igc device driver, an Ethernet frame will be sent from the Ecrin to another PC, and a separate frame will be sent back, with the contents of both frames being compared for verification.

Chapter 2

Current state of research

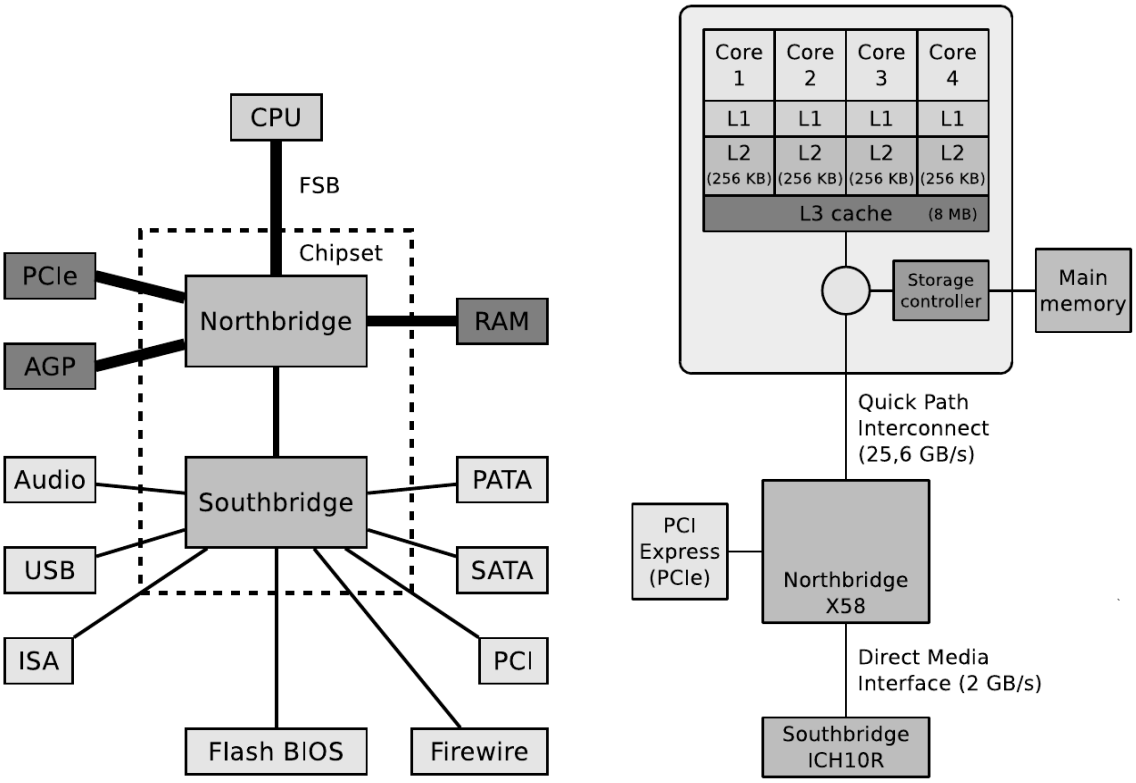
2.1 PC Architecture

2.1.1 CPU & Basic System Overview

It is assumed that the reader has a good understanding of how the processor, also called the Central Processing Unit (CPU), works, and is familiar with the von Neumann Architecture.

The Ecrin utilizes the i7-1185GRE processor in this configuration [ecr25]. This processor utilizes the Intel64 instruction set [Int]. Intel64 is Intel's implementation of the original instruction set developed by AMD, which was called AMD64. This architecture is generally referred to as x86-64 [SGG12, p. 387]. This thesis focuses specifically on the x86-64 architecture.

Baun states that using a single bus to connect all I/O devices to the CPU, as idealized in the Von Neumann architecture, is no longer practical for modern computer systems [Bau23, p. 41]. Instead, different types of bus systems are used for connecting peripheral devices, each fulfilling a different purpose [Bau23, p. 43]. Figure 2.1a provides an abstract overview of a modern PC architecture, illustrating these different bus systems and how they connect to the CPU. The particular bus systems Peripheral Component Interconnect Express (PCIe) and Peripheral Component Interconnect (PCI) will be discussed in detail in subsection 2.1.6. Furthermore, parts of the chipset have been integrated directly into the CPU for better performance and reduced cost. For example, Figure 2.1b visualizes the architecture of the Intel Core i7-975 Extreme Edition, demonstrating that the memory controller was moved onto the CPU itself [Bau23, p. 44].



(a) Abstract Overview of the PC architecture (b) Intel Core i7-975 Extreme Edition

Figure 2.1: Overview of modern computer systems adapted from [Bau23, pp. 43–44]

2.1.2 IRQ

An interrupt, or Interrupt Request (IRQ), is a mechanism that stops the CPU [SGG12, p. 8]. When an interrupt is triggered, the CPU immediately pauses its current execution [SGG12, p. 8] and saves the complete context (data and state) of the ongoing process [SGG12, p. 138]. The CPU then jumps to a specific starting address associated with that interrupt—the Interrupt Service Routine (ISR) —and executes the instructions found there [SGG12, p. 8]. Once the ISR finishes handling the request, the CPU restores the saved context [SGG12, p. 138] and resumes the execution of the interrupted task [SGG12, p. 8]. The process of halting the CPU saving data and state to return at later point is called context switch [SGG12, p. 114].

An interrupt is often used to signal the occurrence of an event. Both software and hardware can trigger an interrupt. Software can trigger an interrupt by executing a special operation known as a system call (System calls are an interface for operating system services [SGG12, p. 62]). Conversely, hardware triggers an interrupt by sending a signal to the CPU, typically via the system bus [SGG12, p. 8].

Furthermore, an interrupt can be temporarily disabled, a process known as masking; conversely, its subsequent reactivation is referred to as unmasking.

2.1.3 Virtual Memory

Virtual memory is a concept used to abstract the hardware implementation of physical memory. It is inefficient to address a large linear physical address space, but applications need a large contiguous address space. By virtualizing physical memory, a large, contiguous virtual address space can be created [SG23, p. 382]. In addition, virtual memory also isolates the address spaces of different applications (see subsection 2.2.2).

Both physical and virtual memory are divided into fixed-size partitions. The partitions of virtual memory are called pages and the partitions of physical memory are called page frames. A page can now be assigned to any location (page frame) in memory. This association is stored in a page table, which is controlled by the operating system. This virtualization of memory allows a contiguous virtual address space to be distributed into different areas of physical memory [SG23, pp. 382–383]. The concept is also visualized in figure Figure 2.2. The translation of virtual memory addresses to physical memory addresses is handled by a hardware device called the Memory Management Unit (MMU), which is integrated into the CPU [TB15, p. 28][Bau23, p. 91][SGG12, p. 356].

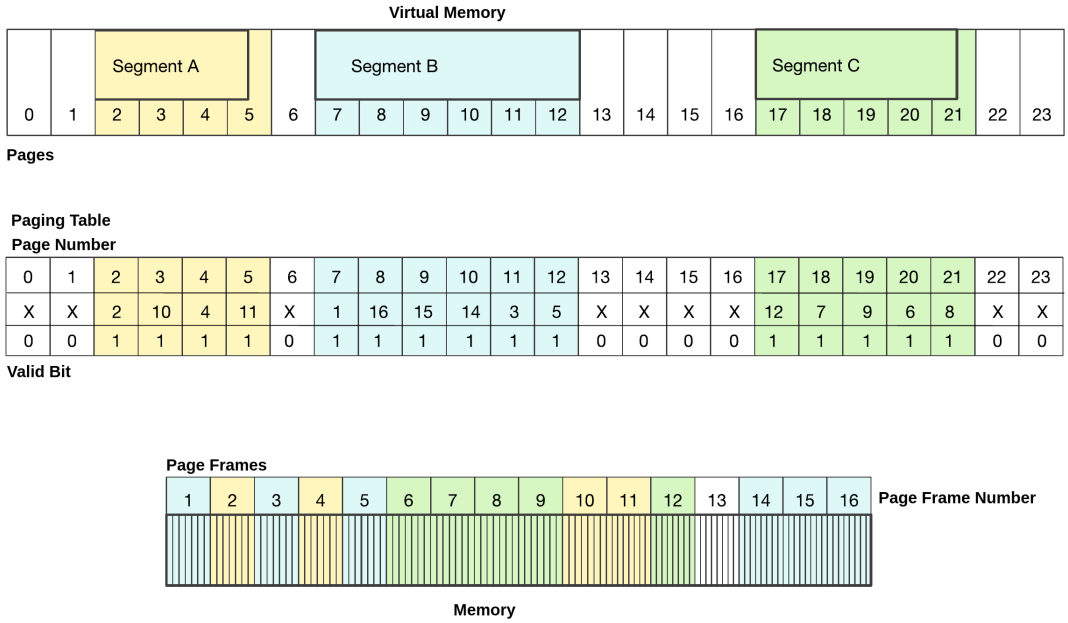


Figure 2.2: Paging Example adapted from [SG23, p. 383]

2.1.4 Memory Mapped I/O

Memory-Mapped I/O (MMIO) describes the concept that the CPU can communicate with a device as if it were regular memory [TB15, p. 30]. This is possible because the device's registers are addressable over the system bus [Bau23, p. 42]. Consequently, the CPU can use the same instructions for device communication as it does for memory [TB15, p. 342]. The key difference that must be accommodated is that these I/O addresses can change their value independently, unlike regular memory [CRK05, p. 236]. Therefore, the compiler

must be instructed not to reorder or optimize these memory access instructions, and the processor's cache must be disabled for these specific addresses [CRK05, p. 237].

2.1.5 Direct Memory Access (DMA)

Direct Memory Access (DMA) is a hardware device that, once configured by the device driver, can copy data between I/O devices and memory without involving the CPU [BC05, p. 547][OJ01].

DMA is especially useful for large data transfers [SGG12, p. 12] because if the CPU had to manage the transfer, it would be blocked simply for copying data, wasting valuable processing time [TB15, p. 344]. A DMA controller can be physically located on a device itself or on the motherboard [TB15, pp. 34, 344].

DMA devices vary in complexity, and consequently, the registers required to control them also differ. Tanenbaum and Bos gives the example of a simple DMA with the following registers that the CPU uses for setup: a memory address register, a byte count register, and control registers [TB15, pp. 344–345].

A challenge arises because DMA controllers often operate using physical memory addresses [TB15, p. 346]. If a DMA operates this way, it requires a large amount of contiguous physical RAM to store the data. As virtual memory is typically scattered across non-contiguous physical memory pages (as explained in subsection 2.1.3), the operating system is required to provide special functions that explicitly allocate contiguous physical memory [Hel01, p. 21].

Another solution to the contiguous physical memory problem is to use a memory management unit to implement the principle of virtual memory (see subsection 2.1.3). Modern systems provide a dedicated MMU for I/O operations, called the I/O Memory Management Unit (IOMMU) [ABY11, p. 1]. The IOMMU translates the virtual addresses requested by the DMA into physical addresses. Consequently, it also prevents DMA devices from accessing physical memory addresses they are not supposed to [Ben+07, p. 9], as the DMA only has access to physical RAM regions that the operating system has explicitly mapped within the IOMMU. The Ecrin processor is an example of a modern architecture that utilizes an IOMMU [ABY11, p. 3][Int].

2.1.6 PCIe/PCI

PCIe stands for Peripheral Component Interconnect Express [SA99, p. 8][BAS03, p. 48]. As the name suggests, it is used to connect peripherals within a computer system.

PCIe is an improved version of its predecessor, the PCI standard, offering significantly enhanced bus speed. Astonishingly, the fundamental concepts for memory mapping and

device configuration remain the same as in PCI [BAS03, p. 48]. Budruk, Anderson, and Solari even state that existing operating systems and driver software developed for PCI will still work on PCIe without modification [BAS03, p. 48]. To focus on PCIe functionality relevant to driver development, electronic details are omitted. The explanation will instead cover the core PCI concepts, because they still remain the same in the PCIe standard.

PCI is a relatively old standard; the specification for version 1.0 was published by Intel in 1992 [SA99, p. 7]. It was designed to be platform independent and to simplify the process of adding and removing devices from the system [CRK05, p. 302].

PCI uses a tree topology, as visualized in Figure 2.3 [CRK05, p. 303]. While the PCI bus specification defines three entity types, the two relevant to this discussion are the PCI-to-PCI bridge, which hosts its own subordinate bus, and the PCI device, which represents an endpoint in the tree [SA99, p. 352].

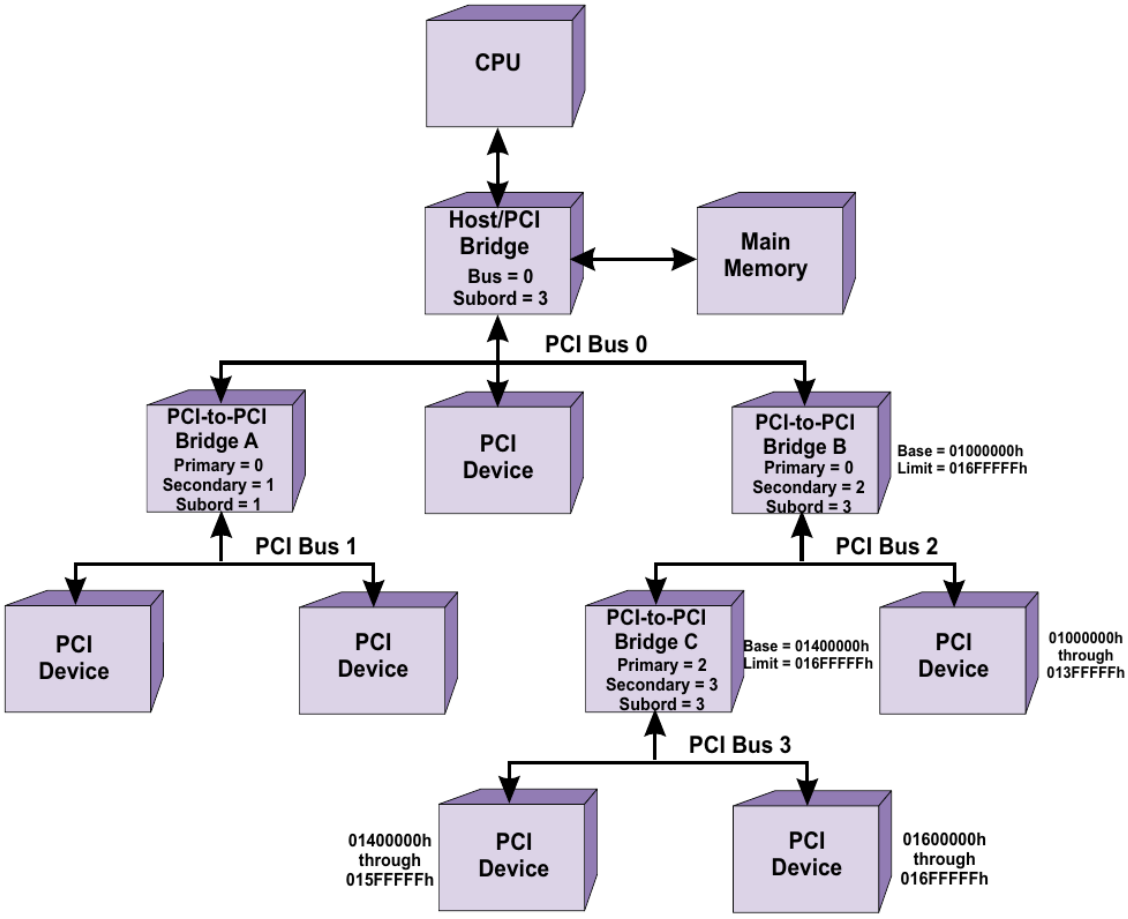


Figure 2.3: PCI System Example [SA99, p. 546]

From the PCI specification, a PCI device (which can be imagined as a physical slot-in card for simplicity) can host up to eight separate, independent device functions [SA99, p. 17]. Corbet, Rubini, and Kroah-Hartman provide the example of a single audio device that could additionally incorporate a CD-ROM drive [CRK05, p. 303]. These independent functions will henceforth be referred to as PCI device functions or simply device functions.

A PCI device function has several distinct address spaces, the most important of which are the memory locations and the configuration registers [CRK05, pp. 248, 305]. Configuration registers are accessed using a specific PCI addressing scheme [CRK05, pp. 305–306]. In contrast, the memory locations can be accessed using standard memory operations once they have been properly set up [CRK05, pp. 305, 307].

PCI has its own addressing scheme [CRK05, p. 305], which identifies device functions based on a bus number and a device number and function number [SA99, pp. 318, 321]. The only hard-coded address is that of the host/PCI bridge, which is always Bus 0 [SA99, pp. 543–544]. All other bridges are assigned a primary bus number, a secondary bus number, and a subordinate bus number during system startup by the BIOS [SA99, pp. 544, 674]. The process of assigning these numbers will not be discussed here. Once these numbers are assigned, it becomes possible to configure each device's configuration space [SA99, p. 547].

This configuration process is essential because every device is initially inactive when the system is powered on. This means no memory regions are mapped yet. A device can only be addressed using the PCI-specific addressing scheme to configure its configuration registers. With the help of these configuration registers, the firmware or operating system can assign memory addresses for the device [CRK05, p. 307].

A PCI device function is identified by the values stored in a combination of different configuration registers (such as the Vendor ID and Device ID). This unique identification allows the operating system to discover the corresponding device driver [SA99, p. 354]. The device driver can then obtain the device's assigned memory addresses from the configuration registers [CRK05, p. 315]. After the device driver obtains the correct memory addresses for its device, it can proceed with initialization [CRK05, p. 303].

Although the concept PCI may sound simple, many details have been omitted here. In order to abstract away specifics, such as how the PCI addressing scheme works or which registers and bit fields contain the information a device driver needs, operating systems provide a PCI subsystem to handle and abstract this underlying complexity [Mic] [CRK05, pp. 302–325].

2.1.6.1 MSI

On PCI there are two primary methods for a device function to signal an interrupt request to the CPU [SA99, p. 252]: traditional pin-based signaling, where a dedicated physical pin on the device is connected to an IRQ line [SA99, pp. 244–245], or Message Signaled Interrupt (MSI) [SA99, p. 252]. To fully understand the benefits of MSI, it is first necessary to examine the architectural disadvantages and performance limitations inherent in the legacy pin-based method.

The legacy x86 interrupt management, historically relying on the Programmable Interrupt Controller (PIC), supported a severely limited number of physical IRQ lines, typically

constrained to 15 [Url23, pp. 8–9]. Simplified, an IRQ line's primary function is to signal an interrupt request to the CPU [MLD18, p. 3]. Due to this limited number of IRQ lines, some PCI device functions were forced to share a single IRQ line [SA99, pp. 244–245]. In simplified terms, when a shared interrupt was asserted, the processor consulted the Interrupt Descriptor Table (IDT), which holds the entry address for the first dedicated ISR, initiating a process where the ISRs for all device drivers sharing that IRQ line were executed in a linked chain. Each driver was then required to poll its corresponding device registers to determine if it was the source of the interrupt [SA99, pp. 244–250]. This mandatory polling cycle resulted in considerable processing overhead. This architectural bottleneck was fundamentally addressed with the introduction of the MSI standard in the PCI Specification 2.2 [SA99, pp. 221, 252].

MSI is only compatible with newer interrupt controllers, such as the Advanced Programmable Interrupt Controller (APIC) [Url23, p. 10]. Instead of relying on physical IRQ lines, a device configured for MSI invokes an interrupt by performing a standard memory write transaction to a special memory address, accompanied by specific message data, on the PCI Bus [SA99, pp. 254–255]. This transaction propagates across all intermediate PCI-to-PCI bridges until it reaches the Host PCI Bridge, which recognizes the unique address and message data, converting the memory write into a distinct interrupt, thus resolving the issues associated with interrupt sharing and polling overhead [SA99, pp. 255, 257]. During system initialization, each PCI device function on the bus is scanned for MSI support and is then assigned a unique memory address and corresponding message data, which are written into dedicated registers on that PCI device function [SA99, pp. 252–255]. While a PCI device function may request multiple interrupts, which are differentiated by varying this message data, the PCI 2.2 specification only guarantees support for one interrupt, and recommends that the device function still support the pin-based approach for backward compatibility [SA99, pp. 252–255].

The evolution to PCI Express (PCIe) made MSI support mandatory for every PCIe device, only allowing the legacy pin-based approach for certain exceptions. Moreover, any signal issued by a device's interrupt pin in the PCIe architecture is translated into a MSI message for delivery [BAS03, p. 75].

With the PCI revision 3.0, the MSI standard was advanced to MSI-X. This next-generation standard brought several key improvements to interrupt handling, most notably: it features a greater number of distinct IRQs per PCI device function, it improves interrupt distribution to multiple CPUs, and it adds the possibility to assign different priority levels to interrupts belonging to the same PCI device function [Bud+12, pp. 821–822].

2.2 Operating System

An operating system serves as a software layer designed to abstract the complexity of the underlying hardware [TB15, p. 1] and provide a convenient environment for running other software [SGG12, pp. 3–4].

The terminology used for software running on the CPU differs across various operating systems. In general, in operating system literature, a process is defined as a program in execution that possesses its own virtual address space, which is typically structured into heap, stack, data, and code segments. A process must also save sufficient state information to be interrupted and resumed at any time [SGG12, pp. 106, 398–399][Sta18, pp. 132–133]. While an application usually corresponds to a single process, one application can also consist of multiple processes [Sta18, p. 131]. On the L4Re microkernel operating system framework (see section 2.4), different terms are used for these execution units on the CPU, such as Task or L4Re Applications, which will be explained in detail later.

Operating systems vary widely, as they are typically designed with different goals in mind, such as flexibility, convenience, security, or performance [SGG12, p. 3] [Sta18, p. 83].

Regardless of their specific goals, all operating systems provide core functionalities [Sta18] [TB15] [SGG12] [Bau23]:

- **Hardware Abstraction and Resource Management:** The operating system provides an abstraction layer over the hardware, managing access to resources (like the CPU, memory, and I/O devices) and ensuring their efficient utilization.

- **Application Execution and Support:** The operating system enables the execution of multiple independent applications and offers mechanisms for synchronization and cooperation, allowing these applications to work together safely.

To manage and secure these tasks, the operating system relies on hardware assistance, primarily from the CPU [Sta18, p. 77]. Modern CPUs support different operational states, most notably a privileged mode and an unprivileged mode [Sta18, p. 157]. When the CPU is operating in privileged mode, the software running has unrestricted access to all hardware resources and can execute any instruction. This mechanism is crucial: it allows the operating system to remain in control, manage resource access, and significantly reduce the impact of a malfunctioning application or defend against a malicious one that attempts to seize control [Sta18, p. 77].

The part of the operating system that runs in privileged mode is called the kernel [Nov25]. Since only software operating in privileged mode can access hardware directly [Sta18, p. 77], the kernel is responsible for managing all fundamental hardware-related tasks.

2.2.1 Kernel & User Space

The virtual address space (see subsection 2.1.3) is separated into kernel space and user space. This separation is dictated by the virtual address architecture of the processor [Bau23, pp. 102–103].

On the x86-64 architecture, with a system bus width of 48 bits (this detail is for architectural correctness and is not critical for understanding the memory separation concept), the user space address range occupies the lower half, specifically between 0x00000000 00000000 and 0x00007FFF FFFFFFFF, while the kernel space resides in the upper half, spanning from 0xFFFF8000 00000000 to 0xFFFFFFFF FFFFFFFF [Bau23, pp. 102–103].

The kernel address space can only be accessed when the CPU is in privileged mode [Bau23, p. 102] [Sta18, p. 77]. Consequently, the kernel—the component of the operating system that runs in this privileged mode —must be stored within the kernel space. Conversely, software running in unprivileged mode is stored in the user space address range. For this reason, applications running in unprivileged mode are often referred to as user applications or user process [SGG12, p. 22]. If a user space application attempts to access the memory designated for the kernel, it will be immediately terminated [Sta18, p. 139].

In order for processes to use kernel services a user space process must issue a system call [SGG12, p. 594]. This performs a context switch and elevates the CPU into privileged mode handing over the control to the kernel [Bau23, p. 146]. Therefore, system calls serve as an interface for requesting kernel services [SGG12, p. 62].

2.2.2 Process Isolation & IPC

Through the concept of virtual memory, processes are isolated: they can only access their own virtual address space or interact with the operating system via system calls. They have no immediate way to communicate directly with other processes [Aik+06, p. 1]. Therefore, the operating system provides mechanisms to allow Inter-Process Communication (IPC). Some common IPC mechanisms include shared memory, message passing, and sockets [Bau23, p. 208][TB15, p. 144].

2.2.2.1 Message Passing

Message passing is a straightforward IPC concept involving a *send* and a *receive* operation. While these operations fundamentally require system calls, the interface is typically abstracted into a library providing wrapper functions for ease of use [TB15, p. 144].

Utilizing such functions, a sender process can invoke *send()* to transmit a message to another process, which the receiver then retrieves by calling *receive()*. This mechanism can be implemented in either an asynchronous or a synchronous manner [TB15, p. 145].

The asynchronous implementation can be compared to a mailbox with a fixed message capacity. It temporarily stores messages that the receiver has not yet retrieved. If the mailbox is full and a sender process attempts to transmit a new message, that sender process is suspended until a slot in the mailbox becomes free. Note that in this implementation, the *send()* and *receive()* functions would address a mailbox rather than a specific process [TB15, p. 145].

In the synchronous implementation, when a sender process calls the *send()* function, it becomes blocked until the receiver issues the *receive()* call. At this point, the message information is copied directly from one process to the other. Conversely, if a receiver process calls the *receive()* function first, it will be blocked until the sender invokes the *send()* function, after which the information is copied between the processes [TB15, p. 145].

This message passing method requires a system call for every function, and because system calls require a context switch, this fundamentally decreases performance. Tanenbaum and Bos have noted that considerable effort has been made to optimize the efficiency of message passing [TB15, p. 145].

2.3 Microkernel OS vs. Monolithic OS

2.3.1 Monolithic OS

Monolithic operating systems are operating systems that we use every day like Windows and Linux [Nov25, p. 23]. They are called monolithic because they have one large binary as kernel [Nov25, p. 21], that contains the core functionality for the operating system for example managing different processes and handling I/O communication [Roc04].

In a monolithic operating system, device drivers are part of the kernel [Sta18, p. 92]. Device drivers are necessary software to interact with the different hardware devices connected to the PC [TB15, p. 358]. They abstract specific hardware details of the device and provide an interface for the operating system to use the device [SGG12, p. 12]. Therefore, they enable the operating system to work with every device that it has a matching device driver for. To support numerous devices, operating systems can dynamically install device drivers when a new device is registered [SGG12, p. 838].

In a monolithic operating system, a bug within a device driver can lead to a system-wide crash because device drivers execute as part of the kernel [TB15, p. 66]. Indeed, Ishikawa, Courbot, and Nakajima identify device drivers as a major cause of operating system failures [ICN08, p. 277].

2.3.2 Microkernel OS

Microkernel operating systems try to keep the kernel as small as possible for optimal performance, flexibility and correctness [Lie96]. Jochen Liedtke, the leading figure in the development of microkernels [Hei+07, p. 4], argued that a microkernel should contain only the most essential functions, and that everything not absolutely necessary should be moved out of the kernel [Lie95, p. 238]. As essential for a microkernel system, Liedtke later defines two core principles for microkernels [Lie95, p. 238]:

1. **Principle of Independence**: the microkernel must give a developer a way to ensure that two applications cannot influence each other.
2. **Principle of Integrity**: this principle requires a reliable way to communicate between applications. This communication channel must be secure against eavesdropping and corruption.

Based on Liedtke's work, a microkernel can be defined as a kernel that is as small as possible, yet still supports the independent and uninfluenced execution of applications, as well as secure and reliable communication between them.

Forcing the kernel to be as small as possible inevitably means that nearly all operating system services have to implemented in user space [Lie95].

Liedtke proposes the minimal concepts for a microkernel operating system in his work "On Micro-Kernel Construction" [Lie95, p. 238]. He demonstrates that the kernel only needs to implement basic functions for memory operations, which allows the major part of memory management to run in user space [Lie95, pp. 238–239]. Since memory can also encompass I/O addresses (see subsection 2.1.4), this further establishes that hardware management can also be moved to user space [Lie95, p. 239].

In this architecture, the initial operating system process in user space is assigned all physical memory as its virtual memory [Lie95, pp. 238, 240][Döb12b, 1:08:38-1:08:47]. This process then uses kernel functions to manage all memory resources [Lie95, pp. 238, 240]. As a result, other processes must communicate with this memory management process to receive memory. This interaction makes the memory manager an essential system component. For example, a user space device driver must communicate with it to request access to I/O addresses [Lie95, p. 240]. This frequent communication between isolated processes makes IPC (see subsection 2.2.2) a vital mechanism for a microkernel operating system.

One advantage of the microkernel approach is that the operating system can be decomposed into a number of different processes, each protected and isolated through the boundaries of virtual memory at a chosen granularity [Lie95, pp. 237, 240]. In contrast, the kernel in a monolithic operating system typically acts as a single, large protection domain [LW09, p. 25].

The primary challenge of microkernels—ensuring that IPC communication is performant enough to make such a flexible system usable in practice [Lie93, p. 175]—was fundamentally solved by Liedtke. His advanced L3 microkernel version had already demonstrated a significant performance increase in IPC [Lie93, p. 175]. Building on this initial success, Liedtke developed the L4 microkernel as a major redesign of L3 [HE16, p. 2]. The introduction of L4 triggered a 20-year evolution of the L4 kernel family [HE16, p. 2], which is visualized in Figure 2.4.

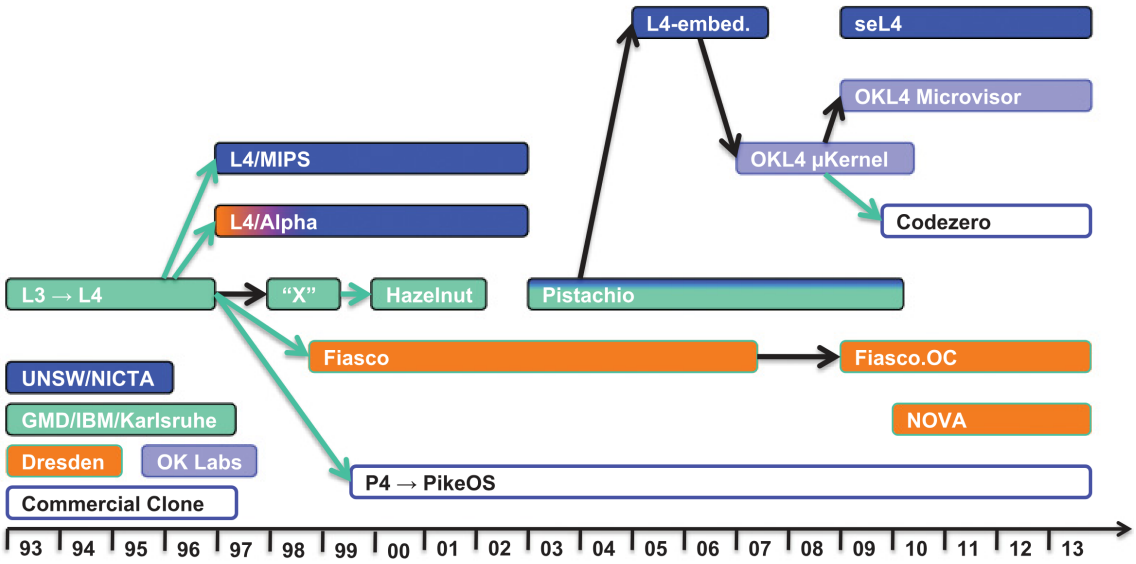


Figure 2.4: The L4 family tree (simplified). The x-axis is a timeline in calendar years. [HE16]

2.4 L4Re

When Liedtke moved from the GMD (today known as Fraunhofer-Gesellschaft) to IBM, both GMD and IBM restricted the usage of the new version "X" through intellectual property. As a result, they forced TU Dresden to develop their own operating system called "Fiasco" which later on evolved into "Fiasco.OC" [HE16, p. 2]. This key development is also visualized in orange in Figure 2.4.

In 2012, Adam Lackorzynski, Michael Hohmuth, and Alex Warg, who researched the Fiasco microkernel at TU Dresden, founded the company "Kernkonzept" [Hoh22]. Kernkonzept currently maintains and develops the open-source version of the L4Re operating system framework in collaboration with individual contributors [L4R25d] [L4R25b]. They also offer consulting services related to L4Re [Hoh22].

2.4.1 System Overview

"The L4Re is an operating system framework" [L4R25d]. It consists of the "L4Re microkernel," also called "Fiasco.OC" [Lac19, p. 7], and the L4Re runtime environment [Lac19, p. 6]. The

microkernel only offers a few basic mechanisms that are necessary in order to support the execution of applications [L4R25c]. Therefore, it is hard to build applications directly on top of the microkernel. The L4Re runtime environment abstracts the microkernel interfaces to make the development of applications more convenient [L4R25c]. With the runtime environment it is possible to run multiple virtual machine (VM)s and Micro-Apps (explained in subsubsection 2.4.3.2) at the same time on the microkernel as depicted in Figure 2.5. The L4Re runtime environment is separated into different software components, such as Sigma0, Moe, Ned, and I/O server, that serve different purposes [L4R25c]. The different software components will be explained in more detail later on.

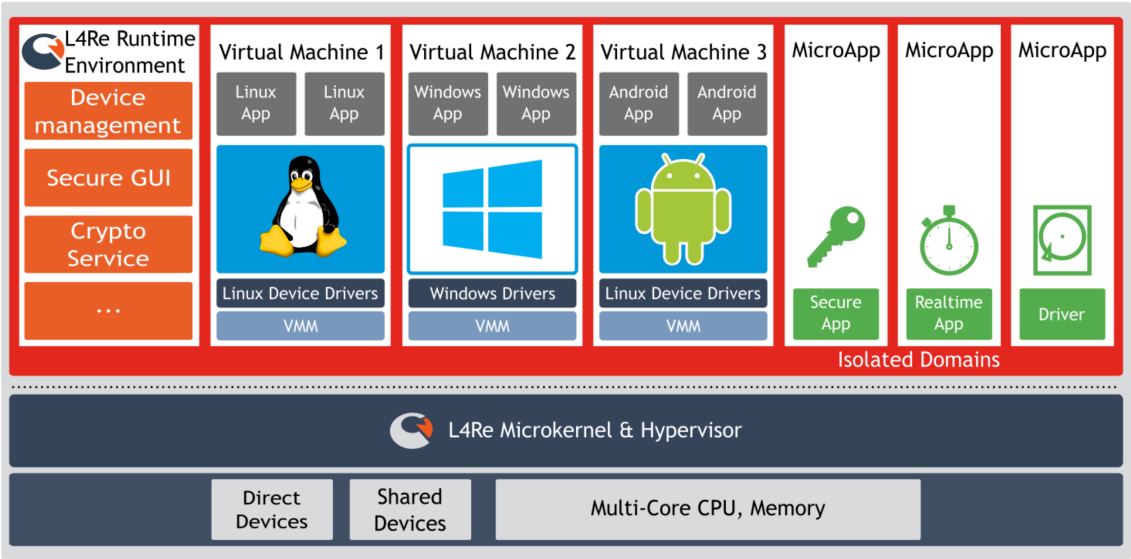


Figure 2.5: L4Re architecture [Lac19, p. 6]

The L4Re runtime environment runs entirely in user space, but by having the permission to use the kernel's interfaces it is able to control the system. Therefore, it is now a user space software with operating system functionality. Every other user space software that either has permissions to use kernel interfaces or the L4Re runtime environment automatically gains the ability to control the system. Without a way to handle which user space software is allowed to use these services the original idea of microkernels running as less code as possible in privileged mode gets pointless. L4Re uses capabilities to control access to system resources [L4R25c].

2.4.2 Capabilities

A capability is a security mechanism to handle access control [LW09, p. 25]. While a capability is often thought of as a key, this analogy does not fully capture its function [MYS03]. A capability not only provides the necessary permission to use a resource but also supplies the means, or pathway, for accessing it [Lev84, p. 4]. Furthermore, a crucial advantage is that a resource is invisible to any user or process unless they possess the

corresponding capability, which effectively reduces the attack surface of that resource [LW09, p. 25].

A more illustrative model can be imagined as an empty room with no doors, symbolizing the containment space for a process where it cannot break out. A capability, in this scenario, would be a portal that instantly appears in the room once the process is granted it. The unique aspect of capabilities is that the mere presence of this portal authorizes the process to access the resource. The portal thus represents both the pathway to the resource and the abstraction of its true location.

Capabilities are not typically used in common operating systems to handle internal access control [LW09, p. 25]. Instead, major operating systems use Access Control Lists (ACLs) or very similar concepts as their security concept [Nem10, pp. 159–162]. ACLs manage access control by explicitly saving the permissions that each user has to access a specific resource [MYS03].

The work by Miller et al. [MYS03] compares various capability models to each other and to Access Control Lists (ACLs). They conclude that object-based capabilities represent the strongest capability model, referring to them as the "true" capabilities model. They further note that all major capability systems implement object-based capabilities, and that literature defining capabilities consistently focuses on this object-based model. This comparative analysis also demonstrates the advantages of capabilities over ACLs.

Object capabilities have desirable properties that make them an interesting access control mechanism [LW09, p. 25]. Specifically, they are very well suited to implement the Principle of Least Authority (PoLA) [LW09, p. 25], which states that an application should only possess the minimum set of rights necessary to fulfill its designated purpose [MYS03]. Furthermore, capabilities are better at preventing the Confused Deputy Problem [MYS03]. This problem arises when a high-privilege application (the deputy) is tricked by a low-privilege application into misusing its power for a malicious purpose [Wu+15]. This problem can be mitigated if privileges are restricted to use for a specific purpose [MYS03]. This is precisely where capabilities become effective: since capabilities can be made fine-grained to serve only one specific purpose [VA11, p. 27], they naturally help developers build more secure systems by being able to enforce PoLA and mitigate the Confused Deputy Problem.

The L4Re microkernel also implements object-based capabilities [LW09, p. 25]. A capability in L4Re grants access to the object it points to and defines the specific permissions for operations on that object. Capabilities enable access to various kernel objects like tasks, threads, factories, IPC gates, IRQs, and DMA spaces [L4R25c].

When a L4Re Application is created, it is assigned an initial set of capabilities predefined by its creator. Each L4Re Application stores its capabilities in a local capability table. In a simplified view, a capability within this table is identified by a numerical index known as the capability selector. Because these tables are local to each L4Re Application, one L4Re Application cannot directly access the capabilities belonging to another. However, L4Re

Applications can share their capabilities with other tasks, provided they hold the necessary capability to communicate with the target task [L4R25c]. This structure allows object-based capability systems to model access control as explicit communication relationships [MYS03]. Given the ability to fine-grain access control at the object level, capabilities serve as an excellent security mechanism that strongly supports the microkernel's motivation of being secure and reliable [Roc04] [Lie96].

In the following sections, the microkernel and, specifically, the different components of the L4Re runtime environment will be explained in more detail.

2.4.3 L4Re microkernel

The L4Re microkernel is known by multiple names, including L4Re Hypervisor, L4Re Microhypervisor, Fiasco.OC, and Fiasco [Lac19, p. 7]. For consistency throughout this thesis, it will be referred to primarily as the L4Re microkernel, or alternatively by the short forms microkernel or Fiasco.

The functionality of the microkernel is grouped into three main categories: memory management, program execution, and IPC [L4R25c, pp. 5, 29].

The microkernel offers only basic functions for memory management, enabling the major part of memory management to be performed in user space [L4R25c, p. 29][Lie95, pp. 238–239]. User space memory management will be explained in more detail in subsection 2.4.4 and subsection 2.4.5.

The microkernel supports the execution of programs that run within tasks [L4R25c, p. 5]. A task can be conceptually viewed as a container or containment zone. It is technically implemented as a virtual address space, defined by a set of pages assigned to it and tracked in the task's page table. Additionally, a task possesses an object space, as discussed in subsection 2.4.2. Although a task itself does not execute anything, a thread runs within the task and executes the code. Furthermore, multiple threads can run concurrently within a single task [L4R25c, p. 4]. In short, the microkernel's role is simply to schedule different threads that operate within their respective virtual environments (tasks) [L4R25c, p. 4].

The microkernel implements IPC. In the L4Re microkernel, IPC not only establishes communication between tasks but is also used for resolving hardware exceptions, handling faults, and managing virtual memory [L4R25c, pp. 3–5]. Furthermore, the L4Re microkernel even replaces traditional system calls with IPC [L4R25c, p. 9]. This unified approach is highly beneficial because it provides a single, common mechanism for communication with the kernel, with L4Re runtime environment components, and with other threads.

Döbel states that in the L4Re operating system, everything is an object [Döb12a, 30:56]. Even concepts previously explained, such as a task and a thread, are implemented as kernel-level objects [L4R25c, p. 4]. Combined with object-based capabilities (see subsection 2.4.2),

one can see the importance of capabilities for the L4Re operating system, as they are used to grant and fine-tune access to anything. In subsection 2.4.2, it is described that capabilities abstract the true location of the object they point to; the L4Re developer documentation confirms this fact by stating that the target object can be located in either user space or kernel space [L4R25c, p. 3]. This abstraction of object location demands a mechanism that can communicate seamlessly across both the kernel and user space—a role fulfilled by IPC. This perfectly demonstrates how the capabilities and IPC mechanisms work in union.

Even though the microkernel provides a strong foundation for developing a flexible system, its raw interfaces are not necessarily suited for building an application directly on top of it. The L4Re runtime environment solves this problem by abstracting these low-level interfaces to provide a more suitable set of interfaces for application development [L4R25c, p. 5].

2.4.3.1 L4Re Applications

Applications build with the L4Re runtime environment are called L4Re Applications in this thesis. If a L4Re Application provides a service to other L4Re Applications, it is called a server, and a L4Re Application that uses a server's interface is a client. Importantly, L4Re Applications do not only describe user-developed programs; the term also encompasses virtual machines and device drivers [L4R25c, p. 5].

2.4.3.2 Micro-App

Kernkonzept's website (the company currently maintaining the L4Re open source project) defines a Micro-App as a native L4 application [Hoh22]. While the term "native" is not explained in further detail, based on Figure 2.5, one can assume that a Micro-App is a L4Re Application, as defined in the previous paragraph, that is neither a virtual machine nor part of the runtime environment. Put simply, a Micro-App is a user-developed L4Re Application.

2.4.4 Sigma0

As explained in subsection 2.4.3, the kernel provides only minimal functions to manage memory from user space. To enable this user-space control, the initial system task, Sigma0, receives a virtual memory space that is a 1-to-1 mapping of all physical memory, which also includes I/O memory addresses [Döb12b, 1:08:38-1:08:47]. Sigma0 is defined as the root pager [L4R25c, p. 5]. Since the kernel's memory management functionality allows any task to grant portions of its own memory to another task [L4R25c, p. 29], Sigma0 is fundamentally responsible for delegating initial address space to all other tasks. This establishes Sigma0 as the fundamental L4 resource manager [Döb12b, 58:37-58:41].

Tasks that manage memory are called pagers [L4R25c, p. 29]. A pager that received memory from Sigma0 can also grant it to other pagers. At some point, L4Re Applications get memory granted by pagers (see Figure 2.6). This system creates a tree structure where Sigma0 is the root [L4R25c, p. 29] [Döb12b, 1:10:15]; an example is depicted in Figure 2.6. To make this system possible, the kernel defers page faults to the pager that handles memory for the L4Re Application [Döb12b, 1:02:57-1:04:10].

A problem arises when a L4Re Application receives memory from multiple pagers, because the kernel cannot identify which pager it has to defer a page fault to. To solve this problem, Moe provides an abstraction called the region mapper [Döb12b, 1:11:10-1:12:09]. Sigma0 also serves as the main memory provider for Moe (see subsection 2.4.5). Additionally, Sigma0 grants the I/O server (see subsection 2.4.7) access to devices through I/O addresses. Typically, only Moe, the I/O server, and Ned (see subsection 2.4.6) receive a capability for Sigma0 [L4R25c, p. 53].

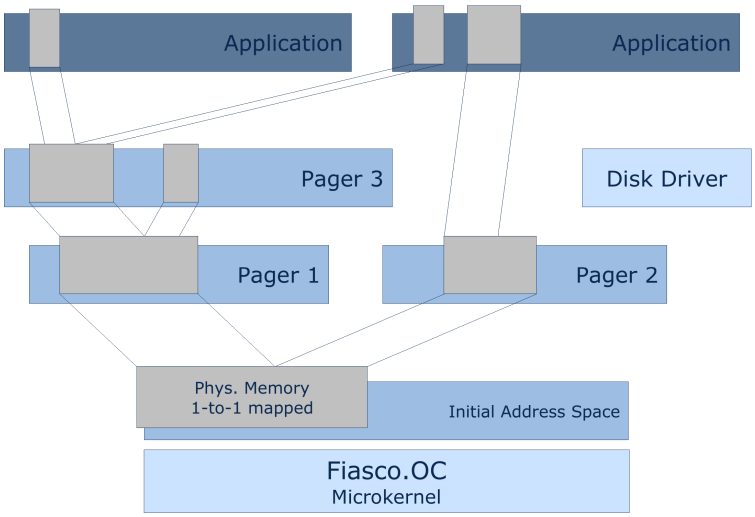


Figure 2.6: Paging Hierarchy in the L4Re [Döb12b, 1:10:57]

2.4.5 Moe

Moe is the L4Re resource manager [Döb12b, 58:46-58:50]. It offers several key services, including region mappers, memory allocators, and an initial program loader [L4R25c, p. 6].

As already mentioned in subsection 2.4.4, the region mapper solves the problem that the kernel does not know to which pager to refer a page fault if a L4Re Application received memory from multiple pagers (this problem is visualized by the L4Re Application on the right in Figure 2.6). The region mapper holds a table that defines which section of memory is handled by which pager. The kernel now always defers a page fault to the region mapper, and the region mapper contacts the pager that is responsible for the memory area that issued the page fault [Döb12b, 1:12:06-1:13:48]. Furthermore, a region mapper also serves as the default pager for an L4Re Application [L4R25c, p. 29].

The concept of pagers has been further abstracted by data spaces and data space managers [Döb12b, 1:14:12-1:16:08, 1:17:35-1:17:53]. A data space is a general abstraction for an area of memory [L4R25c, p. 29]. It serves as a standardized mechanism for exchanging memory regions between system components [Döb12b, 1:13:59]. A data space can represent various memory sources, such as the memory for a binary, a file from a disk or file server, or traditional memory areas like stack and heap [L4R25c, p. 29].

Through the data space abstraction, the region mapper's table now associates each address range with a data space manager, replacing the previous association with pagers. This data space manager will return the corresponding data space when a page fault occurs within that address range [Döb12b, 1:15:14-1:18:00]. The region mapper has the ability to attach data spaces to the address space of an L4Re Application [Döb12b, 1:15:51-1:17:18].

Concluding, the data space and region mapper abstractions create a unified method to map anything that behaves like memory—such as files, I/O memory, and traditional memory like stack and heap—to an L4Re Application. Heap and stack memory is provided by memory allocators with the help of data spaces [L4R25c, p. 29].

Additionally, Moe loads an init process that supports the further boot process. This init process gains access to Moe's resources and the Sigma0 interface [L4R25c, p. 51]. This process is called Ned and is explained in the next section.

2.4.6 Ned

The microkernel, Moe, and Sigma0 provide essential functions for creating L4Re Applications and facilitating communication between them, as well as security features like capabilities to control access. Ned relies on these services, the Developer Documentation explicitly states that Ned requires a loader like Moe that provides data spaces and region mappers [L4R25c, p. 57].

Ned is a configuration service [L4R25c, p. 57]. It provides an initial environment that contains capabilities from Moe and Sigma0 [L4R25c, pp. 51, 58], enabling it to construct the system based on a Lua script [L4R25c, p. 57]. Through this script, a user can configure Ned to create multiple L4Re Applications [L4R25c, p. 57], set their priorities, restrict them to a subset of processors [L4R25c, p. 61], assign them capabilities, and model their communication relationships with each other [L4R25c, pp. 57–58].

2.4.6.1 L4Re Application

At this point, it is useful to define the term L4Re Application more specifically. The Developer Documentation defines a L4Re Application as a program that uses services of the L4Re runtime environment [L4R25c, p. 5]. Additionally, it states:

> A [...] [L4Re] application needs 3 components to be booted beforehand: the L4Re microkernel, the root pager (Sigma0), and the root task (Moe). [...] Moe provides the essential services to normal [L4Re] user applications such as an initial program loader, a region-map service [region mapper] for virtual memory management, and a memory (data space) allocator [L4R25c, p. 6].

As the microkernel only schedules threads within different tasks (explained in subsection 2.4.3), this thesis will define the term L4Re Application more specifically as a task containing a region mapper and one or multiple threads that leverages the user environment interfaces.

This definition makes it clear why thread-to-thread communication can also be interpreted as communication between two L4Re Applications. IPC Gates are a special form of communication used between threads and, consequently, between L4Re Applications. They are commonly employed to allow access to a server's object interface. Without IPC Gates, the alternative to granting interface access would be to grant another thread access to the server's entire thread kernel object. This would be poor for security, as it would automatically grant access to all interfaces exposed by that task, thereby eliminating the ability to decide which specific interfaces the client should have access to. IPC Gates solve these problems [L4R25c, p. 18]. Because they enable L4Re Applications to communicate, IPC Gates are frequently configured within the Ned Lua script.

To summarize, Ned is a highly critical service, as misconfigurations of the system's capabilities or communication paths can prevent L4Re Applications from functioning correctly.

2.4.7 Io Server

All device management in the L4Re is performed in user space, which requires a central management server; this role is filled by the I/O server [L4R25c, p. 52].

The I/O server manages I/O addresses and interrupts, and grants access to these resources to other L4Re Applications, which are referred to as clients (e.g., device drivers) [L4R25c, p. 52]. Upon startup, the I/O server begins device auto-discovery if the system architecture supports it; otherwise, devices must be specified manually. On platforms like x86, auto-discovery includes scanning the PCI bus (for information about PCI, see subsection 2.1.6). Using this gathered information, Io populates a device tree of the physical hardware. This device tree contains all known devices whether auto-discovered or manually specified. Nodes in the tree either represent a device or a system bus, with any node having a child representing a system bus [L4R25c, pp. 62–63].

The I/O server supports granting only parts of the physical system bus to clients in the form of a Virtual Bus (vbus). The vbus is also structured as a device tree. This concept of the virtual bus is depicted in Figure 2.7 [L4R25c, pp. 62–63].

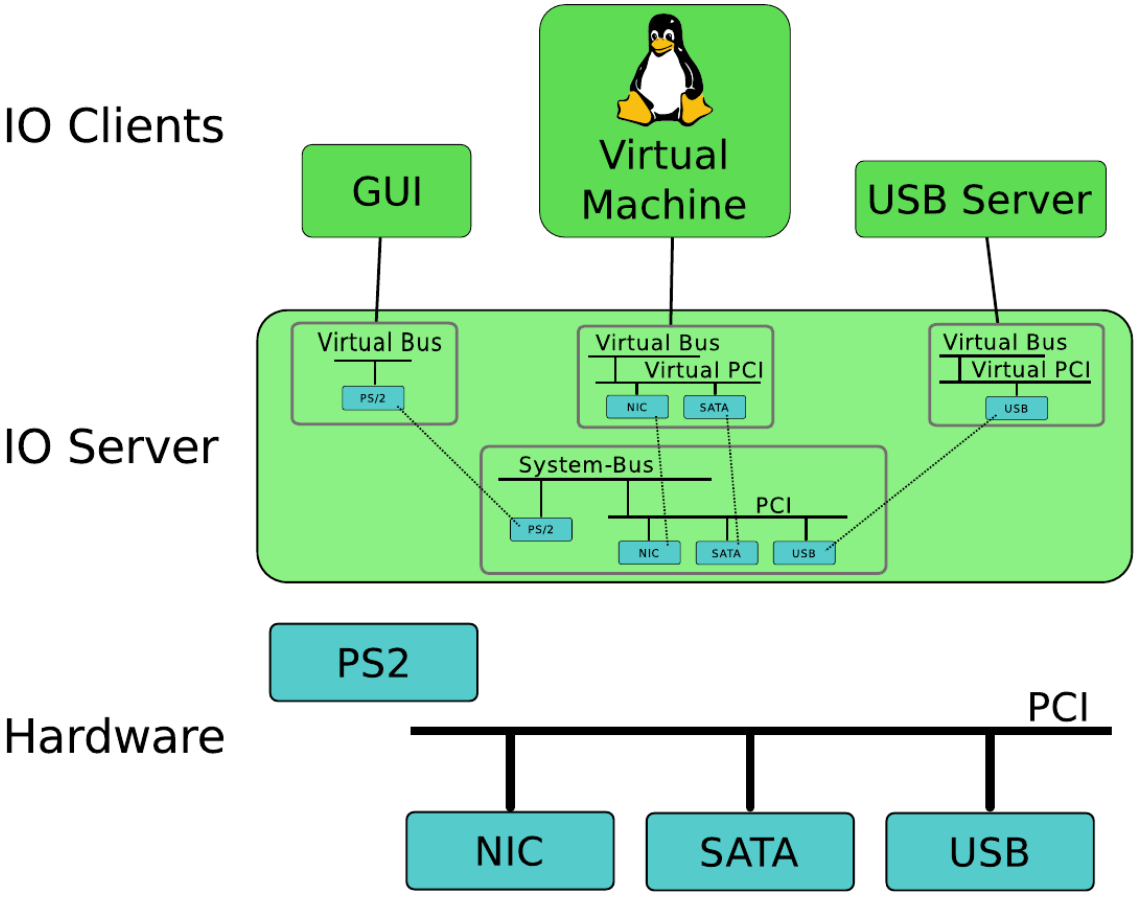


Figure 2.7: I/O Resource Management by the I/O server [L4R25c, p. 63]

Similar to the Ned configuration file, the I/O server requires a file to set up the corresponding vbus for each client. This file can also be used to manually specify devices that cannot be auto-discovered [L4R25c, p. 63].

2.4.8 Virtual Network Switch

To better understand how the Virtio network switch (or virtual network switch (VNS)) works, it is beneficial to first understand the underlying Virtio concept.

Virtio

Virtio is a standardized method for paravirtualizing hardware [Rus+15, p. 10] [Rus08, p. 95]. It was originally developed to provide a unified approach to virtualization because, at the time (2008), various virtualization systems from different vendors existed, each offering a distinct set of features and requiring its own vendor-specific driver [Rus08, p. 95].

Originally, Virtio was developed as a communication mechanism between the host and the virtual machine (also called the guest) [Rus+15, p. 10], but it has since developed into a universal communication format [Tsi18].

The Virtio virtualization architecture consists of three main parts: the Virtio driver, the Virtio device, and virtQueues and VirtRings for communication between them. The Virtio driver resides in the virtual machine, while the Virtio device is located on the host side. In this context, the communication between the Virtio device and the Virtio driver will be called the Virtio transport mechanism, which will not be discussed further here. Virtio supports different device types, such as block devices, console devices, and network devices. For each device class, a separate driver/virtual device pair exists. The Virtio driver provides a standardized interface for the guest's operating system to support the virtual device. Conversely, the Virtio device provides the necessary abstraction on the host side to virtualize the real hardware [Pal22]. The overhead of virtualization for Virtio is relatively low when configured correctly [KVM] [Rus08, pp. 100–101] [ARR24, p. 169].

Virtual Network Switch

The VNS is a virtualized switch. Just as a physical switch uses RJ45 cables for connections, the virtual switch allows any software component equipped with a Virtio driver to connect to a virtual port instead [L4R25c, p. 69]. Since the L4Re microkernel also functions as a hypervisor capable of running multiple VMs, this VNS can connect these multiple VMs to each other.

2.4.9 lwIP

The lwIP project is a lightweight implementation of the TCP/IP stack, originally developed by Dunkels and now maintained as an open-source project. Due to its small memory footprint, it is particularly suitable for embedded devices [Dun].

L4Re adapted lwIP by integrating a virtio interface [L4R25e] [L4R25f].

2.5 Linux Device Driver Interactions

In Linux, everything is a file, including devices [Emb23]. Applications in Linux can interact with devices via Linux Device Files [BC05, p. 541].

A program can use the *open()* system call, as it would for regular files, to open the Linux Device File [Ben05, pp. 539–540]. This established system call provides an easy way to switch from user space to kernel space. When these files are opened, the virtual file system recognizes, with the help of the file's metadata, that the content of the file does not actually exist and invokes the device drivers to read information from the associated hardware device [Ben05, pp. 539–540].

Linux device files can be sorted into two classes: character device files and block device files [BC05, p. 536]. "Network cards are a notable exception to this schema, because they are hardware devices that are not directly associated with device files." [BC05, p. 536]

The type of device also influences how the driver works. Therefore, understanding the porting challenge requires examining the specific hardware interaction mechanisms of NICs.

2.5.1 Core Concepts of a Network Interface Card

A NIC is the hardware component that establishes a computer's physical connection to the outside world across various physical media, such as fiber optic cables, radio waves, or electrical signals. Data is transmitted over these media using different protocols, like Ethernet or Wi-Fi. From a device driver's perspective, a NIC provides two primary interfaces for data transfer: the transmit queue and the receive queue [Her+19, 7:58-8:51].

In order to send a network package over the network, the host system must create a transmit descriptor that holds the address of the package in host memory and the length of the package. Then, it puts this transmit descriptor into the NIC's transmit queue and notifies the NIC about the new data. The NIC will then copy the data into its own local memory with the help of a DMA and will then send the package [Her+19, 9:14-10:17].

When the NIC receives a package, it loads the package into its own memory to process it. Then it looks for receive descriptors in the receive queue. If the NIC finds a receive descriptor, it copies the package to a location in the receive descriptor. After the transfer is completed, the NIC notifies the host system with the help of an interrupt that there is new data to process [Her+19, 10:21-11:28].

2.5.2 Linux Ethernet Device Drivers

A network interface is the abstract endpoint of the kernel, allowing applications to communicate with a network [CRK05, p. 497]. User-level applications do not communicate with the network interface directly; instead, they communicate via sockets that are associated with network interfaces [CRK05, p. 497]. A network interface is created when the network device driver is loaded into the kernel [CRK05, p. 502]. When the network device driver is built into the kernel, it is initialized at boot time. If the network device driver is loaded as a module, it is initialized at runtime [Ben05, p. 137].

It registers the interface as a special data structure in the kernel [CRK05, p. 502]. After registering the interface with the kernel, it is seen as ready to send. At that point, the network device driver must have initialized everything [CRK05, p. 503].

During initialization, a network device driver must acquire necessary resources, such as interrupts [Ben05, p. 86][CRK05, pp. 502–503]. The initialization tasks performed by a

network device driver typically include mapping I/O addresses, registering an Interrupt Request (IRQ) handler, allocating Direct Memory Access (DMA) buffers, and performing a full reset and re-initialization of the hardware configuration [Boo09, pp. 1, 4, 5, 11, 13].

The driver is also responsible for administrative operations such as setting addresses, adjusting transmission parameters, and maintaining statistics for traffic and errors [CRK05, p. 497]. However, the device driver usually does not have to support higher-level network functions like IP numbering, network protocols, or ARP [CRK05, pp. 498, 502]. An Ethernet device driver must only concern itself with the creation of the lower-level header, the Ethernet header, but the content of the package is irrelevant to the device driver [CRK05, p. 498]. The main task of an Ethernet device driver is to enable the host system to transmit and receive Ethernet packages.

A device driver's implementation should be decoupled from the specifics of the rest of the system's hardware [CRK05, p. 444] [BC05, p. 548]. This enables the device driver to work on PCs with different hardware configurations. Therefore, it has to use operating system functions that abstract from the hardware of the rest of the system [BC05, p. 546]. This makes the device driver operating-system-specific.

2.6 OSI Model Context

The OSI (Open Systems Interconnection) reference model, developed by ISO, defines a seven-layer framework for network interconnection [Hua23, p. 15]. The reader is assumed to have prior knowledge of this model. For the scope of this thesis, the Data Link Layer (Layer 2) and the Physical Layer (Layer 1) are particularly important.

- The Physical Layer (Layer 1): The NICs supported by the igc device driver use copper wire as medium [Int24]. The physical details of how bits are encoded and transmitted over this wire are handled by the NIC. The primary challenge addressed by the device driver is to configure the NIC to manage these physical encodings correctly.

- The Data Link Layer (Layer 2): This layer defines the structure of the message packets (frames) on the physical medium [EC24, p. 14], which in this case is Ethernet. The device driver operates at this layer, preparing data for transmission and processing received frames.

2.7 Device Driver Porting Frameworks

2.7.1 Porting framework for Linux Network Device Drivers to the L3 microkernel

The work by Stange was originally an investigation into how to generically port device drivers to the L3 microkernel. The thesis proposed focusing on Linux device drivers because they were openly available and numerous. However, during the research, the author recognized that different device classes are integrated into the Linux kernel differently [Sta96, p. 14]. Due to time constraints, the scope of the thesis was limited to porting only NIC device drivers to L3 [Sta96, pp. 4, 5], specifically focusing on device drivers for Ethernet NICs [Sta96, p. 14]. The result was an L3 driver capable of running the unchanged Linux driver code of different Linux Ethernet device drivers [Sta96, p. 14].

The phrasing "unchanged Linux device driver code" can be misleading. While not explicitly stated, "unchanged" in this context means that the original logic of the device driver was unmodified, although the source code itself was adjusted. Stange described how the original Linux Device Code was used, but unnecessary code was removed because only a subset of functions could be emulated. The author noted that deciding what code was unnecessary was difficult. Furthermore, leaving unnecessary functions in place carried a significant risk: if the driver executed code expecting a hardware function without the necessary underlying emulation support, the resulting error might not be detected by the compiler [Sta96, p. 54].

2.7.2 Porting framework for Linux Drivers to the DROPS operating system

In its earlier days, the L4Re operating system was known under the project name DROPS (Dresden Real-time Operating System) [Lac19, p. 4]. As part of his work, Helmuth investigated how to generically port Linux device drivers to the DROPS architecture [Hel01].

To achieve this, Helmuth created a Device Driver Environment (DDE) designed to run Linux device driver code [Hel01, p. 17]. Similar to the framework explained in subsection 2.7.1, the term "unmodified" is not explicitly detailed. However, given that the DDE is strictly limited to emulating only essential functions, one can assume the same definition for "unmodified" holds true as in subsection 2.7.1. The DDE must be limited in this way because emulating the entire Linux environment would be unfeasible [Hel01, pp. 17, 18]. As illustrated in Figure 2.8, the DDE is split into a common part and a driver-specific part [Hel01, p. 27]. The common part manages generic services like memory management, synchronization, and scheduling, which are generally required by device drivers. The driver-specific part provides an interface tailored to a specific hardware type, such as sound cards, since device drivers for similar hardware have common requirements [Hel01, pp. 20–21, 27].

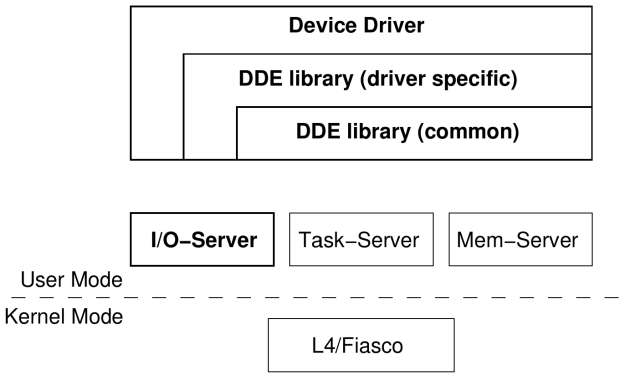


Figure 2.8: Final DDE architecture [Hel01, p. 27]

The implementation efforts concentrated only on providing the driver-specific interface for sound cards [Hel01, pp. 25, 37]. The DDE architecture was designed to be generic, but the framework only supported sound card device drivers when it was written. Unfortunately, even this sound card implementation could not be finished. As a result, the performance of the framework could not be tested or evaluated [Hel01, pp. 25, 37].

2.7.3 Genode & DDE

Genode is an Operating System Framework that focuses on security and robustness, with the goal of becoming a general-purpose operating system [Fes11]. It shares its origin with the L4Re Operating System Framework at TU Dresden. The Genode Project itself began in 2006 at TU Dresden under the name Bastei [Fes11]. In 2008, Christian Helmuth and Norman Feske left the Operating Systems Chair at TU Dresden to found their company, Genode Labs, to further develop the Genode Project [Gen25][Fes11].

The Genode Operating System Framework functions as a toolkit to build secure operating systems. It offers a collection of reusable building blocks with which a system can be constructed. These building blocks can be either applications or operating system functions [Fes25, p. 14]. Examples of these include device driver, a GUI stack, a TCP/IP stack, and applications conforming to POSIX interfaces, such as `bash` [Fes25, p. 14]. These components run on top of a microkernel [Fes25, pp. 12–13]. Genode supports several kernels of the L4 family, including Fiasco.OC [Fes25, p. 14]. Genode Labs also develops its own microkernel, NOVA [Fes25, p. 14], which also originated as a project at TU Dresden [Fes11].

Genode structures its processes in a hierarchical tree structure, where the microkernel is the root [Fes25, pp. 12–13]. A parent process can create a child process and owns it. Crucially, all processes run in an isolated environment. A child process can, in turn, spawn its own child processes, thereby expanding the tree structure. An example of this architecture is depicted in Figure 2.9 [Gen25]. The Genode Foundations Document defines the meaning of ownership as follows:

> [...] the parent has to explicitly provide the resources needed by its children out of its own resources. It is also responsible to acquaint children with one another and the outside world. In return, the parent retains ultimate control over each of its children. As the owner of a child, it has ultimate power over the child's environment, the child's view of the system, and the lifetime of the child [Fes25, pp. 12–13].

With this system, Genode aims to keep the Trusting Computing Base (TCB) as small as possible. The TCB is defined as the amount of code that must be assumed to be correct and cannot be compromised for the system to remain secure [Gen25].

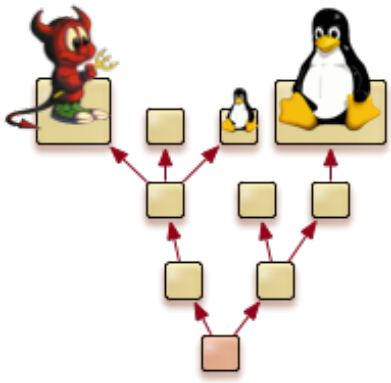


Figure 2.9: Simplified Example of Genode Architecture [Gen25]

The Genode website states that to be a useful operating system, it must support compatibility with existing applications. Therefore, Genode supports running VMs. Even more significantly, Genode supports Linux device drivers by executing them within the Linux Device Driver Environment (DDE). The small penguin symbol in Figure 2.9 symbolizes a Linux device driver running inside a Genode process [Gen25].

The Genode DDE is still actively maintained today. The Genode Release notes from August 2025 state that the Linux DDE enables any supported microkernel to run Linux device drivers as sandboxed components. Additionally, they noted that the device drivers for the x86 architecture have been updated to Linux kernel version 6.12 [Gen25][Fes25, p. 23].

The DDE has undergone a long and evolving development process. Its origin traces back to the *dde_kit* introduced in 2008. The *dde_kit* provided a C API for device drivers, serving as a translation layer between Genode's C++ API and a C-based interface. According to the Genode release notes from November 2008, the *dde_kit* was the result of:

> long-year experiences with porting device drivers from Linux and FreeBSD to custom OS environments. The following references are the most significant contributions to the development of the API. Christian Helmuth [one of the founders of Genode Labs] created the initial version of the Linux device-driver environment for L4. He describes his effort of reusing unmodified sound drivers

on the L4 platform in his thesis "Generische Portierung von Linux-Gerätetreibern auf die DROPS-Architektur" [described in section subsection 2.7.2]. Gerd Griessbach approached the problem of re-using Linux USB drivers by following the DDE approach in his diploma thesis "USB for DROPS". Marek Menzer adapted Linux DDE to Linux 2.6 and explored the DDE approach for block-device drivers in his student research project "Portierung des DROPS Device Driver Environment" and his diploma thesis "Entwicklung eines Blockgeräte-Frameworks für DROPS". Thomas Friebel generalized the DDE approach and introduced the DDE kit API to enable the re-use of device driver from other platforms than Linux. In particular, he experimented with the block-device drivers of FreeBSD in his diploma thesis "Übertragung des Device-Driver-Environment-Ansatzes auf Subsysteme des BSD-Betriebssystemkerns". Dirk Vogt successfully re-approached the port of USB device drivers from the Linux kernel to L4 in his student research project "USB for the L4 Environment" [Gen25].

The November 2008 release notes also describe a use case where the *dde_kit* could serve as a backend for the Linux DDE, illustrated in Figure 2.8. However, the notes only reference the Linux DDE as part of this example and do not explicitly clarify whether the Linux DDE was integrated into the *dde_kit* or if it was a separate component.

Despite this ambiguity, it is also interesting to determine if this DDE is the same as the one described in subsection 2.7.2. The Genode DDE architecture provides a clue, as it highlights the need for a *device API*—a software layer abstracting the device class—which matches the *DDE Library (driver specific)* layer described in subsection 2.7.2 and shown in Figure 2.8. This functional similarity suggests that the Linux DDE at that time was very similar, or even identical, to the framework described in subsection 2.7.2. This assumption is further supported by the fact that the author of the framework described in subsection 2.7.2 later became one of the founders of Genode Labs [Gen25].

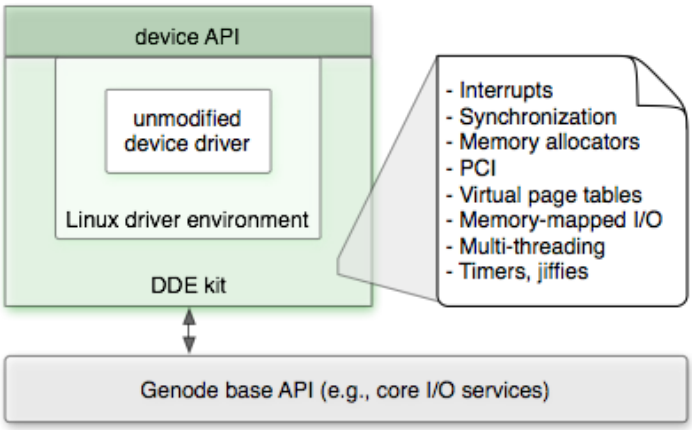


Figure 2.10: dde_kit Architecture [Gen25]

The Genode release notes from May 2016 clearly state that the *dde_kit* approach was discontinued. A Genode developer explained in a blog post that the *dde_kit*, together with a Linux-specific library, formed the foundation for all Linux device drivers. This Linux-specific library was built for a single, fixed Linux kernel version. Moreover, providing a library for a fixed Linux kernel version is not feasible given the variety of kernel versions in use. Over time, the Linux-specific library grew more complex, since sophisticated device drivers required additional code not shared with simpler ones. To address these limitations, Genode began developing dedicated DDEs for each device driver. However, this led to significant code duplication. Consequently, some functionality was centralized into the *lx_kit* (Linux kit), which is organized into multiple layers that are not discussed here in detail [Gen25].

According to the release notes from August 2021, Genode introduced an improved version of the DDE, which exists alongside the legacy implementation. The motivation for this redesign was that porting and updating device drivers to newer kernel versions using the legacy DDE required tremendous effort. For example, porting the i.MX8 graphics driver reportedly took two months. A Genode developer noted in a blog post that updating a driver to a new kernel version required nearly the same effort as porting it originally. To address this, the improved DDE was structured into reusable components, allowing custom runtime environments to be created more easily. In addition, Genode provided tools to automate repetitive tasks involved in driver porting [Gen25].

In summary, the Genode DDE is particularly interesting because it allows Linux device drivers to be executed within the framework. Furthermore, Genode itself can run on multiple microkernels, including Fiasco.OC, the microkernel underlying L4Re.

2.7.4 Ixl

Ixl is a collection of user space network device drivers for the L4Re. It is based on the open source project ixy [L4R25a].

The ixy project implements a concise packet framework written in C. Its architecture is characterized by a single layer of abstraction designed to interface with the NIC driver. A application interacts with the ixy framework by supplying the address of the desired network interface. In return, the application receives a driver object—technically a pointer to a struct containing function pointers—that represents the corresponding NIC driver [Emm+18, pp. 91–92]. Ixy currently supports the ixgbe and virtio-net device drivers [Emm+18, p. 91] [Emm25].

The ixl framework is architecturally derived from the ixy project, specifically reusing the ixgbe driver code. A key distinction is that ixl is implemented in C++, unlike its C predecessor. ixl provides several specific functionalities, including dedicated support for DMA memory management, interrupt setup, and PCI. While ixl facilitates the core task

of transmitting and receiving network packets, it does not support all more advanced networking features [L4R25a]. The framework extends its compatibility by additionally integrating the e1000 and igb device drivers [L4R25a]. Notably, all currently supported device drivers—ixgbe, e1000, and igb—are designed for Intel NICs [Int25]. The igc device driver, which is planned to be ported, also targets Intel NICs.

The documented curl example on the l4re.org website showcases the full network data path: a Micro-App (curl in this instance) accesses the Internet through the virtio switch, which in turn connects to the ixl framework, which then finally accesses the physical network. This chain of connectivity demonstrates that ixl successfully interfaces with the VNS environment [L4R25d].

Chapter 3

Porting Framework Evaluation

The primary goal of this chapter is to define a robust set of comparison criteria to evaluate the frameworks identified in the current state of research. This evaluation aims to determine which framework provides the best fit for porting the igc device driver to the L4Re operating system.

3.1 Comparison Criteria

As the goal of the thesis is to port the igc device driver to the L4Re operating system, compatibility with the L4Re operating system will be a key criterion.

Furthermore, while reviewing the frameworks as part of the current state of research, it became apparent that the effort to integrate a device driver and to maintain the combination of framework and device driver can be challenging. Therefore, maintainability and integration effort will serve as additional comparison criteria.

As it is not feasible to try out every framework, the assessment relies on a comprehensive static and architectural analysis derived from source code review and available documentation.

3.1.1 Compatibility

This comparison criterion evaluates the framework's fitness for integration with the core L4Re operating system components, specifically focusing on its primary resource management interface. These components are qualitatively ranked based on the level of integration complexity and convenience: the most convenient and highest-level interface, the I/O server, is categorized as high compatibility. The Moe interface holds an intermediate position, categorized as moderate compatibility. The fundamental interfaces of the Fiasco microkernel and its associated memory manager, Sigma0, represent the least convenient

and most complex integration method, categorized as low compatibility. Since higher-level interfaces abstract the underlying ones, a framework is considered exclusively compatible with the highest-ranking interface it primarily uses for resource management.

Table 3.1: Compatibility Overview

Criterion	Interface	Compatibility
C1.1	I/O server interfaces	high
C1.2	Moe interfaces	moderate
C1.3	Fiasco / Sigma0 interfaces	low

3.1.2 Integration Effort

This criterion evaluates the predicted effort required for the initial deployment of the framework, primarily assessed through the steps necessary for integration into the L4Re Operating System (C2.1) and for porting an existing driver (C2.2). The integration effort for both processes will be assessed qualitatively as low, moderate, or high.

C2.1 Communication Glue Code Overhead: Evaluation of the custom development work (glue code) required to enable an L4Re Application to utilize the driver ported via the framework. Frameworks requiring minimal or no glue code are preferred. A low integration effort is assigned if the required glue code is non-existent or negligible.

C2.2 Driver Porting Difficulty: Description of the necessary steps to integrate an existing device driver with the framework. This description is followed by a qualitative assessment of the overall porting difficulty. A framework that facilitates easy integration of an existing driver will be assessed as low integration effort.

3.1.3 Maintainability

This criterion evaluates the long-term cost and effort required to sustain the system. Maintainability is assessed by examining the internal architecture of the framework, including the number of abstraction layers, through code review, and evaluation of documentation.

C3 Number of Abstraction Layers: This metric measures the number of abstraction layers that the framework introduces between the L4Re operating system and the driver code (e.g., igc device driver). While a smaller layer count generally correlates with lower maintenance effort, the final assessment of maintainability will be a qualitative judgment (categorized as low, moderate, or high). This judgment is derived from the layer count, but is moderated by an evaluation of the overall effect of the layers in conjunction on the maintenance burden.

3.2 Framework Evaluation

3.2.1 Porting framework for Linux Network Device Drivers to the L3 microkernel

The evaluation of the C1 Compatibility criteria was challenging due to historical changes in the underlying operating system. The work being assessed originated on the L3 microkernel, which subsequently underwent a radical redesign to become L4 [HE16, p. 2], suggesting potential architectural incompatibility with the current system target. While one source indicates that a port of this specific driver to the L4 kernel was achieved [Hel01, p. 15], the required source code for neither the original L3 nor the subsequent L4 implementation is publicly available. With no source code available, both static analysis and the practical attempt to port the driver are impossible. Consequently, a further comparison would not be useful. However, the foundational methodology and design principles described in the original work remain valuable.

3.2.2 Porting framework for Linux Drivers to the DROPS operating system

The source code for this framework is also no longer publicly available, with the primary reference pointing to an outdated source [TU 05]. Consequently, direct analysis and comparison against the C-series criteria are not feasible. With the code missing, neither static analysis nor a practical porting attempt can be executed. Nevertheless, this work's design experience was directly influential on the architectural choices of the Genode DDE (see subsection 2.7.3), making its methodology foundational to the Genode framework being evaluated.

3.2.3 Genode & DDE

3.2.3.1 Evaluating Compatibility to L4Re for Genode & DDE (C1)

The evaluation of criterion C1 Compatibility for the Genode framework, as detailed in subsection 3.1.1, is presented in this paragraph.

Central to this evaluation is the internal software architecture of Genode, which is designed for portability across multiple microkernels. This flexibility is achieved through Genode Core, an abstraction layer that unifies the different microkernel interfaces. Genode Base then builds upon this foundation to provide the standardized API used by Genode processes [FHB25].

As previously noted in subsection 2.7.3, Genode maintains compatibility with the Fiasco microkernel. Within the Fiasco-specific portion of Genode Core, the code contains direct

references to Sigma0. This shows that Genode is compatible with Sigma0. However, the complete absence of references to Moe serves as proof that Genode is currently incompatible with Moe.

Furthermore, a comment in the file *genode/repos/base-fiasco/src/core/include/map_local.h* (last modification 2 years ago) and a release note from August 2009 state that Genode requests all RAM in a one-to-one fashion from Sigma0 [Gen25][FHB25]. Critically, this mechanism of requesting all memory from Sigma0 creates a major technical challenge that will be addressed in the integration effort criterion (see subsubsection 3.2.3.2).

Since Genode is not compatible with Moe, which serves as the mandatory foundation for the L4Re runtime environment, it is by extension also not compatible with the I/O server.

Consequently, the findings establish Fiasco and Sigma0 as the highest supported interfaces of Genode with respect to the C1 criterion. Therefore, Genode is assessed as having low compatibility with the C1 criterion.

3.2.3.2 Evaluating the Integration of Genode & DDE in L4Re (C2)

The evaluation of sub-criterion C2.1 (Communication Glue Code Overhead) for the Genode framework reveals that a high amount of glue code and custom modifications would be necessary to achieve functionality with L4Re. As already mentioned in subsubsection 3.2.3.1, Genode requests all memory from Sigma0 in a one-to-one fashion. This design means no memory would be left for Moe or any L4Re service or L4Re Application to run. Custom modification is required simply to adjust Genode's Resource Management, transitioning from requesting access to all memory to only receiving a managed portion. Furthermore, custom integration is necessary to specifically grant Genode access to the I/O memory addresses of the device intended to run within the Genode DDE. The consequences of operating Genode with fragmented memory are currently unforeseeable and could cascade into unexpected areas, resulting in an unstable system.

Additionally, substantial glue code is required to establish communication between L4Re Application and the DDE running in the Genode process. Since Genode uses an abstraction layer over IPC because IPC is implemented differently by various microkernels [Gen25], this requires a custom implementation either to bypass that layer for task-based communication or to integrate a new mechanism into the abstraction layer specifically for L4Re Application to communicate with Genode processes. As two major custom integrations (Resource Management and Communication) are necessary for this sub-criterion, the Genode framework is assessed as requiring high integration effort.

Evaluating the sub-criterion C2.2 (Driver Porting Difficulty) for the Genode framework reveals the specific challenges involved in integrating a new device driver. Integrating a new device driver into Genode requires building a custom DDE for that specific device driver. To simplify this process, Genode provides commonly needed code in the form of modular

components. In the case of porting a device driver for a NIC that is connected through PCIe/PCI, like the igc device driver, the existing code base already shows support for the NIC component and the PCI component [FHB25].

Despite this existing support, it would still be necessary to identify which device driver-specific functions are not covered by the provided modular components and then implement a translation layer for those uncovered functions. Implementing this translation layer demands a deep understanding of the Genode framework—not just what the device function is intended to do, but precisely how its action can be realized within Genode's operating system environment.

Since code to emulate NIC device driver functionality already exists, but the remaining uncovered functions require a highly specialized understanding of the Genode operating system framework, the overall porting effort for this sub-criterion is assessed as moderate.

3.2.3.3 Evaluating Maintainability of Genode & DDE within L4Re (C3)

This section evaluates the C3 Maintainability Criterion (see subsection 3.1.3) for the Genode framework by analyzing its software architecture.

The Genode framework, coupled with its DDE, introduces several distinct abstraction layers between the underlying microkernel and the ported Linux driver. The layers, listed from the lowest abstraction level to the highest, are:

Layer 1: Fiasco & Sigma0

Layer 2: Genode Core. This fundamental layer provides the abstraction for different kernels, including Fiasco, and handles low-level functions (e.g., Fiasco & Sigma0) [FHB25].

Layer 3: Genode Base Libraries. Built on the Core, these libraries provide the necessary functions for internal Genode processes to communicate with each other [FHB25].

Device Driver Environment (DDE). This is the primary abstraction stack containing the ported driver. It is further segmented [FHB25][Gen25]:

Layer 4: lx_kit: Provides C++ functionality to interact with Genode's hardware-related functions.

Layer 5: lx_emul: Wraps the C++ functions from *lx_kit* in C to create the runtime environment expected by the Linux kernel code.

Layer 6: Linux Device Driver Code

Layer 7: lx_user: Serves as connection layer between Linux kernel interfaces and the Genode service interfaces [Fes23, p. 150]

Layer 8: genode_c_api: Works with *lx_user* to provide the final interface to other Genode processes, converting back from C to C++.

Layer 9: Genode process

Additionally, the custom glue code required for integration (as mentioned in subsubsection 3.2.3.2) must be maintained alongside these existing layers. The glue code necessary to adjust memory management would be integrated into the Genode Core layer. The separate glue code to establish communication between the DDE and L4Re Application could be located in layer 8 or 9, or between them. In case of modifying the Genode IPC abstraction layer, it would require a modification to layer 2 to 3, or just bypassing every layer and directly using the IPC functionality of Fiasco in any layer.

The combination of the numerous abstraction layers, the essential need to maintain multiple pieces of complex custom glue code, and the additional requirement to deeply understand an entirely separate operating system framework (Genode) in addition to L4Re, makes this system exceptionally hard to maintain. Therefore, the Genode framework is assessed as requiring high maintainability effort.

3.2.4 Ixl

3.2.4.1 Evaluating Compatibility to L4Re for Ixl (C1)

This section evaluates the C1 Compatibility Criterion for the ixl framework (for definition details, see subsection 3.1.1).

Evidence from the L4Re project website, specifically the curl example demonstrating how a L4Re Application can access the internet, shows that the ixl framework operates as a separate software module. It can be compiled and integrated as part of the VNS when a compile flag is enabled [L4R25f].

The Ned configuration file used in this example illustrates the key inter-component connections [L4R25f]:

- The configuration instantiates three primary components: the VNS (which incorporates ixl), the I/O server, and the curl L4Re Application.
- The I/O server manages the physical Ethernet NIC and exposes it via a vbus capability.
- The VNS receives a capability to access the vbus (containing the Ethernet NIC) from the I/O server.
- The curl L4Re Application receives a capability for a port on the VNS.

This resource delegation chain confirms that the ixl framework operates using capabilities provided by the I/O server. The I/O server represents the highest-level interface defined in the C1 Criterion. Therefore, the compatibility of the ixl framework is assessed as high.

3.2.4.2 Evaluating the Integration of Ixl in L4Re (C2)

Based on the architecture described in subsubsection 3.2.4.1, it is apparent that no custom glue code is required for sub-criterion C2.1 (Communication Glue Code Overhead). Instead, communication is achieved through the existing VNS, which connects the L4Re Application to the ixl framework. Since the VNS is a standard, pre-existing component of the L4Re environment, no custom development or bridging is necessary. Therefore, the integration effort for this sub-criterion is assessed as low.

Evaluating sub-criterion C2.2 (Driver Porting Effort), the ixl framework differs in its methodology for device driver implementation when compared to other porting frameworks: it does not involve porting an existing device driver but rather requires rewriting the device driver within the framework's architecture. Although the source code for the Linux device driver already exists, the task is to essentially rebuild the device driver using ixl.

To facilitate this, the ixl framework provides an abstract C++ class that serves as a necessary structure for developing the new device driver [L4R25a]. Additionally, as mentioned in subsection 2.7.4, ixl provides convenient abstraction layers for writing device drivers, including dedicated support for DMA memory management, interrupt setup, and PCI.

The effort required is inherently dependent on the complexity of the specific device driver being rewritten. However, a comparison of the ixl network device drivers to their corresponding Linux counterparts shows that only approximately one-tenth of the lines of code (LOC) are necessary. This significant reduction in complexity is substantiated by Table 3.2, which compares the LOC for every device driver implemented in ixl against its Linux counter-part.

Table 3.2: LOC comparison for network drivers between ixl and Linux [L4R25a][Tor25]

Device Driver	**Linux Kernel LOC**	**ixl Framework LOC**
e1000	17089	1169
igb	29092	1265
ixgbe	54477	5082
igc	18506	?

For fairness, it must be noted that the ixl device drivers currently only support basic functionality for sending and receiving packets and may lack some advanced features found in the full Linux device drivers [L4R25a].

Based on the provision of an abstract virtual class for structure, the availability of convenient abstraction layers, and the clear evidence that rewriting a device driver with ixl requires

only a fraction (approximately one-tenth) of the code compared to the Linux device driver, the overall effort for this sub-criterion is assessed as low.

3.2.4.3 Evaluating Maintainability of Ixl for L4Re (C3)

This section evaluates the C3 Maintainability criterion for the ixl framework. The combination of the ixl framework and the VNS introduces two additional layers to the core L4Re operating system environment. The complete stack, listed from the lowest abstraction layer to the highest, includes:

Layer 1: Fiasco & Sigma0.

Layer 2: Moe.

Layer 3: Ned. The configuration service that sets up all L4Re Application and defines the necessary capabilities between them.

Layer 4: I/O server. Provides the physical NIC and exposes its resources to the VNS in the form of a vbus.

Layer 5: Virtual network switch. This component connects L4Re Application to one another and links them to the ixl framework.

Layer 6: Ixl Framework. Connected to one port of the VNS. Communication over this port allows access to the physical network attached to the NIC.

Layer 7: L4Re Application client. The client application that requires communication with the physical network. This L4Re Application is also connected to a port of the VNS.

Viewing the entire system reveals seven distinct layers of abstraction that must be considered during maintenance. This overall architecture involves numerous upstream dependencies. Changes in a lower-level component of the L4Re system (Layers 1-4) could potentially cascade and require updates in the layers above. However, the components responsible for the core networking logic—Layer 5 (the VNS), which is 4,203 LOC, and Layer 6 (the ixl framework base), which is only 2,321 LOC when deducting all implemented drivers—are very small in code size. While the entire dependency chain introduces maintenance risk, the minimal size of these two key layers means that adjusting them in response to upstream changes should remain manageable. Therefore, the maintainability for ixl is assessed as moderate.

3.3 Framework Selection

Table 3.3 summarizes the assessment results for the core criteria (C1, C2, C3) applied to the evaluated frameworks: Genode and ixl. As demonstrated by the scores, the ixl framework is superior to Genode in every category. Therefore, the ixl framework will be selected as the basis for porting the igc device driver to the L4Re operating system.

Table 3.3: Summary of Evaluation Assessments for Genode and ixl Frameworks

Criterion	**Sub-Criterion**	**Genode**	**ixl**
C1	Compatibility (Interface Level)	low	high
C2	C2.1 Communication Glue Code Overhead	high	low
	C2.2 Driver Porting Effort	moderate	low
C3	Maintainability	high	moderate

Chapter 4

Concept

This chapter defines the goals for the igc device driver integration in the form of functional requirements. These requirements will be used in chapter 7 to evaluate the success of the driver implementation. Furthermore, the tests used to evaluate whether these requirements are met are also defined at the end of this chapter. The rest of this chapter will describe the system used to realize these functional requirements.

4.1 Functional Requirements

The goal of this thesis is to port the Intel igc device driver to the L4Re operating system and enable multiple L4Re Application to use it. Igc is the name of Intel's open-source device driver that is capable of controlling the following NICs: I225-LM, I225-V, I225-IT, I226-LM, I226-V, and I226-IT [Int25]. Its source code can be found in the Linux Kernel [Int25]. The plan is that the Linux igc device driver code serves as the basis to implement a corresponding device driver within the ixl framework. The implementation will be tested on the Ecrin platform, which uses the Ethernet NIC I225-IT [ecr25]. The goals of the implementation are described in more detail in the following requirements:

R1 Core Network Functionality: The ixl device driver must support the minimal necessary initialization, configuration, and operation of the Intel® Ethernet Controller I225-IT on the Ecrin to enable reliable Ethernet frame transfer (send and receive).

R2 L4Re Application API: The system (the combination of ixl and VNS) must provide a defined interface accessible by L4Re Applications (e.g., Micro-Apps and VMs) to allow them to utilize the network device driver functionality.

R3 Multi-Client Access: The system must support the use of the network device driver by multiple independent L4Re client applications.

4.2 Architecture

4.2.1 Hardware Architecture

The architecture of the test system and the specific Intel I225-IT NIC used is first described, providing the hardware context and outlining the NIC's features that enable Requirement R1. As visualized in Figure 4.1, the Ecrin features two distinct Intel Ethernet NICs—the I225-IT and the I210-IT—both connected to the central CPU via PCIe Generation 3. The Intel igc device driver is designed to communicate with this specific hardware. The general concepts of a NIC are assumed to be known; otherwise, refer to subsection 2.5.1.

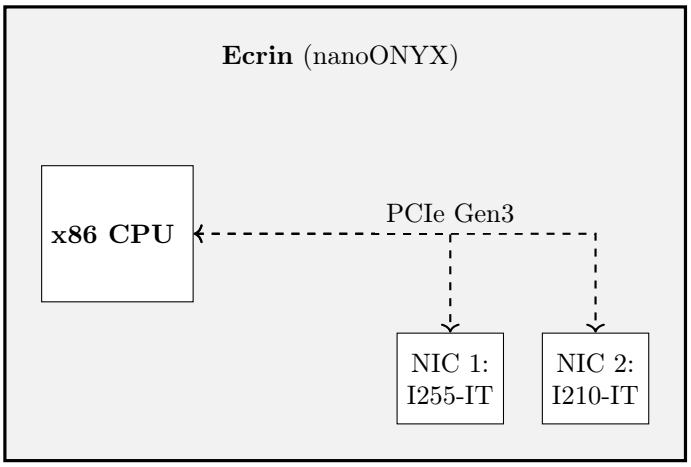


Figure 4.1: Overview of NIC and CPU Connection on the Ecrin

Some information regarding the I225-IT NIC is derived from the *Intel® Ethernet Controller I225 Software User Manual*. This document is officially classified as "Intel Confidential" and is not publicly available on the official Intel website; however, it was made available online by a third party and accessed through the non-official document repository DocumentPub [Int20].

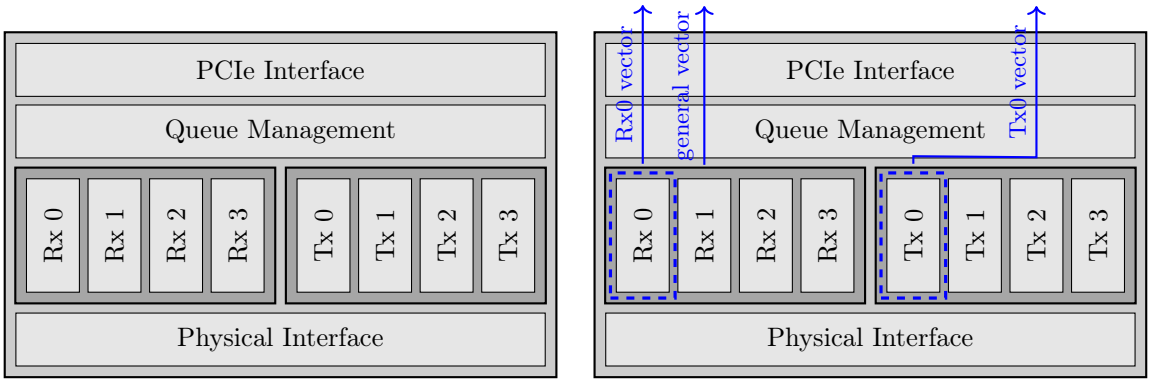
The igc device driver source code confirms that the I225-IT NIC supports a maximum of four transmit and four receive queues [Tor25]. Utilizing multiple queues offers significant advantages: on the receive side, queues can be assigned to different cores for efficient workload balancing [WDC12, p. 607]; on the transmit side, multiple queues allow for traffic prioritization [Che22]. Since Requirement R1 only demands minimal functionality, this implementation will be limited to using only one receive queue and one transmit queue for basic packet transfer, as highlighted in Figure 4.2b.

For event notification, the I225-IT primarily utilizes MSI-X, where individual interrupts are often referred to as vectors. The type of interrupt used directly determines the number of distinguishable interrupt signals a device can generate: the pin-based approach offers only one signal [SA99, p. 224], MSI supports up to 32 [SA99, p. 260], and MSI-X supports up to 2048 [Bud+12, p. 822].

While the NIC can technically operate without interrupts, doing so would require the CPU to constantly poll the device registers, a highly inefficient process. Therefore, at least one interrupt is necessary to avoid constant CPU polling. However, even with a single interrupt, the CPU must still check various device registers to determine which of several possible events triggered the signal. To avoid this overhead and distinguish NIC events more effectively, this implementation plans to use three distinguishable interrupts (Rx0 vector, Tx0 vector, and a general vector, as visualized in Figure 4.2b). One vector is dedicated to the receive queue and one to the transmit queue, as these events occur frequently and warrant differentiation. The final vector is reserved for all other infrequent events (e.g., updates to the link status).

Because three distinguishable interrupts are required, the pin-based approach cannot be used as it only generates a single interrupt signal; only MSI or MSI-X is suitable since they provide multiple distinct interrupt signals. The NIC supports up to five MSI-X vectors [Int24]. It also provides MSI and pin-based PCI interrupt methods for backward compatibility [Tor25]. Since existing ixl framework drivers (e.g., igb and ixgbe) already showcase the usage of MSI-X interrupts [L4R25a], MSI-X will be utilized. For simplicity, the igc device driver implementation will only support MSI-X and will not implement support for MSI or the pin-based approach.

The I225-IT features DMA functionality [Tor25]. Crucially, the DMA operations will be executed using physical memory addresses because the IOMMU will be intentionally left disabled within the L4Re environment.



(a) I225-IT NIC architecture

(b) Planned igc device driver features

Figure 4.2: I225-IT NIC architecture and planned device driver features adapted from [Rut]

4.2.2 Software Architecture

The necessary components and their connections for the software architecture were initially introduced as part of the compatibility criterion C1 for ixl (see subsubsection 3.2.4.1). Figure 4.3 visualizes the setup within the L4Re environment, where all components below the L4Re runtime environment should be familiar from chapter 2.

4.2.2.1 Component Initialization and Capabilities

The core process begins with the Ned configuration service, which creates the necessary components based on a Lua configuration script. This initialization process (visualized by black arrows in Figure 4.3) involves creating the VNS (which integrates ixl), the I/O server, and any L4Re Applications requiring network functionality (abbreviated as "app" in the figure). The Ned configuration service is also responsible for establishing the communication channels between these components by creating and granting the necessary capabilities, visualized as dashed arrows.

- To access the hardware, the VNS receives a vbus capability that contains the NIC.
- Any L4Re Application requiring network communication—whether internal (to another VM or Micro-App on the VNS) or external (to the ixl port)—must be granted a capability for a port on the virtual network switch.
- Although ixl is integrated into the VNS, it is still internally connected to one of the switch's ports. Communication targeting this specific port means routing traffic through the ixl device driver and subsequently to the physical NIC and the external physical network.

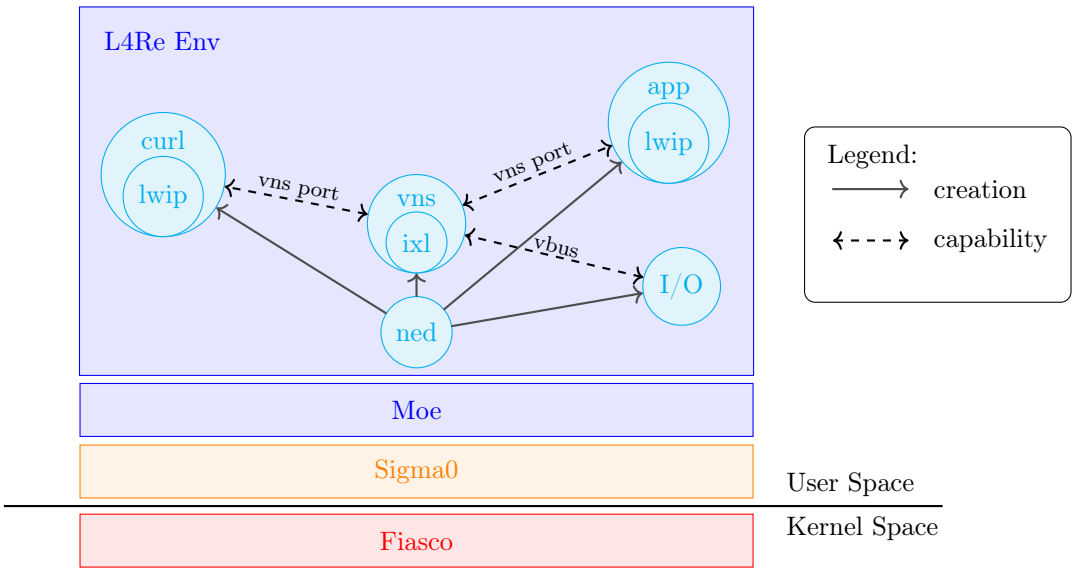


Figure 4.3: Software Architecture

4.2.2.2 Client Connection and Data Flow

The connection between the VNS and a client connected to one of its ports is established using the virtio interface (see section 2.4.8). This standardized interface allows both VMs and Micro-Apps to connect to the switch.

- When a VM connects, the application within the guest OS utilizes the guest's TCP/IP stack, with the bottom layer of that stack using a virtio driver connected to the VNS.
- A Micro-App does not contain its own TCP/IP stack; instead, it uses lwIP as a bridge (as explained in subsection 2.4.9).

This entire setup proves Requirement R2: the combination of the VNS and ixl successfully offers a defined virtio interface to all client L4Re Applications.

4.3 Testing

This section describes the test suite designed to validate the igc device driver implementation. The tests are structured to prove fulfillment of the mandatory requirements (R1, R2, and R3) and include an additional bandwidth test to assess the performance of the implemented device driver. The detailed hardware and software configurations for these tests are provided in chapter 6.

4.3.1 Test 1: Ethernet Frame Reception

The goal of this test is to verify the first half of Requirement R1: the ability of the ported igc device driver to successfully receive Ethernet frames.

The test setup consists of the L4Re target machine running the igc device driver, a network switch, and an external sending device. The sending device runs a utility that transmits an Ethernet frame containing a known test payload directed at the L4Re target.

Success Criterion The test is successful if the igc device driver running on the L4Re machine correctly receives, processes, and extracts the exact test payload sent by the external device.

4.3.2 Test 2: Ethernet Frame Transmission

This test verifies the second half of Requirement R1: the ability of the igc device driver to successfully transmit Ethernet frames.

The setup connects the L4Re target machine (running igc device driver) and a dedicated receiving machine to a network switch. The L4Re machine continuously transmits a sequence of Ethernet frames containing a known test payload to the receiving machine. The receiving machine monitors the network interface for incoming frames and displays their content.

Success Criterion The test is successful if the receiving machine correctly captures and verifies the exact test payload transmitted by the L4Re target.

4.3.3 Test 3: Multiclient test

This test serves two primary functions:

1. To verify Requirement R3 (Multiclient Access): that multiple L4Re Applications can independently utilize the igc device driver.
2. To also validate Requirement R2 (Defined Interface): the functional interface (lwIP and VNS) is proven functional because the successful execution of this test inherently requires the L4Re Applications to successfully connect to the driver via the defined interface.

The setup involves the L4Re machine instantiating two separate L4Re Applications, both connecting to the VNS (which integrates ixl and igc device driver). The L4Re machine and a single external receiver are connected via a switch. Both L4Re Applications continuously transmit UDP messages with distinct test payloads. The UDP protocol was chosen for simplicity, although any supported upper-layer protocol (such as TCP) would also be viable via lwIP.

Success Criterion The test is successful if the external receiver machine correctly captures and distinguishes the UDP test payloads originating from both L4Re Applications (i.e., proving access from each client).

4.3.4 Test 4: Performance (Bandwidth) Assessment

The bandwidth test does not validate a specific requirement but is necessary to assess the performance of the implemented igc device driver.

The setup utilizes a common network performance utility structured as a client/server pair. The L4Re target machine will run the server-side component, and an external client machine will connect to the same switch to execute the bandwidth measurement. The test will measure the maximum sustainable data throughput (Mbit/s).

Chapter 5

Implementation

The implementation of the igc device driver is presented in this chapter. First, the Linux igc device driver and the structure of an ixl device driver are analyzed in order to match functions with equivalent functionality. This analysis is followed by a detailed explanation of the most important functions of the ixl igc device driver implementation. In chapter 6, the implementation of the igc device driver is tested. Finally, chapter 7 evaluates the effort required to port other device drivers to the ixl framework.

5.1 Analysis

This section is divided into two parts. First, the Linux igc device driver is analyzed to identify its most critical functions and their relationships. This step provides the foundation for the porting effort. Second, the existing structure of a device driver within the ixl framework is reviewed. A comparison of the two structures identifies which igc device driver components can be reused to create a minimal-functional version in the ixl environment. This review simultaneously highlights necessary adjustments required for integrating alternative network drivers.

5.1.1 Analysis of Linux igc device driver

The Linux kernel manages hardware abstraction and driver registration using specialized data structures tailored for different peripheral buses (e.g., USB or PCI/PCIe). The specific structure for PCI drivers is illustrated in Listing 5.1.

Crucially, nearly every registration structure contains a mandatory `probe()` function. This function is responsible for the driver's initial setup, including hardware reset, data structure initialization, and registering the network interface with the kernel. The detailed steps performed by `igc_probe()` are listed in Table 5.1.

Listing 5.1: pci_driver struct (igc_main.c Linux kernel) [Tor25]

```
static struct pci_driver igc_driver = {
  .name        =   igc_driver_name,
  .id_table    =   igc_pci_tbl,
  .probe       =   igc_probe,
  .remove      =   igc_remove,
  .driver.pm   =   pm_ptr(&igc_pm_ops),
  .shutdown    =   igc_shutdown,
  .err_handler =   &igc_err_handler,
};
```

The resulting network interface object contains the `net_device_ops` structure, which abstracts the core driver operations (see Listing 5.2). When the network interface state changes to 'up', the `ndo_open()` function is called to bring the device and driver into an operable state [Tor25].

Listing 5.2: Snippet from the file igc_main.c from the Linux kernel [Tor25]

```
static const struct net_device_ops igc_netdev_ops = {
    .ndo_open            = igc_open,
    .ndo_stop            = igc_close,
    .ndo_start_xmit      = igc_xmit_frame,
    .ndo_set_rx_mode     = igc_set_rx_mode,
    .ndo_set_mac_address = igc_set_mac,
    .ndo_change_mtu      = igc_change_mtu,
    // none relevant functions are excluded
};
```

Data transmission is then initiated by the operating system calling `ndo_start_xmit()` whenever data needs to be sent. For reception, the igc device driver binds a specific IRQ handler based on the interrupt mode used: `igc_msix_ring()` (for MSI-X), `igc_intr_msi()` (for MSI), or `igc_intr()` (for pin-based IRQ). All these handlers ultimately utilize the Linux NAPI API to process the incoming packet data efficiently. A comprehensive description of the tasks for the most critical functions mentioned in this paragraph can be found in Table 5.1.

The NAPI is an efficient mechanism in the Linux kernel designed to mitigate performance degradation caused by interrupt storms in high-throughput network environments [Ben18, 46:03-47:33]. The core principle is "polling over interrupts": upon receiving the first packet, the system disables further receive interrupts for that queue and subsequently polls the hardware for new packets until the queue is empty. Only then are interrupts re-enabled [The25] [CRK05, pp. 525–526].

Crucially, while NAPI is primarily discussed in the context of packet reception [CRK05, pp. 525–526] [Ben18, 46:03-47:33], it is also responsible for cleaning up transmitted packets that have been successfully sent [The25]. For transmission, NAPI exclusively handles this cleanup process, while the initial packet submission remains the responsibility of the `ndo_start_xmit()` function, as previously detailed.

To understand the igc device driver's NAPI utilization, it is essential to note how the driver manages its queues. The NICs of the igc device driver have four hardware transmit (Tx) and four receive (Rx) queues. The driver creates four corresponding software queues where each structure pairs a Tx and an Rx hardware queue (e.g., software queue 0 manages Rx0 and Tx0). This bundling aligns with the hardware architecture: the igc NICs can utilize five distinct MSI-X vectors (interrupts).

In the Linux igc device driver, the first MSI-X vector serves as a general-purpose interrupt for non-queue events. The remaining four MSI-X vectors are mapped to the four software queues. Specifically, each vector is triggered by events occurring on its respective Rx and Tx hardware queue pair; for example, MSI-X vector 1 is assigned to Rx queue 0 and Tx queue 0. The specific hardware events triggering this interrupt are Rx descriptor writebacks, which occur when a packet is received, and Tx descriptor writebacks, which indicate that a transmitted packet has been processed by the NIC.

When an interrupt is issued by one of the dedicated queues, the ISR, `igc_msix_ring()`, is invoked. At this stage, subsequent interrupts for the queue are automatically disabled by the hardware, as configured by the igc device driver. This removes the requirement for manual masking within the ISR, whose main function is to schedule a NAPI worker. When the worker executes, it calls the `igc_poll()` function—registered during `igc_probe()`—which handles two tasks: processing received packets and freeing transmitted packets. Once the `igc_poll()` function determines there are no more packets to process or free, the interrupt for that specific queue is enabled again [Tor25].

By understanding key entry points that the Linux kernel expects for a network device driver, one can quickly navigate and analyze other drivers. This generalized structure is defined by the following components, which hold true across most Linux network drivers:

1. **Driver Initialization:** The driver's lifecycle begins with the registration routine, typically implemented in the `probe()` function.

2. **Core Operations:** Core networking functionalities (transmission, configuration, etc.) are exposed through the `net_device_ops` structure, which contains the `ndo_...` functions.

3. **Efficiency/Interrupt Handling:** By following the specific ISRs and the associated NAPI routine, one can understand how packets are received or how transmitted packets are freed.

These three function categories collectively serve as reliable navigational anchors to locate and adapt specific functionality, providing a generalized roadmap for understanding the architecture of virtually any Linux network device driver.

Table 5.1: igc device driver essential functions found in igc_main.c [Tor25]

Function	**Origin**	**Description**
`igc_probe`	`struct pci_driver`	The driver's initialization entry point, called when a matching PCI device is found. It performs essential hardware setup, including MMIO mapping and DMA configuration. Crucially, the function resets the NIC, disables its interrupts to establish a known good state, and signals the NIC's firmware that the host device driver has taken control; it does not yet make the device operational.
`igc_open`	`struct net_device_ops`	Called when the network interface state transitions to 'up' (e.g., via `ifconfig up`). It allocates necessary resources, establishes the link, requests and enables IRQs (installing the ISR), and prepares the NIC for packet operation.
`igc_close`	`struct net_device_ops`	Called when the network interface transitions to 'down' (e.g., via `ifconfig down`). It halts the device, disables and unregisters the IRQs, frees allocated resources, and relinquishes hardware control over the NIC.
`igc_remove`	`struct pci_driver`	The final cleanup routine, called when the PCI device is removed (e.g., hot-unplug) or the device driver is unloaded from memory. It stops the device, unregisters the network interface, frees resources, and releases MMIO regions.
`igc_xmit_frame`	`struct net_device_ops`	This function is the kernel's primary entry point for submitting a network packet to the device driver for transmission.
`igc_set_mac`	`struct net_device_ops`	Updates the NIC's hardware MAC address and configures the corresponding address filter for incoming packets.
`igc_msix_ring`, `igc_intr_msi`, `igc_intr`	MSI-X, MSI, or Legacy ISR function	The routine invoked upon a hardware interrupt. The specific function called depends on the most modern IRQ type supported by the system and the NIC (with MSI-X being preferred). It immediately disables the interrupt source and schedules the NAPI worker.
`igc_poll`	`struct napi_struct`	The core NAPI polling routine. Executed by the NAPI worker, it is responsible for simultaneously checking and processing received packets and freeing already transmitted packets. When processing is complete, it re-enables the corresponding queue's interrupt.

5.1.2 Analysis of ixl abstract device driver class

To implement an device driver, the ixl framework provides an abstract class `Ixl_device`. Initialization within this class is split between two distinct functions: the constructor and the `reset_and_init()` function.

When the device driver object is constructed, it performs basic, prerequisite operations such as mapping MMIO and registering IRQs. However, the NIC is not yet in an operational state at this point.

Immediately following the constructor call, the `reset_and_init()` function is executed. This function handles the crucial steps of resetting the NIC and configuring all necessary hardware settings to bring the NIC into a fully operational state.

Once initialized, the core packet processing functions, `tx_batch()` and `rx_batch()`, are available for sending and receiving packets [L4R25a]. These are among the most essential functions that a driver must implement, though they represent only a subset of the virtual functions required by the abstract `Ixl_device` class. Table 5.2 provides a detailed explanation of the referenced functions, derived from an analysis of the igb device driver implementation.

Table 5.2: Essential Functions of an ixl device driver [L4R25a]

Function	**Origin**	**Description**
`Igb_device` (Constructor)	ixl igb source code	The primary initialization routine called when the driver is instantiated. It maps MMIO, registers IRQs, sets up DMA, and initializes core driver settings and allocates memory for data structures.
`reset_and_init`	ixl igb source code	Called immediately after the constructor to transition the device to an active state. It performs hardware resets, allocates run-time resources, enables interrupts, and establishes the network link.
`tx_batch`	virtual function of (ixl_device)	The entry point of the device driver to transmit packets. It handles the cleanup of previously sent packets and forwards a batch of new packets to the hardware. It returns the count of packets successfully sent.
`rx_batch`	virtual function of (ixl_device)	The primary entry point of the device driver to receive and process incoming packets.
`set_mac_addr`	virtual function of (ixl_device)	Optional function used to program a different MAC address into the NIC hardware.

5.1.3 Matching ixl & Linux kernel API

Following the analysis of the Linux igc device driver and the abstract `Ixl_device` class, it is clear that the high-level sequence of tasks required to bring a NIC to an operational state remains consistent across both environments. However, these tasks are distributed differently across the respective function calls in the Linux and ixl network drivers. Table 5.3 presents a direct functional mapping between the core interfaces of the Linux driver and the ixl framework, explicitly detailing the key differences and functional shifts between the two environments.

5.1.4 Evaluating ixl and linux function compatibility

Although a functional mapping exists between the high-level Linux and ixl interfaces (as detailed in subsection 5.1.3), the Linux igc device driver implements a significantly broader scope of features. This highly optimized implementation is designed for maximum performance, incorporating many functionalities unnecessary for fulfilling Requirement R1 (basic Ethernet frame transmission and reception ability, see chapter 4).

The substantial difference in scope between the drivers presents a fundamental challenge when considering the Linux code as a template: The implementation complexity of the Linux igc device driver makes it difficult to clearly separate the essential, core functionality required by Requirement R1 from the numerous additional features. This difficulty arises because the code base is approximately ten times larger (see Table 3.2) and utilizes a vast number of registers beyond those needed for a basic Rx/Tx implementation. Consequently, using the Linux igc device driver as a direct template for the ixl port is not beneficial, requiring a major shift from the original methodology of this thesis.

During the review of the existing igb driver and the search for the igc device driver datasheet, evidence emerged suggesting a strong relationship between the igc device driver and igb drivers. This assumption was initially triggered by an online discussion [sob21] where a user, citing an Intel engineer involved in the development of the igc device driver (for the i225 NIC), noted that the i210 NIC (compatible with the igb driver) is very similar to the i225. Furthermore, another user suggested that all basic Rx/Tx functionality is compatible across all Intel NICs because they originally shared a common Linux driver foundation [sob21].

This assumption was confirmed through a register-level comparison: All registers utilized by the existing igb ixl driver were cross-referenced against the datasheet of the i225 NIC. It became apparent that all necessary igb registers also exist on the i225 hardware.

Therefore, the new methodology is to use the already existing igb ixl driver as the template for creating the igc device driver within the ixl framework. This approach is beneficial because the igb driver uses the basic configuration required for Rx/Tx functionality and

Table 5.3: Match between Linux and Ixl Device Driver Interface

Linux Function	Ixl Function	Differences
igc_probe	Igb_device (Constructor)	igc_probe performs basic initialization and resets the NIC to a known good state. In contrast, the ixl constructor (for Igb_device) handles basic initialization but omits the hardware reset. Critically, the ixl constructor registers IRQs immediately, a task deferred until igc_open in the Linux driver.
igc_open	reset_and_init	Both functions are responsible for making the underlying network device operational. The key difference lies in the sequence of tasks: igc_open does not perform a device reset (as it was handled by igc_probe), but it is the point where the IRQs are registered. Conversely, reset_and_init executes the hardware reset (omitted by the constructor) before configuration, and the IRQs were already registered in the ixl constructor.
igc_close, igc_-remove	No direct functional match in ixl	There is no equivalent function in the ixl framework to gracefully deactivate or shut down the device driver (such as igc_close or igc_remove). This omission is likely intentional, as the underlying L4Re operating system is designed for embedded systems where the NIC is permanently enabled and hot-plug events are not expected.
igc_xmit_frame, igc_poll	tx_batch	The Linux framework splits transmission functionality: igc_xmit_frame handles packet transmission, while igc_poll manages the cleanup of already transmitted packets. In the ixl framework, these two separate tasks are consolidated and handled entirely within the tx_batch function.
igc_poll	rx_batch	Both functions handle packet reception. The Linux igc_poll is dual-purpose and includes Tx cleanup; rx_batch is dedicated to reception only.
igc_set_mac	set_mac_addr	Functionality is nearly identical. The Linux igc_set_mac also creates a receive filter for the new address. Notably, set_mac_addr is not universally implemented in ixl, being supported by only one of the three current drivers [L4R25a].

already adheres to the necessary ixl interfaces. The comparison of the implemented features between the resulting ixl igc device driver and the original Linux igc device driver will be presented at the conclusion of this chapter.

5.2 Igc Driver Implementation in ixl

This section details the critical functions required for developing an ixl device driver, using the igc device driver implementation as a primary example. These consist of mandatory functions defined by the `ixl_device` virtual class, as well as conventional functions that are not strictly required but are implemented across all ixl drivers to provide a consistent and enhanced structure. Table 5.4 provides an overview of these functions, highlighting the distinction between virtual and conventional implementations.

This shared architecture exists because the e1000 and igb drivers within ixl were originally modeled after the ixgbe driver from the ixy project. Since the igc device driver is largely based on the existing igb driver, it utilizes these same conventional functions. The following description encompasses driver registration, the initialization process, and the core management of Rx, Tx, and interrupts. The verification of these functionalities through Tests 1–4 is documented in chapter 6 to determine if the driver implementation is fully operational. Specifically, Tests 1 and 2 ensure basic functionality, while Tests 3 and 4 assess system integration and performance.

Table 5.4: Virtual and conventional functions of an ixl device driver

Function	**Type**
`Igc_device` (Constructor)	conventional
`igc_init`	conventional
`reset_and_init`	conventional
`init_rx`	conventional
`init_tx`	conventional
`start_rx_queue`	conventional
`start_tx_queue`	conventional
`rx_batch`	virtual
`tx_batch`	virtual
`msix_init`	conventional
`enable_rx_interrupt`	conventional
`check_recv_irq`	virtual
`ack_recv_irq`	virtual

5.2.1 Driver Registration

To determine the exact point where integration is required, it is necessary to trace the ixl initialization path within the VNS. When the preprocessor macro `CONFIG_VNS_IXL` is defined, the ixl module is added to the VNS. If the ixl module is active, the VNS executes the `discover_ixl_devices()` function, which subsequently calls the static initialization method `ixl_init()` of the abstract class `Ixl_device`.

For simplicity, throughout this chapter, the term PCI refers collectively to both the PCI bus system and the PCIe standard, unless explicitly stated otherwise.

When `ixl_init()` is called, it receives the vbus, a device index, and a configuration structure. Inside the function, it searches the vbus for available Ethernet devices and selects one based on the provided index. Currently, the VNS statically passes index 0, selecting the first device found. This creates a potential issue: since the vbus contains all PCI Ethernet devices by default, ixl initialization will be aborted if the first device lacks a compatible driver, even if a subsequent device is compatible. This problem can be resolved by either adjusting the device index or limiting the scope of the vbus to only compatible devices. It should also be noted that `ixl_init()` currently only supports PCI interfaces, excluding NICs connected via other bus systems like USB.

After the desired Ethernet device is selected, its vendor and device IDs are read from the PCI configuration space, as these are the primary identification attributes (see subsection 2.1.6). Based on these IDs, the framework checks for a compatible driver using the nested `switch`/`case` statement shown in Listing A.1. This is the critical point where a new device driver must be registered in ixl.

The `switch` statement is structured to first check the vendor ID, followed by a nested `switch` statement for the specific device ID. Since Intel is already listed as a supported vendor (line 4 in Listing A.1), it was only necessary to add the new `case` entries for the igc device driver within the existing Intel vendor block. However, if a driver for a new, non-Intel device vendor needed to be integrated, a new primary `case` entry would first have to be created for that vendor ID, and the supported device IDs would then be integrated within the corresponding nested `switch` statement.

Lines 25 to 33 display the new additions for the igc device driver (some specific NIC variations are omitted for brevity). The device ID definitions, such as `IGC_DEV_ID_I225_IT`, were directly copied from the header files of the Linux igc device driver. Interestingly, the naming scheme of the `case` statement definitions for other supported devices closely matches the Linux igc device driver definitions, suggesting that the existing ixl drivers have also copied definitions from the Linux kernel headers.

Once the correct device driver is determined via the `switch` statement, its initialization function—typically named `<driver-name>_init()`—is called. This static function is comparable to a constructor: it dynamically creates a new instance of the driver object, and calls the instance method `reset_and_init` to bring the driver and the corresponding NIC to an operational state (see subsection 5.1.2), and finally returns a pointer to the newly created driver object to the VNS for device control. If no compatible driver is identified, `ixl_init()` returns a null pointer. Upon receiving a valid pointer, the VNS creates the corresponding ixl-port; otherwise, the initialization of the port is aborted.

5.2.2 Driver Initialization

As previously established in the context of the ixl driver analysis (see subsection 5.1.2), the complete initialization process is logically split between two primary functions. The first is the constructor, which performs the basic setup. The second is the instance method `reset_and_init`, which handles all remaining steps necessary to bring the device to an operational state. Both of these functions will be explained in detail within the following subsections.

5.2.2.1 Basic Initialization

As shown in Listing A.2, the constructor first initializes internal values based on the provided configuration structure `cfg`, which specifies parameters such as the number of queues to be used and the interrupt timeout value. It then validates the requested queue count. The igc device driver currently supports only a single Rx and Tx queue, so an error message is printed if the configured number of queues exceeds one.

Following initial configuration (lines 16–33), the constructor sets the interrupt timeout and determines the interrupt operating mode. The device driver supports three distinct modes: interrupt disable, interrupt notify, and interrupt wait. As illustrated by the conditional structure (lines 23–30), the value of the interrupt timeout directly selects the operating mode.

The code explicitly demonstrates the usage of the interrupt disable mode exclusively within the ixl test utilities (`ixl-pcap` for packet reception and `ixl-pktgen` for transmission). The interrupt notify mode is the one utilized by the VNS. Although the interrupt wait mode is currently not used anywhere in the available implementation, the VNS explicitly overrides the default interrupt timeout (1000 ms) with a value of -1, which forces the use of the interrupt notify mode. If the VNS did not provide this explicit override, the device driver would default to the interrupt wait mode. This fact suggests that the interrupt wait mode might also be usable with the VNS; however, this remains unclear without further investigation and experimentation. Because the thesis targets only basic functionality of the device driver, the ixl igc device driver will therefore be tested exclusively in interrupt notify mode, and configurations using interrupt wait mode are left outside the current test scope.

After the interrupt mode is determined, the interrupt rate is set (line 35). Currently, this interrupt rate is not configurable via the VNS but is instead statically assigned within the `igc_init` function. This static assignment is a pattern currently observed across all implemented drivers.

Between lines 39 and 43, the device's MMIO space is mapped, and the starting address of this device memory is saved to the variable `baddr`. This address is subsequently used very frequently by the driver to access and write to the hardware.

Line 47 creates a DMA space capability (an abstraction for the DMA address space [L4R25c, p. 1640]) for the device, which is essential for allocating DMA memory in subsequent steps. Furthermore, if the device driver is operating in an interrupt-enabled mode, a capability for a virtual Interrupt Control Unit (ICU) is set up (lines 49 to 52), and the MSI-X vectors are initialized. The specifics of the interrupt vector setup are described in detail in subsection 5.2.5.

The function concludes by dynamically allocating memory for the necessary Rx and Tx queue structures.

5.2.2.2 Device reset & complete initialization

Listing A.3 displays the `reset_and_init` function of the igc device driver. As previously noted, this implementation was primarily copied from the existing igb device driver. The function's main responsibility is configuring the NIC by manipulating its hardware registers. Intel NIC registers are typically 32 bits long [Int21, p. 364] [Int20, p. 360]. The ixl framework provides simple helper functions for register manipulation: `get_reg32()` for reading and `set_reg32()` for writing. The first argument to these functions is the base address of the mapped MMIO space, while the second argument is the offset to the specific register being accessed. For `set_reg32()`, a third argument provides a bit mask to determine which bits within the register should be modified.

The register offsets and bit mask definitions required by the igc device driver were sourced directly from the Linux igc device driver header files. This strategy mirrors the approach used during the development of the ixl igb driver, which also leveraged definitions from its Linux counterpart. Due to the high degree of naming consistency between the igb and igc device driver register sets, porting definitions is often straightforward. For instance, the definition matching `IGB_RCTL` can typically be found by searching for `IGC_RCTL` in the Linux igc device driver source. If a definition is not readily available through this method, it must be manually extracted from the device's datasheet and defined by the developer.

Much of the functionality within `reset_and_init` is made clear by the existing source code comments. Consequently, the discussion below will focus only on functions and concepts that require additional context or deeper technical understanding.

The function `init_link`, which is called on line 30 and detailed in Listing A.4, configures the necessary flags to enable the NIC to automatically determine the link connection speed and duplex settings.

The function `set_promisc` places the NIC into promiscuous mode, causing it to accept all incoming packets, regardless of the destination MAC address. Ordinarily, the NIC filters traffic and only accepts packets destined for its own MAC address. Using promiscuous mode ensures that packets are received even if MAC filters are not configured correctly. The consequence of this setting is that the host system must process unnecessary packets addressed to other machines, potentially introducing overhead. Therefore, setting up the NIC's MAC filters correctly to reliably receive only locally addressed packets would represent a significant improvement to efficiency.

Near the end of the function, the `wait_for_link` routine blocks execution by busy polling a specific status register of the NIC until the network link has come up. It maintains this busy polling state for a maximum duration of 10 seconds.

Following these steps, the NIC is fully operational.

5.2.3 Implementing Receive (Rx) Functionality

The Rx functionality is also copied from the igb driver, with register definitions adjusted to match the target hardware. These definitions were sourced primarily from the Linux kernel's igc device driver source code and supplemented by the datasheet.

5.2.3.1 init_rx()

The overall Rx functionality is initialized as part of the driver's setup routine, specifically within the top-level function `reset_and_init()`. This function, in turn, calls the specialized initialization function, `init_rx()`.

Listing A.5 presents the source code for the `init_rx()` function. Line 9 provides a good example of the challenge of locating register definitions: the definition `IGC_RCTL_VFE` (which disables VLAN filtering) could not be located directly in the Linux igc device driver source code.

Instead, the Linux igb source code was referenced, which defined the corresponding bit position for VLAN filtering on the same register. By cross-referencing this information with the i225 datasheet, it was confirmed that both the register address and the bit position were identical across the igc device driver and igb hardware. Therefore, the Linux igb definition was successfully copied and simply renamed to fit the igc device driver context.

After clearing the VLAN bit on Line 9, the function initiates a `for` loop designed to iterate over the configured number of Rx queues for initialization. Although the igc device driver currently supports only a single queue, rendering this loop redundant, the structure was intentionally preserved. The loop was directly copied from the igb driver, which had incorporated the multi-queue design pattern from the ixgbe driver for convenience. This

redundancy is maintained solely to simplify the future implementation of multi-queue support in the igc device driver.

To fully enable multi-queue support in the future for the igc device driver (or the igb) driver, the presence of the `for` loop alone is insufficient. The primary limitation to a single queue is the absence of key multi-queue register definitions. For example, definitions like `IGC_RDBAL` must be updated. While the original Linux ixgbe driver utilizes a macro that calculates register offsets based on the queue index (see Listing 5.3 line 2), this specific multi-queue macro definition was not ported to the ixl igb driver. Consequently, the igc device driver (and the igb) is limited to a single queue due to this missing macro definition (Listing 5.3). Theoretically, the same calculation performed by the macro could be accomplished via inline code, but this alternative was not pursued in the current implementation.

Listing 5.3: Showcasing igb and igc macro limitation [L4R25a] [Tor25]

```
1  // ixgbe definition copied from ixl -> orginally copied from
      dpdk, but the same defintion exists in the linux kernel to
2  #define IXGBE_RDBAL(_i) (((_i) < 64) ? (0x01000 + ((_i) *
      0x40)) : (0x0D000 + (((_i) - 64) * 0x40)))
3
4  // igb definition for the same register from the linux kernel
      (renamed from E1000_RDBAL to IGB_RDBAL to make point more)
5  #define IGB_RDBAL(_n)   ((_n) < 4 ? (0x02800 + ((_n) *
      0x100)) : (0x0C000 + ((_n) * 0x40)))
6
7  // igb definition from ixl -> shows that the definition
      limits the driver to only use one queue
8  #define IGB_RDBAL     0x02800
9
10 // igc definition from ixl -> shows that the definition
      limits the driver to only use one queue
11 #define IGC_RDBAL       0x0C000
```

In the following, the most important tasks that occur within this initialization `for` loop are explained. First, the DMA memory for the Rx queue is allocated. A queue for NICs is typically realized in software as a ring buffer, and the same approach is used by this driver. This Rx ring buffer will later contain the descriptors, which hold the physical addresses of the host memory where the NIC should store received packets. Within this function, the memory for this Rx ring buffer is allocated based on the size of the descriptor and the size of the ring. The size of the ring is statically defined by the variable `NUM_RX_QUEUE_ENTRIES` to 256 descriptors per ring, which is the default of the Linux driver counterpart [Tor25]. The entries are initialized to -1, indicating they do not yet contain any valid data.

Next, the start address of the Rx ring buffer is programmed into the hardware using the `IGC_RDBAL` and `IGC_RDBAH` registers (see line 38-39). There are two important details regarding this process:

1. **Address Splitting:** Since the memory address is 64 bits long, but a register on the NIC is only 32 bits long, the address must be split into two registers [Int20, p. 422].
2. **Register Naming:** RDBA stands for Read Descriptor Base Address; the trailing "L" or "H" stands for register low or high, respectively [Int20, p. 422].

Additionally, the usage of `mem.phy` and `mem.virt` is a critical detail. When the start address is written to the NIC, `mem.phy` (physical address) is used because the NIC's DMA engine operates solely with physical memory, as detailed in chapter 4. Conversely, the device driver, as a software component, always interacts using `mem.virt` (virtual address).

Next, the length of the Rx ring is configured in the NIC using the `IGC_RDLEN` register. Finally, the ring buffer's initial state is set by setting the internal head and tail pointers to 0 (lines 42-43), which signifies that the ring is currently empty.

After the `for` loop completes, the function performs final configuration steps, including setting up checksum offloading, and then enables the Rx queue.

Summarized, this function sets up the Rx ring in software and programs all necessary information into the NIC.

5.2.3.2 start_rx_queue()

The function `start_rx_queue()`, shown in Listing A.6, is called within the `reset_and_-init()` function, immediately following the `init_rx()` function. The `init_rx()` function allocated an empty Rx ring. The primary task of the `start_rx_queue()` function is to create the receive descriptors, save them into the Rx ring, and back them with host memory where the received packets will be stored.

In line 17, a mempool is created. This mempool serves as an abstraction for memory within the ixl framework. The most critical detail for this explanation is that the mempool utilizes DMA memory, which guarantees that the physical addresses of the allocated buffers are contiguous.

In line 24 begins the main `for` loop, which iterates through every descriptor position from the start to the end of the Rx ring.

1. **Volatile Descriptor Access (Line 25):** The current queue descriptor is saved to a variable marked as `volatile`. The `volatile` keyword instructs the compiler that the value of this memory location could change at any time, even if not modified by

the software. This is a critical practice here because although the queue is not yet started, descriptors are shared memory accessed by both the software (driver) and the hardware (NIC).

2. **Buffer Allocation (Line 26):** A buffer is allocated from the mempool. In the subsequent lines, the physical address of this buffer is assigned to the current queue descriptor. This action effectively backs the queue descriptor with host memory, ensuring the NIC knows exactly where to deposit a received packet.

3. **Initialization:** The error and status flags within the descriptor are zeroed out. These fields will be populated by the NIC later on when it uses this descriptor to signal information or status about the received packet [Int20, p. 255]. Since the default settings were not changed, the NIC uses legacy receive descriptors [Int20, p. 255].

The loop repeats this process until all 256 descriptors in the ring are successfully initialized and backed by host memory.

After the descriptor initialization loop completes, the function proceeds to enable Rx queue 0 in line 44. Line 45 then implements a critical step: it waits until the corresponding bit is set in the hardware status registers, successfully signaling that the queue has been enabled and is ready for use by the NIC.

The final step of the function is to update the head and tail pointers of the ring. The head pointer is set to the first position (0), and the tail pointer is updated to the end of the ring. By setting the tail pointer to the end, the driver signals to the NIC that the entire ring is currently full of available descriptors, ready to receive packets.

5.2.3.3 rx_batch()

The `rx_batch()` function, shown in Listing A.7, serves as the primary interface of the driver for receiving packets. Its core tasks are:

1. To determine if the NIC has received packets by checking if descriptors are marked as done.

2. For any completed descriptor, to return the virtual address of the received packet in host memory back to the caller.

3. To back the used descriptors with newly allocated host memory for future packet reception.

The function ultimately returns the count of packets successfully processed and returned to the caller during the execution.

The `rx_batch()` function includes interrupt wait mode logic in lines 2–3, 9–19, and 97–116 that is not used and is therefore excluded from this description. Line 6 represents the first non-interrupt-related code, providing a safety check to ensure the exclusive use of queue 0.

Between line 21 and 25, several variables are set up before entering the main `for` loop. The most critical variable is `rx_index`, which is set to the last index that was processed the previous time the function ran. This index is persistently stored within the helper structure `queue`. If `rx_batch()` has not been executed yet, this variable is correctly initialized to 0 by the preceding `init_rx()` function.

The function accepts the argument `num_bufs`, which specifies the maximum number of packets (the "budget") the function should attempt to process in this batch. This budget determines the maximum run time of the loop.

Inside the loop, the current Rx descriptor is selected on line 29 based on the current `rx_index`. On line 34, the status information of that descriptor is checked. The presence of a specific flag signals that the NIC has already processed this descriptor, meaning it has successfully copied a received packet into the backing memory. If this flag is absent, the loop terminates, as no more packets have been received yet.

If the status check confirms that the descriptor was processed by the NIC, a local copy of that descriptor is created in lines 41 to 42. A crucial step follows in line 43: the virtual address of the received packet, which is backed by the current descriptor, is saved to the variable `buf`. Since the descriptor itself only holds the physical address for DMA and loses the virtual address information, the driver relies on a helper array, `queue->virtual_addresses[]`. This array serves as a lookup table, mirroring the Rx ring but storing the corresponding virtual addresses [Emm+19, p. 6]. This table was populated during the earlier execution of the `start_rx_queue()` function.

As the reference to the received packet is saved now, new DMA memory for the new packet buffer is allocated in line 49. Lines 65-67 then update the Rx descriptor to be backed by this newly allocated packet buffer (specifically, by writing its physical DMA address to the descriptor). Afterwards, in line 69, the virtual address of the packet buffer backed by that descriptor is also updated within the helper array `queue->virtual_addresses[]`, ensuring that the driver retains the correct host memory address for future lookups. Finally, in line 70, the saved reference (the virtual address of the received packet) is written to the array provided by the caller as an input argument, completing the hand-off to the application.

At the conclusion of each loop iteration, the `rx_index` is incremented using the `wrap_ring()` function. This function employs bitwise operations to handle the wrapping logic at the end of the ring, which necessitates that the ring size be a power of two for computational efficiency.

The loop continues this process until either no further processed descriptors are found or the pre-defined budget is exhausted. Following the loop, a conditional check on line 81 verifies

if any packets were handled by comparing `rx_index` with `last_rx_index`. If packets were processed, the driver must update the Rx ring tail register on the NIC. This hardware update informs the controller that fresh descriptors and memory buffers are available for future incoming traffic. The `rx_index` is then persistently saved within the `queue` structure to ensure the next call to `rx_batch()` resumes from the correct position.

On lines 90–94, the function performs a final check to determine if unprocessed packets remain in the ring by reading the current hardware head pointer. If the tail pointer is immediately behind the head, the ring is considered empty for the time being. The remaining lines of the function manage interrupt-related logic, and finally, the function returns the total count of processed packets to the caller.

5.2.4 Implementing Transmit (Tx) Functionality

The Tx functionality was ported from the existing igb driver, with register definitions updated to support the igc device driver hardware. These definitions were primarily sourced from the Linux kernel source code and cross-referenced with the official Intel i225-IT datasheet.

5.2.4.1 init_tx()

The function `init_tx()` is executed within the `reset_and_init` function after the device has been reset and the network link has been re-established.

Listing A.8 shows the implementation of this function. The functionality is almost identical to `init_rx()`. It allocates the DMA memory necessary for the Tx descriptor ring, initializes it with −1, and configures the hardware with the base physical DMA address and the total length of the ring. Additionally, the virtual start address of the Tx ring is saved for the lookup table. The `init_tx()` function mirrors `init_rx()` so closely that subsubsection 5.2.3.1 should be consulted for further details, as both the operational logic and the register nomenclature are functionally equivalent.

5.2.4.2 start_tx_queue()

The `start_tx_queue()` function is invoked within `reset_and_init()` immediately following `init_tx()`. Unlike its Rx counterpart, this function does not pre-allocate packet buffers or back descriptors with memory. In the transmit path, the caller is responsible for providing the packet memory, passing these buffers to `tx_batch()` as needed.

Consequently, the primary role of `start_tx_queue()` is to configure and enable hardware-specific settings. As shown in Listing A.9, the Tx queue is activated on line 8. While this technically prompts the NIC to fetch memory associated with the Tx descriptors, no data

is actually transferred at this stage. This is because the Tx ring was initialized with −1 in `init_tx()`, effectively pointing every descriptor to an invalid address. Furthermore, the NIC will not begin transmitting until the global enable bit in the Transmit Control Register (TCTL) is set [Int20, p. 444].

Lines 10 through 19 configure the collision threshold to 16, ensuring the hardware adheres to Ethernet standard IEEE Std 802.3 by attempting atleast 16 retransmissions in the event of a collision [LAN22, pp. 257, 282]. This setting is specifically relevant when the NIC operates in half-duplex mode [Int20, p. 438]. Additionally, hardware padding for small packets is enabled. In the final line of the function, these configurations are committed to the TCTL register, and transmission for the queue is officially enabled. At this point, the NIC is ready to transmit packets.

5.2.4.3 tx_batch()

The `tx_batch()` function (see Listing A.10) performs two primary tasks. The first half (lines 8–52) handles the cleanup of transmit descriptors for packets already sent by the hardware, which is performed in batches to optimize performance. The second half (lines 54–91) manages the actual transmission of new packets.

To illustrate the function's logic, this section describes its second part first, as the cleanup phase is effectively skipped during the initial execution when no descriptors are yet pending.

The operational logic of the ring is demonstrated using a simplified configuration of eight descriptors. Figure 5.1 illustrates the initial state where all indices—including the hardware pointers and the software-managed `clean_index` and `tx_index`—are initialized to zero. This is ensured by the allocation of the `tx_queue` structure via `calloc`, which guarantees that the memory is zeroed upon instantiation.

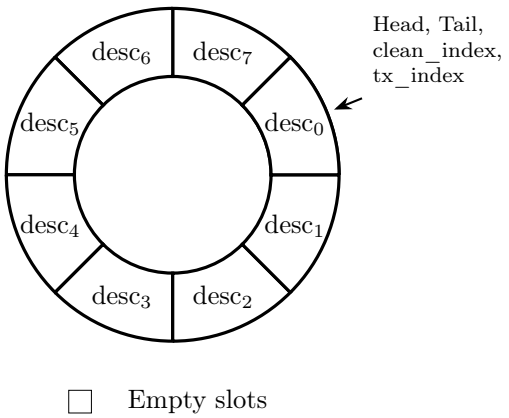


Figure 5.1: Initial Tx Ring State

In this example, `tx_batch()` is called to send five packets. Since there are no descriptors to clean yet, the function moves to the transmission loop. Here, `tx_index` points to the

next available slot. On lines 57–61, the driver checks if the ring is full by verifying that `clean_index` is not exactly one position ahead of `tx_index`.

If there is space, the function retrieves the first packet from the `bufs` array (line 62), which contains the packet buffers provided by the caller for transmission. Its virtual address is saved in a lookup table (line 64) so the memory can be freed later. The driver then selects the descriptor at the current `tx_index`, connects it to the packet's physical address, and increments the index.

The loop continues until all packets are processed or the ring is full. Finally, the driver updates the NIC tail pointer to the current `tx_index`, which notifies the hardware that new packets are ready to be sent. In this example, as the ring was initially empty, all packets were successfully connected to descriptors. The ring state at this point is illustrated in Figure 5.2.

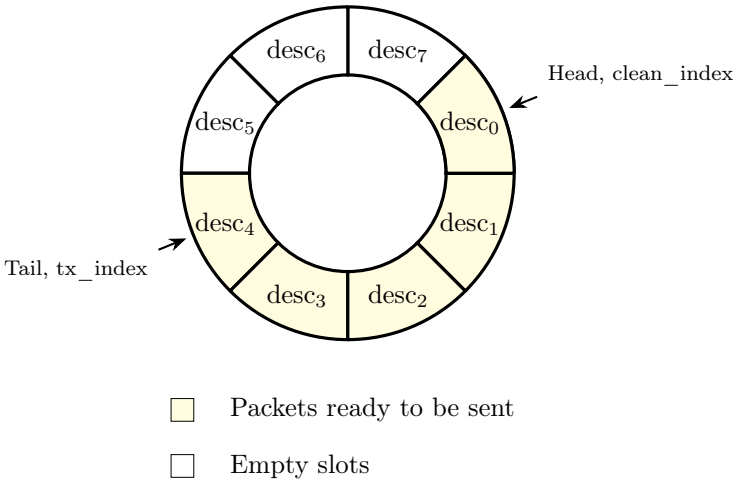


Figure 5.2: State of Tx Ring after enqueueing 5 packets

In this scenario, we assume some time elapses before `tx_batch()` is invoked again, during which the NIC successfully transmits four of the five queued packets. Figure 5.3 illustrates the updated state of the ring. The hardware head pointer has advanced to the index of the last packet sent, marking those descriptors as completed and ready for reclamation by the driver.

When `tx_batch()` is invoked again, the function enters the cleanup phase. The driver calculates the number of pending descriptors by subtracting `clean_index` from `tx_index` (line 17). In this scenario, the calculation identifies five in-flight packets. Because the software does not read the hardware head pointer directly, the exact number of processed packets is not yet known at this stage.

To optimize performance, cleaning is performed in batches. On line 22, the driver checks if enough descriptors are pending to make a cleanup cycle worthwhile. While the actual driver uses a batch size of 32, the current example assumes a batch size of three. Since five packets are pending, the check evaluates to positive. The driver then calculates the target

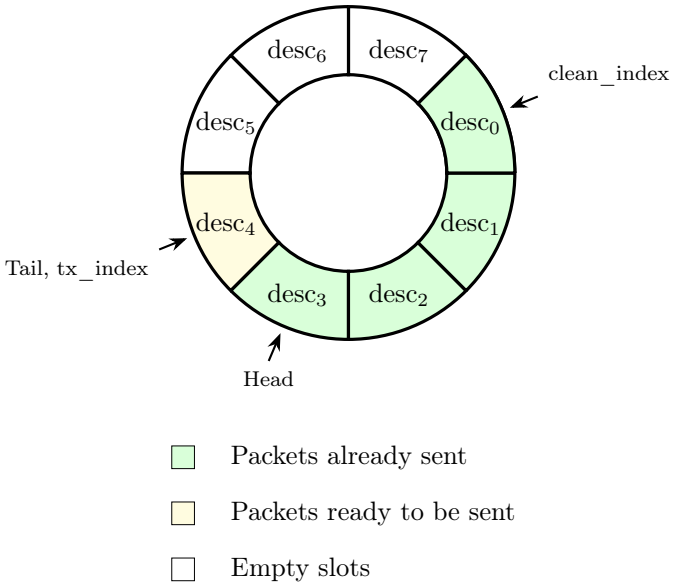


Figure 5.3: State of Tx Ring after sending 4 packets

index, `cleanup_to`, by adding the batch size to the `clean_index` (lines 27–31), which in this case corresponds to index 2.

Crucially, on line 34, the driver inspects the descriptor at the `cleanup_to` index to verify if it has been processed by the NIC. If the hardware has not yet reached this point, the cleanup process is aborted. In this scenario, the hardware has already passed the `cleanup_to` index. Consequently, the driver iterates through the batch (lines 34–45), retrieving the virtual addresses from the lookup table and freeing the associated memory. Once the batch is processed, the `clean_index` is advanced to the position immediately following the `cleanup_to` index. Finally, on line 52, this new position is saved back to the queue structure. The resulting state of the ring is shown in Figure 5.4. Under normal operating conditions, the transmission functionality would follow this cleanup; however, as this was detailed in the preceding section, it will not be illustrated further.

5.2.5 Implementing Interrupt (IRQ) Handling

This section details the interrupt notify mode implementation. The igc device driver configures a single MSI-X vector to signal Rx queue events. This signal triggers the VNS to initiate packet reception using the device driver function `rx_batch()`. A crucial design feature is that the next interrupt is only generated after the preceding one has been explicitly acknowledged.

5.2.5.1 msix_init()

The igc device driver calls `msix_init()` within its constructor when the device driver is operating in either interrupt notify mode or interrupt wait mode. Listing A.12 shows the

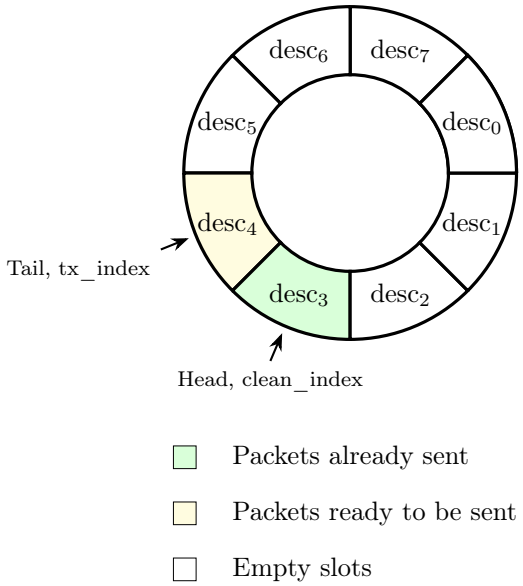


Figure 5.4: State of Tx Ring after 3 packets are cleaned

function, whose primary task is to allocate and configure MSI-X vectors. The critical aspects of this process are outlined below:

- **Legacy Check:** The function first verifies device support for MSI-X. Since the current igc device driver implementation does not support legacy interrupt modes, a lack of MSI-X support forces the initialization to abort and switches the driver to interrupt disable mode.

- **Vector Allocation:** If MSI-X is supported, the function allocates one MSI-X vector per Rx queue. As the igc device driver currently only supports a single Rx queue, it allocates MSI-X vector 0.

- **Thread Binding:** In line 42, the allocated MSI-X vector is bound to the current thread using `create_and_bind_irq()`. Since the device driver executes as part of the VNS, this binding ensures that the VNS thread is notified when MSI-X vector 0 is issued by the hardware. This architecture allows the VNS to handle interrupts and call corresponding device driver functions, detailed further in subsubsection 5.2.5.4.

- **Configuration Storage:** Essential interrupt setup parameters are stored within the `interrupt` helper structure for the respective Rx queue. Modifying the desired interrupt behavior will likely necessitate changes to this structure.

This initialization process adheres to the PCIe specification and utilizes PCI helper functions provided by ixl. Consequently, for other device drivers that support MSI-X, the core logic within the `msix_init()` function will remain largely identical, making it a highly reusable component for other PCI NIC device drivers.

5.2.5.2 Interrupt Settings

Within the `reset_and_init()` function (Listing A.3), the General Purpose Interrupt Enable (GPIE) register is configured to manage interrupt handling. This involves setting several flags crucial for the igc device driver's operation, particularly for MSI-X and advanced interrupt features:

- **Multiple MSI-X:** When this flag is set, it enables the mapping of different events to a single MSI-X vector using the Interrupt Vector Allocation Registers (IVAR). Crucially, setting this bit alters the layout of several other interrupt-related registers, without which the configuration applied by `enable_rx_interrupt()` would be ineffective.
- **EIAME:** This bit enables support for Extended Interrupt Auto Mask Enable Register (EIAM), which is detailed in the subsequent section.
- **NSICR and PBA:** These bits are primarily important for legacy interrupt modes. They must be set to one to ensure proper operation in MSI-X mode [Int20, p. 310].

5.2.5.3 enable_rx_interrupt()

The `enable_rx_interrupt()` function, shown in Listing A.11, is called during the `reset_-and_init()` process. Its primary role is to set up the interrupt handling for the Rx queue 0.

The function performs two main configurations:

1. **MSIX Vector Assignment**: The IVAR register is used to assign a specific MSI-X vector (vector 0 in this case) to events originating from any of the device's four Tx and four Rx queues [Int20, p. 416]. In this configuration, it notifies the driver about events that happen on Rx queue 0, such as a receive descriptor writeback.
2. **Self-Disabling Interrupt**: The interrupt is configured to automatically disable itself immediately after it is issued. This is a crucial feature that avoids interrupt flooding by keeping the interrupt source masked until the device driver has serviced the event and re-enabled the interrupt (unmasked).

The behavior described above is defined using several key registers:

- **Extended Interrupt Cause Register (EICR)**: This register indicates that an event assigned to a particular interrupt has occurred [Int20, p. 405]. For example, if the first bit (Bit 0) is set, it signals that the event corresponding to MSI-X vector 0 has happened. Given our configuration, where MSI-X vector 0 is assigned to Rx queue 0, this event is most likely an Rx descriptor writeback.

- **Extended Interrupt Mask Set/Read (EIMS)**: This register is used to mask (disable) or unmask (enable) individual interrupts. For an interrupt to be signaled to the host CPU, the corresponding bit in the EIMS register must be unmasked [Int20, pp. 406–407].

An interrupt is issued only when the corresponding event bit is set in EICR and the corresponding mask bit is unmasked in EIMS. (Note: The Extended Interrupt Throttle Register (EITR) register controls the minimum delay between interrupts, currently set to a maximum of 40 microseconds [Int20, pp. 405, 415].)

- **Extended Auto Clear register (EIAC)**: By setting the appropriate bit in EIAC, the corresponding bit in EICR is automatically cleared after the interrupt is delivered [Int20, p. 408], preventing the need for the device driver to manually clear the cause bit.
- **Extended Interrupt Auto Mask Enable Register (EIAM)**: Enabling this register automatically masks the corresponding interrupt in the EIMS register after the interrupt is issued [Int20, p. 405]. This implements the self-disabling behavior: no new interrupts for that specific MSI-X vector can be issued until the device driver explicitly unmasks the bit in EIMS.

5.2.5.4 check_recv_irq() & ack_recv_irq()

The functions `check_recv_irq()` and `ack_recv_irq()` are crucial for implementing a NAPI-like interrupt mitigation scheme in the VNS. Since the hardware automatically masks the interrupt upon generation for the Rx queue, the VNS uses this signal solely to initiate controlled packet polling.

Upon hardware notification of an interrupt, the VNS first calls `check_recv_irq()` to verify the interrupt source by reading the EICR register. This check is necessary for legacy NICs like `e1000` compatible devices (which lack unique MSI-X vectors) or for device drivers such as `ixgbe` when operating in its shared MSI mode for legacy support. However, since the configuration in the `igb` and igc device driver uniquely binds MSI-X vector 0 to Rx queue 0, this specific check is technically redundant.

The driver then calls `rx_batch()` to process a batch of received packets within a budget. If the budget is depleted but packets remain, the VNS reschedules `rx_batch()` to maintain polling. The interrupt is only re-enabled after all packets are processed and no further rescheduling is required, using `ack_recv_irq()` to unmask the corresponding bit in the EIMS register, allowing the NIC to issue new interrupts.

This finalizes the implementation of all interrupt functionality required for the igc device driver to operate within the VNS environment.

5.3 Feature Comparison between ixl igc driver and igc hardware functionality

This section evaluates notable features supported by igc-compatible NICs. As the ixl implementation was designed with minimal core functionality, a comprehensive comparison of all hardware-supported features is beyond the scope of this research. Instead, this analysis focuses on unimplemented features identified during development that could enhance throughput or provide specific benefits to an Real Time Operating System (RTOS).

Features such as VLAN and Time-Sensitive Networking (TSN) are particularly significant for real-time environments. TSN provides essential time synchronization, low packet latency, and increased reliability [LL, p. 1]. Similarly, VLAN support allows for the creation of multiple virtual networks on a single physical interface, enabling the isolation of sensitive data [Hua23, pp. 238–239].

The NIC also supports multiple Rx and Tx queues, allowing traffic to be distributed across different cores to balance workload [Int20, pp. 239, 303]. From an RTOS perspective, this enables the assignment of specific incoming traffic—filtered by MAC, VLAN, or TCP/UDP ports [Int20, p. 243]—to dedicated Rx queues. These queues can be associated with specific MSI-X vectors to facilitate message prioritization. As a separate mechanism for load balancing, Receive Side Scaling (RSS) can be used to distribute received packets across queues based on hashes generated from header fields such as IP or TCP/UDP. For outgoing traffic, the availability of multiple Tx queues allows for the establishment of a hardware-based priority scheme [Int20, p. 303].

Performance-oriented features represent another critical category for reducing host CPU load. Hardware-based checksum offloading for Ethernet, IP, UDP, and TCP [Int20, p. 5] relieves the CPU of calculation duties, preserving cycles for time-critical tasks. The hardware also supports TCP Segmentation Offload (TSO) and, notably for embedded applications, UDP segmentation. These features enable the NIC to automatically segment large messages, thereby reducing the processing overhead on the host CPU [Int20, p. 295]. Hardware-level MAC filtering further improves performance by discarding irrelevant messages before they reach the host system.

Currently, the ixl igc device driver only supports receive checksum offloading for TCP and UDP; for transmission, only the Ethernet frame checksum is offloaded. The driver also utilizes legacy descriptors, which are discontinued and not recommended for new implementations [Int20, pp. 295–296]. Transitioning to advanced descriptors is a prerequisite for many of the advanced features discussed.

Chapter 6

Testing

6.1 Test 1: Ethernet Frame Reception

The general concept and goal of this test are detailed in the high-level overview in subsection 4.3.1. For this test, the Ecrin target runs Linux and provides an emulated environment for L4Re using QEMU. The i225 NIC is passed through directly to the guest system. This setup was chosen to simplify development: since L4Re does not support dynamic loading of software via user input, driver changes normally require a full system restart. Using QEMU allows for faster recompilation and restarts of the guest environment on the Linux host compared to native hardware reboots.

Within L4Re, the I/O server provides the vbus capability containing the i225 NIC to the `ixl-pcap` test utility. This utility instantiates the device driver using the `ixl_init()` function, which scans the vbus for the first available Ethernet device. It then instantiates the corresponding driver based on the PCI vendor and device IDs (as described in subsection 5.2.1). In this case, it discovers the i225 NIC and instantiates the igc device driver object with interrupts disabled. The utility then enters an infinite loop, continuously calling `rx_batch()` to receive and print the contents of incoming packets.

A Linux sender PC is connected to the Ecrin platform through a network switch. As shown in Listing B.1, the sender utilizes a Python script and a raw socket to transmit Ethernet frames directly rather than IP packets. To monitor and verify the contents of the transmitted frames, Wireshark is also run on the sender PC. Utilizing raw Ethernet frames is necessary because, on the receiver side, only the igc device driver is instantiated. As a result, the communication operates solely within OSI layers 1 and 2, which correspond to the physical and data link layers, without implementing the higher layers—such as the network layer (Layer 3)—required for IP-based communication. Consequently, the sender must address the target system using its MAC address to ensure the frames are correctly received.

6.1.1 Execution of the Test

First, the target system is initialized to ensure it is successfully polling the NIC for packets. The MAC address is noted from the driver's initialization logs. On the sender PC, the Python utility is executed using the target's destination MAC address and a test payload: "ixy". During transmission, Wireshark runs on the sender PC to capture and verify the content of the sent packet.

6.1.2 Results

The target system successfully instantiated the device driver and reported the correct MAC address, confirming that the NIC passthrough was operational and the driver had access to the MMIO space. However, when the Python utility was executed on the sender, no messages were received by the target.

Initial assumptions pointed to a defect in the igc device driver implementation. To isolate the issue, the test was repeated using the established igb driver with the i210 NIC already available on the Ecrin. This reference test yielded the same negative result, suggesting an issue with the emulation environment rather than the driver code itself.

To verify this, the test was moved from QEMU to native hardware. On native hardware, the igb driver successfully received the test messages. Finally, the same native test was performed with the igc device driver, which also successfully received the frames. As shown in Figure B.1 and Figure B.2, the captured Ethernet frame from the sender and the frame received by the target system contain identical data, confirming that the message was transmitted and received correctly. This result confirms that the igc device driver is functional for packet reception, fulfilling the first part of requirement R1.

6.2 Test 2: Ethernet Frame Transmission

The general concept and objective of this test are outlined in the high-level overview in subsection 4.3.2. The L4Re operating system runs natively on the Ecrin platform, executing the `ixl-pktgen` test utility provided by ixl. The I/O server provides the vbus capability, containing the i225 device, to the `ixl-pktgen` application. The utility then uses the `ixl_init()` function to identify the matching device driver based on the PCI device ID and instantiates the igc device driver object.

The `ixl-pktgen` utility provides a template for a test packet that includes all necessary Ethernet, IP and UDP header fields. These fields are hardcoded into the packet because no higher-level protocol layers are active on the target system. The receiver PC runs Linux and Wireshark, continuously monitoring the interface connected to the network switch.

6.2.1 Execution of the Test

Both the Ecrin and the receiver PC are connected to the network switch. The MAC and IP addresses of the receiver PC are determined and hardcoded into the `ixl-pktgen` utility, along with the test payload: "ixy" (see Listing B.2 for the implementation example). The application is then compiled and executed on the Ecrin.

6.2.2 Results

Figure B.3 confirms that the transmitted packet was received correctly and contains the exact same bytes as those sent. As in the original, the payload "ixy" is preserved. Furthermore, Wireshark reports no packet errors, indicating that the Ethernet, IP, and UDP headers were hardcoded and transmitted correctly. This result successfully demonstrates the transmit capabilities of the driver, thereby fulfilling the second half of requirement R1 and satisfying the requirement in its entirety.

6.3 Test 3: Multiclient test

The overall goal and concept of this multi-client test are detailed in subsection 4.3.3. The test utilizes the L4Re operating system running natively on the Ecrin platform. Within the L4Re runtime environment, the following L4Re Applications are executed: the I/O server, the VNS (integrating the ixl driver), and two separate user applications, UDP Sender 1 and UDP Sender 2.

The I/O server provides the vbus capability containing the i225 NIC hardware to the VNS. In turn, the VNS provides a capability to access a VNS port for each of the UDP Sender applications. Each UDP Sender application is configured to obtain its IP address via DHCP. Additionally, the IP address of the external receiver is provided as an argument, while the payload message itself is hardcoded and only the numeric identifiers (15 for UDP Sender 2 and 21 for UDP Sender 1) are passed as command-line arguments to be embedded into the messages. Both Sender applications use the lwIP library to continuously transmit UDP packets containing their assigned test payload.

The Ecrin platform is connected to an external network switch. This switch serves as the central connection point for the receiver PC (running Linux and Wireshark on its interface) and the separate DHCP server.

6.3.1 Execution of the Test

After the receiver PC is connected to the switch, its IP address must be determined (in this case, `192.168.241.155`) and the Wireshark capture tool must be initiated. This receiver IP

is then set as a command-line argument for both UDP Sender applications within the Ned configuration file. Unique test payloads ("15" and "21") are also assigned to the respective senders. The operating system binary is then compiled and executed on the Ecrin hardware.

6.3.2 Results

Both UDP Sender applications successfully connected to the VNS, obtained unique IP addresses from the DHCP server, and successfully transmitted UDP packets. As shown in Figure B.4 and Figure B.5, the receiver PC successfully read both the unique "15" and "21" messages, each originating from a different L4Re Application IP address.

6.3.3 Conclusion

This successful outcome establishes multiple functional requirements for the igc device driver:

- **Basic Functionality:** The igc device driver's basic functionality is fully operational and interacts with the VNS without errors.
- **Requirement R2 (Application Interface):** The combination of the VNS and the lwIP library provides a functional interface for L4Re Applications to interact with the igc device driver, thus fulfilling R2.
- **Requirement R3 (Multi-Client Support):** The ability of multiple L4Re applications (UDP Sender 1 and 2) to simultaneously and successfully utilize the single igc device driver proves R3.

Therefore, all requirements concerning basic functionality could be successfully proven. The next test will focus on measuring the bandwidth performance of the igc device driver.

6.4 Test 4: Performance (Bandwidth) Assessment

The primary objective of this test is to measure the throughput of the igc device driver, as conceptualized in subsection 4.3.4. The measurement utilizes iperf, a standard tool for network bandwidth analysis. The setup runs the L4Re operating system natively on the Ecrin platform. The software environment is identical to the previous multi-client test, with the modification that only a single L4Re Application is connected to the VNS: an iperf server. This server obtains its IP address via DHCP and listens on port 68 for incoming client connections.

The Ecrin platform is connected to an external network switch, which provides connectivity to a dedicated DHCP server and a separate Linux PC acting as the iperf client.

6.4.1 Execution of the Test

The Ecrin platform is initialized first to determine the IP address assigned to the iperf server. In this specific instance, the address was `192.168.1.245`. On the Linux PC, the measurement is initiated using the command `iperf -c 192.168.1.245 -p 68`.

6.4.2 Results

Initial measurements yielded a throughput between 107–109 Kbit/s (0.107 Mbit/s). Even without a reference measurement, this was clearly a very low throughput. To provide a comparative baseline and verify the igc device driver configuration, the test was repeated on the Ecrin using the established igb driver on the integrated i210 NIC.

The igb driver consistently achieved speeds between 42.7–43 Mbit/s, approximately 400 times faster than the igc device driver. This discrepancy was unexpected, as the igb and igc device driver share nearly identical code logic.

Further investigation revealed that the igc device driver contained significantly more debugging statements than the igb driver. Specifically, a debug statement within the `rx_batch()` function (line 32) was suspected of causing the bottleneck, as it was being executed for every Rx descriptor processed. Such statements require system calls that trigger expensive context switches within the operating system. To verify this, the line was commented out and the test was repeated. Following this change, the igc device driver achieved a bandwidth between 46.4–47.5 Mbit/s as shown in Figure B.6, aligning its performance with that of the igb driver.

Chapter 7

Evaluation of porting efforts

7.1 Evaluating igc porting efforts

The igc device driver was successfully ported to the ixl framework, fulfilling requirements R1–R3 as verified by corresponding tests.

A key realization during the porting process was that the Linux igc device driver implementation was too extensive to easily isolate the essential configurations required for basic Rx and Tx functionality. Consequently, the methodology was adjusted to use the ixl igb driver as a template for the ixl igc device driver. This shift was justified by evidence that both NIC hardware families provide nearly identical base functionality.

As a result, the code base for the new igc device driver closely resembles the igb driver, with the primary differences lying in the register definitions and the absence of packet statistics. A further distinction exists in hardware filtering: while the igb driver configures a filter for its own MAC address, it does not utilize it as it operates in promiscuous mode. In contrast, the current igc device driver implementation does not implement these filters at all and also defaults to promiscuous mode.

This promiscuous mode is the preferred configuration for integration with the VNS. While enabling MAC address filtering could theoretically reduce unnecessary traffic in high-load environments, it introduces significant architectural constraints. Unlike standard operating system configurations, setting a single primary MAC address filter is insufficient; since packets are addressed to the specific recipient ports of the VNS rather than the NIC itself, a hardware filter would be required for every individual port. Because the i225 NIC supports a maximum of 16 MAC address filters [Int20, pp. 429–430], this alternative would restrict the VNS to a total of 17 ports (including the ixl port). Furthermore, such an implementation would require the VNS to dynamically update hardware filters for every newly created port, significantly increasing system complexity.

Despite these minor architectural differences, the ixl igb and igc device driver provide equivalent functionality for their respective hardware, with the igc device driver performing marginally better in bandwidth evaluations.

By following the igb architecture, the current implementation does not yet incorporate every feature proposed in chapter 4. Specifically, the original plan to enable a general-purpose interrupt and a dedicated Tx queue interrupt was discarded to maintain alignment with the igb driver, which only implements Rx queue interrupts.

7.1.1 Evaluation of Performance Results in Comparison to Linux igc Driver

Although the igc device driver was successfully integrated into the ixl framework, comparison with its Linux counterpart reveals significant potential for future feature support, as discussed in section 5.3.

The most notable difference is the bandwidth disparity. Repeating the bandwidth test with the Linux igc device driver showed a consistent throughput of 2.3 Gbit/s, which is near the maximum theoretical limit of 2.5 Gbit/s for compatible hardware. In contrast, the ixl igc device driver only reached approximately 47 Mbit/s. While a definitive explanation for this discrepancy is beyond the scope of this thesis, the following analysis highlights several factors that may account for it.

First, the Linux igc device driver utilizes four bundled queues for Rx and Tx, distributed across different CPU cores, which potentially quadruples the available processing power. However, this may not be the sole limiting factor; older Intel Gigabit NICs, such as e1000-compatible devices, utilize only a single queue [Int09]. Analysis of the Linux e1000 driver shows it utilizes a single interrupt and a NAPI instance to handle both packet reception and descriptor cleanup [Tor25]. According to NAPI documentation, interrupts must be disabled before scheduling the NAPI instance to prevent race conditions [The25]. Since the e1000 poll function operates on a single queue without additional race condition protections, it serves as theoretical evidence that Gigabit speeds are achievable using a single core and a single queue pair. This could be verified by re-testing the Linux igc device driver restricted to a single software queue.

Another factor could be the Maximum Transmission Unit (MTU) size. While ixl device drivers currently support the standard MTU of 1518 bytes, the Linux driver supports Jumbo frames with an MTU of up to 9000 bytes. This theory could be verified by inspecting captured traffic via Wireshark during bandwidth test with the Linux igc device driver. Furthermore, in the current L4Re network setup, lwIP currently performs IP, UDP, and TCP checksum calculations and packet segmentation in software, whereas the Linux driver offloads these tasks to the hardware.

In addition to these hardware features, the role of interrupt handling can be further researched. As described in subsubsection 5.2.2.1, ixl device driver support different interrupt modes. It could be tested if the VNS also supports the interrupt wait mode and measure if utilizing this mode benefits the performance.

Finally, the organizational logic of the driver may influence performance. The Linux igc device driver handles Tx descriptor cleanup and packet reception within the same NAPI poll function. In the ixl igc device driver, these tasks are currently separate, with Tx cleanup handled in `tx_batch()`. Since the VNS employs `rx_batch()` similarly to the Linux NAPI poll function, performance could benefit from moving the Tx descriptor cleanup to the start of the `rx_batch()` function.

7.2 Evaluating the generic aspects of porting additional Ethernet drivers

Porting a new device driver requires implementing a class based on the abstract `Ixl_device` and registering it within `ixl_init()`, as described in subsection 5.2.1. Currently, the ixl only supports PCI NICs. Consequently, supporting other interfaces, such as USB NICs, would require significant architectural adjustments.

The evaluation of implementing a new device driver is categorized into porting Intel-based drivers and non-Intel drivers. This evaluation focuses on porting a basic non-Intel device driver by rewriting it within the ixl framework. In this context, a basic driver is defined as one that utilizes a single Rx and Tx queue and operates on a single core. This limitation is necessary because the current VNS implementation only supports one pair of queues; extending support to multiple cores or queues would require significant modifications to the VNS itself.

7.2.1 Evaluating the porting of Intel Ethernet drivers

Porting an Intel driver with minimal functionality is relatively straightforward because Intel NICs are architecturally similar at the hardware level. The implementation of igc device driver demonstrates this, as it is nearly an identical copy of the igb driver with modified register addresses.

When porting a new Intel NIC, it is necessary to identify the most compatible existing device driver. Since official documentation may not explicitly state these compatibilities, they must often be determined by analyzing open-source drivers such as the Linux kernel [Tor25] or DPDK [Ope; Lin]. It is generally effective to seek compatibility with drivers designed for similar throughput rates.

Analysis of the Intel device drivers already implemented in ixl shows that the most critical registers are the interrupt registers (typically prefixed with "EI" for Extended Interrupt) and the registers managing Rx and Tx queues. Specifically, the Rx path utilizes RDBAL, RDBAH, RDH, and RDT, while the Tx path uses TDBAL, TDBAH, TDH, and TDT. If the register definitions align with a currently implemented ixl driver, that driver can be utilized as a structural template for the porting process.

The ixgbe driver in ixl provides the most advanced functionality, such as the use of advanced descriptors. Therefore, it serves as an ideal starting point for implementing more complex driver features.

7.2.2 Evaluating the porting of non-Intel Ethernet drivers

The porting of a non-Intel device driver to the ixl framework is based on the premise that all Ethernet NICs share a fundamental set of operations. Every NIC, regardless of manufacturer, manages hardware interrupts, configures DMA transfers, and handles both transmit and receive queues along with their corresponding descriptors and device-specific settings [Her+19, 8:31-11:28]. These core tasks are universal requirements for any device driver to enable functional networking.

While the core tasks of a network driver are universal, the exact placement and structure of this logic are determined by the operating system's specific driver model. Consequently, implementing a non-Intel driver within the ixl framework requires ensuring that the correct functionality is situated within the corresponding ixl functions. To streamline this process and reduce development effort, it is recommended to use an existing driver for the target hardware as a template. The Linux kernel serves as an excellent resource for this purpose due to its extensive library of available drivers. However, if no publicly available source code can be found, or if the reference implementation is overly complex, the driver must be implemented or refined using the hardware datasheet.

Table 7.1 summarizes the most common core functionalities of a network device driver and indicates where they are implemented in ixl compared to a Linux device driver. The table serves as a guideline for identifying which logic belongs to which ixl function and provides additional support when a Linux Ethernet driver is used as a reference for porting. Further information regarding the structure of a Linux device driver and the location of the functions referenced in the table is provided in subsection 5.1.1. It is important to note that not all listed ixl functions must be implemented identically, as discussed previously in section 5.2. In the table, functions that must be implemented are highlighted in red. Conventional functions that should remain close to their original structure because they are tightly integrated into the driver registration process are highlighted in orange. All remaining ixl functions may be modified or regrouped if this simplifies the porting process; however, it is still recommended to use them according to their intended purpose to maintain consistency within the ixl framework. In both Linux and ixl device drivers, functionality is

commonly encapsulated in helper functions that are invoked by a primary initialization function. For ixl, the corresponding helper function is explicitly named in Table 7.1, where the "→" symbol denotes a function call. This convention is not applied to Linux device drivers, as Linux does not follow a consistent naming schemes for helper functions.

A major difference when porting or rewriting a device driver for ixl compared to a Linux device driver lies in interrupt handling. In a Linux device driver, the ISR is implemented within the driver itself, allowing the driver to define custom logic that is executed when an interrupt occurs. In contrast, in the combined ixl and VNS architecture, the VNS is notified of an interrupt and determines how it is processed. Specifically, it uses the function `check_recv_irq()` to identify whether the interrupt was triggered by the reception of a packet. If the interrupt is not receive-related, the VNS takes no further action; otherwise, the received packets are forwarded to the appropriate VNS port. Although this design grants the ixl device driver less direct control over interrupt handling, it does not negatively affect the porting effort; however, this difference should be taken into account.

Porting a non-Intel device driver to ixl is generally more complex than porting an Intel device driver, as no non-Intel device driver is currently implemented in ixl that could serve as a reference template. Despite this challenge, the core functionalities shared across all NICs provide the necessary foundation for adapting drivers from other manufacturers.

Table 7.1: Mapping of common network driver tasks between Linux and ixl

Functionality	**Ixl Function**	**Linux Function**
Map MMIO regions	Constructor	`probe()`
Initialize driver data structures and allocate memory	Constructor	`probe()`
Determine PCI interrupt mode, configure it, and register interrupt with the operating system	Constructor→`msix_init()`	`open()`
Configure NIC interrupt behavior (e.g., throttling, interrupt causes)	`reset_and_init()`, `reset_and_init()`→`enable_rx_interrupt()`	`open()`
Reset NIC to a known clean state	`reset_and_init()`	`probe()`
Allocate DMA memory for the Rx descriptor ring and configure it in the NIC	`reset_and_init()`→`init_rx()`	`open()`
Populate the Rx ring with Rx descriptors and associate each with dedicated DMA memory for packet buffers, then configure the NIC accordingly	`reset_and_init()`→`start_rx_queue()`	`open()`
Check Rx descriptors for completed receptions and replace their packet buffers with newly allocated DMA memory	`rx_batch()`	device driver's NAPI poll function
Allocate DMA memory for the Tx descriptor ring and configure it in the NIC	`reset_and_init()`→`init_tx()`	`open()`
Configure the Tx ring and activate it on the NIC	`reset_and_init()`→`start_tx_queue()`	`open()`
Transmit packets by associating Tx descriptors with valid packets and marking them as ready to send	`tx_batch()`	`.ndo_start_xmit`
Free the memory of Tx packets that have already been transmitted	`tx_batch()`	device driver's NAPI poll function
Interrupt handling	VNS, `check_recv_irq()`, `ack_recv_irq()`	Linux device drivers use ISRs for interrupt handling, which do not follow a consistent naming scheme.

Chapter 8

Summary & Future Work

This thesis focused on porting the Linux igc device driver to the L4Re operating system framework and evaluating the process for porting further device drivers.

The literature review established that hardware devices within the same class—represented in this work by NICs—share fundamental operational principles, which is reflected in the similar structure of their respective device drivers.

To determine the best approach for porting the igc device driver, Genode—including its DDE—was compared to the ixl framework. These frameworks utilize different methodologies: while the DDE aims to integrate existing Linux device drivers without modifications, the ixl framework simplifies the development of network device drivers but requires the driver logic to be rewritten to fit the framework's architecture. Based on criteria C1-C3, ixl was assessed as the most suitable framework. This evaluation also established that rewriting the driver code to fit a new environment is a valid form of porting a device driver.

To reuse parts of the Linux igc device driver, the interfaces of the Linux and ixl network drivers were compared. The analysis showed that while many functions provided similar functionality, they were not identical in the achieved tasks. Additionally, the Linux driver proved to be overly complex due to its extensive feature set, making it unsuitable as a direct template for the igc device driver implementation within ixl.

An essential observation of this work was the high degree of hardware compatibility across Intel NIC families. By leveraging this similarity, the existing igb driver served as a reliable blueprint for the ixl igc device driver implementation, resulting in a nearly identical codebase. Performance evaluations further underscored the success of this strategy, as the two device drivers delivered nearly identical results in bandwidth testing.

Following these results, it can be evaluated that porting additional Intel device drivers to ixl will be a straightforward process. Furthermore, the work established that porting non-Intel device drivers is also feasible because NICs generally operate on similar principles. However, this remains more challenging as the ixl framework currently lacks non-Intel templates.

Without an ixl reference, developers can identify the necessary core functionalities by analyzing other implementations, such as the corresponding Linux device driver, to guide the porting process.

The performance evaluations revealed that the ixl igc device driver is significantly slower than its Linux counterpart. While the Linux igc device driver achieved a maximum speed of 2.3 Gbit/s, the ixl implementation reached only 47 Mbit/s. Although the exact reason for this disparity could not be determined within the scope of this thesis, several hypotheses were proposed to account for the difference. A major difference between the Linux and ixl implementations is that Linux uses multiple cores for packet processing, whereas ixl relies on a single core; however, an older Intel NIC with a device driver configured similarly to the ixl igc device driver is known to be Gigabit-capable, demonstrating that Gigabit speeds are achievable even with the current single-core ixl igc device driver setup.

Future work should focus on porting remaining features from the igb driver, such as statistics reporting, to achieve full functional parity. A critical next step is the transition from legacy to advanced descriptors, as legacy support is being phased out and advanced descriptors are a prerequisite for many modern NIC features. Since the ixgbe driver in ixl already implements advanced descriptors, it can serve as a reference template for this transition. For future implementations within the context of RTOSs, features such as VLAN and TSN support represent particularly relevant areas of research. Finally, determining the root cause of the current performance limitations remains a primary objective for the continued development of the driver.

Appendix A

Igc code examples

Listing A.1: Case statement adapted from function ixl_init() from file device.cc [L4R25a]

```
/* Create and initialize the driver for a certain device. */
switch (vendor_id) {
  // Intel
  case 0x8086:
    switch (device_id) {
      // E1000-compatible devices
      case E1000_DEV_ID_82540EM:
        // The device emulated by QEMU when choosing an e1000 NIC
        ixl_info("Trying e1000...");
        return E1000_device::e1000_init(std::move(dev), cfg);
      // Hereinafter all igb-driver devices
      case IGB_DEV_ID_82576:
      case IGB_DEV_ID_I210:
      case IGB_DEV_ID_I350:
        ixl_warn("The Igb driver provides only a limited feature " "set. You have been warned!");
        return Igb_device::igb_init(std::move(dev), cfg);
      // Hereinafter all ixgbe-driven devices
      case IXGBE_DEV_ID_X540T:
      case IXGBE_DEV_ID_82598:
        ixl_info("Trying ixgbe...");
        return Ixgbe_device::ixgbe_init(std::move(dev), cfg);

      // Hereinafter all igc-driven devices
      // i225 NICs
```

```
25          case IGC_DEV_ID_I225_IT:
26            // ... excluded 6 i225 NIC variations to shorten
                the code example
27          case IGC_DEV_ID_I225_BLANK_NVM:
28          // i226 NICs
29          case IGC_DEV_ID_I226_K:
30            // ... excluded 4 i226 NIC variations to shorten
                the code example
31          case IGC_DEV_ID_I226_BLANK_NVM:
32            ixl_info("Trying IGC...");
33            return Igc_device::igc_init(std::move(dev), cfg);
34
35          default:
36            ixl_error("Unsupported device %x of vendor %x. "
                "No suitable driver found.", device_id,
                vendor_id);
37          break;
38        }
39        break;
40      default:
41        ixl_error("Unknown vendor %x. No suitable driver
            found.", vendor_id);
42        break;
43    }
```

Listing A.2: Adapted igb constructor from file igb.h [L4R25a]

```
1  /*** Constructor                                  ***/
2  Igc_device::Igc_device(L4vbus::Pci_dev&& dev, struct Dev_cfg
     &cfg, uint32_t itr_rate) {
3      l4_timeout_s l4tos;     // L4 timeout object with us
         granularity
4
5      if (cfg.num_rx_queues != 1) {
6          ixl_error("Currently, an Igc device supports exactly
             one receive queue.");
7      }
8      if (cfg.num_tx_queues != 1) {
9          ixl_error("Currently, an Igc device supports exactly
             one transmit queue.");
10     }
11
12     // (see igb for reference)
```

```
13      num_rx_queues = cfg.num_rx_queues;
14      num_tx_queues = cfg.num_tx_queues;
15
16      // Set up IRQ-related data
17      if (cfg.irq_timeout_ms < 0) {
18          l4tos = l4_timeout_from_us(L4_TIMEOUT_US_NEVER);
19      } else {
20          l4tos = l4_timeout_from_us(cfg.irq_timeout_ms * 1000);
21      }
22
23      if (cfg.irq_timeout_ms == 0) {
24          // seems to only be used by ixl driver test packages
25          interrupts.mode = interrupt_mode::Disable;
26      } else if (cfg.irq_timeout_ms == -1) {
27          // default option set by the Virtual Network Switch
28          interrupts.mode = interrupt_mode::Notify;
29      } else {
30          // does not seem to be used anywhere
31          interrupts.mode = interrupt_mode::Wait;
32      }
33
34
35      interrupts.itr_rate = itr_rate;
36      interrupts.timeout  = l4_timeout(l4tos, l4tos);
37
38      /* map io memory based on memory address provided by pci
           register*/
39      pci_dev  = dev;
40
41      // Map BAR0 region
42      ixl_debug("Mapping BAR0 I/O memory...");
43      baddr[0] = pci_map_bar(pci_dev, 0);
44
45      /* dma todo*/
46      // Create a DMA space for this device
47      create_dma_space();
48
49      if(interrupts.mode != interrupt_mode::Disable) {
50          setup_icu_cap();
51          init_msix();
52      }
53
```

```
    rx_queues = calloc(num_rx_queues, sizeof(struct igc_rx_queue) + sizeof(void*) * MAX_RX_QUEUE_ENTRIES);
    tx_queues = calloc(num_tx_queues, sizeof(struct igc_tx_queue) + sizeof(void*) * MAX_TX_QUEUE_ENTRIES);
}
```

Listing A.3: Adapted igb reset_and_init function from file igb.cc [L4R25a]

```
// Reset the device and bring up the link again in a fresh state
void Igc_device::reset_and_init(void) {
    ixl_info("Resetting Igc device");

    // disable interrupts
    disable_interrupts();

    // Stop receive and transmit units, wait for pending transactions to
    // complete
    clear_flags32(baddr[0], IGC_RCTL, IGC_RCTL_EN);
    clear_flags32(baddr[0], IGC_TCTL, IGC_TCTL_EN);
    usleep(10000);

    // reset device
    // Issue the reset command (done by setting the reset bit in the ctrl reg)
    uint32_t ctrl_reg = get_reg32(baddr[0], IGC_CTRL);

    set_reg32(baddr[0], IGC_CTRL, ctrl_reg | IGC_CTRL_DEV_RST | IGC_CTRL_PHY_RST);
    // Wait for NIC to read default settings from EEPROM
    usleep(10000);

    // disable interrupts again after reset
    disable_interrupts();

    ixl_info("Reset completed, starting init phase.");
    /* TODO: perform configuration */

    // init link connection
    ixl_debug("Init link connection");
    init_link();

```

```
    /* TODO: initalize stats */
    /* TODO: initialze Receive & Transmit */
    init_rx();
    ixl_debug("Receive should be enabled: %d",
       get_reg32(baddr[0], IGC_RCTL) & IGC_RCTL_EN);

    init_tx();

    // Start RX and TX queues
    start_rx_queue(0);
    start_tx_queue(0);
    ixl_debug("Transmit should be enabled: %d",
       get_reg32(baddr[0], IGC_TCTL) & IGC_TCTL_EN);
    usleep(1000);

    /* TODO: enable interrupts*/

    // Enable IRQ for receiving packets
    if (interrupts.mode != interrupt_mode::Disable) {
        // Configure the NIC to use multiple MSI-X mode (one
           IRQ per queue),
        // also set other flags recommended in section
           7.3.3.11
        set_flags32(baddr[0], IGC_GPIE, IGC_GPIE_MMSIX |
                                        IGC_GPIE_NSICR |
                                        IGC_GPIE_EIAME |
                                        IGC_GPIE_PBA);

        // For now, only enable the IRQ of the first receive
           queue
        enable_rx_interrupt(0);
    }

    /* enable promiscuous to receive all network packages
       (without promiscuous the NIC drops all pkgs that are
       not addressed to it. Destination MAC != NIC MAC)*/
    set_promisc(true);

    // if no link is found it waits for 10 secs for link to
       come up
    wait_for_link();

```

```
}
```

Listing A.4: init_link function adapted from igb constructor [L4R25a]

```
void Igc_device::init_link(void) {
  uint32_t ctrl;

  ixl_info("Setting link up");

  ctrl = get_reg32(baddr[0], IGC_CTRL);

  ctrl |= IGC_CTRL_SLU;
  ctrl &= ~(IGC_CTRL_FRCSPD | IGC_CTRL_FRCDPX); // force
     auto-neg and duplex

  set_reg32(baddr[0], IGC_CTRL, ctrl);
}
```

Listing A.5: init_rx function adapted from igb.cc [L4R25a]

```
void Igc_device::init_rx(void) {
    // function is based on ixl igb driver

    // For now we assume that this function is only called
       immediately after
    // a reset operation with RX and TX disabled, so we do
       not need to
    // disable them again here.

    // Disable VLAN filtering as we do not support it anyways
    clear_flags32(baddr[0], IGC_RCTL, IGC_RCTL_VFE);

    for (uint16_t i = 0; i < num_rx_queues; i++) {
        ixl_debug("initializing rx queue %d", i);


        // Instruct NIC to drop packets if no RX descriptors
           are available
        // FIXME: When using multiple RX queues, choose the
           rights SRRCTL reg
        set_flags32(baddr[0], IGC_SRRCTL0, IGC_SRRCTL_DREN);

```

```
19          struct igc_rx_queue* queue = ((struct igc_rx_queue*)
               rx_queues) + i;
20
21          uint32_t ring_size_bytes = NUM_RX_QUEUE_ENTRIES *
               sizeof(struct igc_rx_desc);
22          struct dma_memory mem = memory_allocate_dma(*this,
               ring_size_bytes);
23
24          // neat trick from Snabb: initialize to 0xFF to
               prevent rogue memory
25          // accesses on premature DMA activation
26          memset(mem.virt, -1, ring_size_bytes);
27
28          // Keep a reference to mem in the queue, otherwise
               the object will go
29          // out of scope, leading to the revocation of the
               backing capability
30          queue->descr_mem = mem;
31
32          // private data for the driver, 0-initialized
33          queue->num_entries = NUM_RX_QUEUE_ENTRIES;
34          queue->rx_index    = 0;
35          queue->descriptors = (struct igc_rx_desc*) mem.virt;
36
37          // Tell the device where it can write to (its iova,
               so DMA addrs)
38          set_reg32(baddr[0], IGC_RDBAL, (uint32_t) (mem.phy &
               0xFFFFFFFFull));
39          set_reg32(baddr[0], IGC_RDBAH, (uint32_t) (mem.phy >>
               32));
40          set_reg32(baddr[0], IGC_RDLEN, ring_size_bytes);
41          // Set ring to empty at start
42          set_reg32(baddr[0], IGC_RDH, 0);
43          set_reg32(baddr[0], IGC_RDT, 0);
44          ixl_debug("rx ring %d phy addr:  0x%012llX", i,
               mem.phy);
45          ixl_debug("rx ring %d virt addr: 0x%012lX", i,
               (uintptr_t) mem.virt);
46      }
47
48      // Enable checksum offloading for CRC and received
           UCP/TCP packets
```

```
    set_flags32(baddr[0], IGC_RXCSUM, IGC_RXCSUM_TUOFL |
        IGC_RXCSUM_CRCOFL);

    // Merely kicks of the RX part by setting the RX enabled
        bit in RCTL
    // Also enables reception of broadcast frames (BAM)
    set_flags32(baddr[0], IGC_RCTL, IGC_RCTL_EN |
        IGC_RCTL_BAM);
}
```

Listing A.6: start_rx_queue() function adapted from igb.cc [L4R25a]

```
void Igc_device::start_rx_queue(int queue_id) {
    ixl_debug("starting rx queue %d", queue_id);
    struct igc_rx_queue* queue = ((struct igc_rx_queue*)
        rx_queues) + queue_id;

    // Allocate packet buffers and set backing memory for
        descriptors
    // 2048 as pktbuf size is strictly speaking incorrect:
    // we need a few headers (1 cacheline), so there's only
        1984 bytes
    // left for the device but the 82599 can only handle
        sizes in increments
    // of 1 kb; but this is fine since our max packet size is
        the default
    // MTU of 1518 this has to be fixed if jumbo frames are
        to be supported
    // mempool should be >= the number of rx and tx
        descriptors for a
    // forwarding application
    int mempool_size = MIN_MEMPOOL_ENTRIES << 4;

    // Create the RX memory pool and reserve the minimum
        number of packets
    // for use by the driver.
    queue->mempool = new Mempool(*this, mempool_size,
        PKT_BUF_ENTRY_SIZE,
                                 MEMPOOL_LIMIT);
    queue->mempool->reserve(MIN_MEMPOOL_ENTRIES);
    if (queue->num_entries & (queue->num_entries - 1)) {
        ixl_debug("Numb of queue entries: %u",
            queue->num_entries);
```

```
        ixl_error("number of queue entries must be a power of 2");
    }
    for (int j = 0; j < queue->num_entries; j++) {
        volatile struct igc_rx_desc* rxd = queue->descriptors + j;
        struct pkt_buf* buf = queue->mempool->pkt_buf_alloc();
        if (!buf) {
                ixl_error("failed to allocate rx buffer");
        }

        // Set buffer address and clear RXD flags
        rxd->buf_addr = buf->buf_addr_phy + offsetof(struct pkt_buf, data);
        rxd->errors   = 0;
        rxd->status   = 0;
        // we need to return the virtual address in the rx function
        // which the descriptor doesn't know by default
        queue->virtual_addresses[j] = buf;
    }

    ixl_debug("SRRCTL0 = %x", get_reg32(baddr[0], IGC_SRRCTL0));

    // Wait for the enable bit to show up
    // FIXME: Implement this for other queues as well
    set_flags32(baddr[0], IGC_RXDCTL0, IGC_RXDCTL_EN);
    wait_set_reg32(baddr[0], IGC_RXDCTL0, IGC_RXDCTL_EN);

    // Only now set the final head and tail pointers (were initialized to 0)
    set_reg32(baddr[0], IGC_RDH, 0);
    set_reg32(baddr[0], IGC_RDT, queue->num_entries - 1);
}
```

Listing A.7: rx_batch() function adapted from igb.cc [L4R25a]

```
uint32_t Igc_device::rx_batch(uint16_t queue_id, struct pkt_buf* bufs[], uint32_t num_bufs) {
    struct interrupt_queue* interrupt = NULL;
    bool interrupt_wait = interrupts.mode == interrupt_mode::Wait;
```

```
4
5     // For non-debug builds insert an additional bounds check
         for the queue_id
6     l4_assert(queue_id == 0);
7     struct igc_rx_queue* queue = ((struct igc_rx_queue*)
         rx_queues) + queue_id;
8
9     if (interrupt_wait) {
10        interrupt = &interrupts.queues[queue_id];
11    }
12
13    if (interrupt_wait && interrupt->interrupt_enabled) {
14        if (! queue->rx_pending) {
15            // We only listen on IRQs caused by this RQ, so
                 nothing to do
16            // afterwards as auto clearing is enabled
17            interrupt->irq->receive(interrupts.timeout);
18        }
19    }
20
21    // rx index we checked in the last run of this function
22    uint16_t rx_index = queue->rx_index;
23    // index of the descriptor we checked in the last
         iteration of the loop
24    uint16_t last_rx_index = rx_index;
25    uint32_t buf_index;
26
27    for (buf_index = 0; buf_index < num_bufs; buf_index++) {
28        // rx descriptors are explained in section 3.2.3
29        volatile struct igc_rx_desc* desc_ptr =
             queue->descriptors + rx_index;
30        uint8_t status = desc_ptr->status;
31
32        // ixl_debug("Queue Discriptor status: 0x%x", status);
33
34        if (status & IGC_RXD_STAT_DD) {
35            if (!(status & IGC_RXD_STAT_EOP)) {
36                ixl_error("multi-segment packets are not
                     supported - "
37                          "increase buffer size or decrease
                             MTU");
38            }
```

```
39
40              // got a packet, read and copy the whole descriptor
41              struct igc_rx_desc desc;
42              memcpy(&desc, (const void *) desc_ptr, sizeof(desc));
43              struct pkt_buf* buf = (struct pkt_buf*) queue->virtual_addresses[rx_index];
44              buf->size = desc.length;
45              // this would be the place to implement RX offloading by translating
46              // the device-specific flags to an independent representation in
47              // the buf (similiar to how DPDK works)
48              // need a new mbuf for the descriptor
49              struct pkt_buf* new_buf = queue->mempool->pkt_buf_alloc();
50              if (! new_buf) {
51                  // At this point, we just have to trust in this problem not to
52                  // be caused by a real packet buffer leak or bug (as immediately
53                  // terminating the driver might not be a good idea w.r.t. upper
54                  // layers). Hence, we hope for the best and periodically check
55                  // whether new buffers arrived (On rare occasions, there might
56                  // be false negatives in the current mempool implementation).
57                  ixl_warn("failed to allocate new mbuf for rx, you are either "
58                           "leaking memory or your mempool is too small");
59                  usleep(1000000);
60                  new_buf = queue->mempool->pkt_buf_alloc();
61              }
62
63              // reset the descriptor (new buffer address and zero out flags)
64              // TODO: Is the zeroing of flags actually necessary for an E1000?
```

```
            desc_ptr->buf_addr = new_buf->buf_addr_phy + offsetof(struct pkt_buf, data);
            desc_ptr->errors   = 0;
            desc_ptr->status   = 0;

            queue->virtual_addresses[rx_index] = new_buf;
            bufs[buf_index] = buf;

            // want to read the next one in the next iteration,
            // but we still need the last/current to update RDT later
            last_rx_index = rx_index;
            rx_index = wrap_ring(rx_index, queue->num_entries);
        }
        else {
            break;
        }
    }
    if (rx_index != last_rx_index) {
        // tell hardware that we are done
        // this is intentionally off by one, otherwise we'd set RDT=RDH if
        // we are receiving faster than packets are coming in
        // RDT=RDH means queue is full
        set_reg32(baddr[0], IGC_RDT, last_rx_index);
        queue->rx_index = rx_index;

        // Check whether there are unprocessed descriptors left
        uint32_t head = get_reg32(baddr[0], IGC_RDH);
        if (head == ((last_rx_index + 1) % (uint32_t) queue->num_entries))
            queue->rx_pending = false;
        else
            queue->rx_pending = true;
    }

    // Perform IRQ bookkeeping
    if (interrupt_wait) {
        interrupt->rx_pkts += buf_index;
```

```

        if ((interrupt->instr_counter++ & 0xFFF) == 0) {
            bool int_en = interrupt->interrupt_enabled;
            uint64_t diff = device_stats::monotonic_time() - interrupt->last_time_checked;
            if (diff > interrupt->interval) {
                // every second
                check_interrupt(interrupt, diff, buf_index, num_bufs);
            }

            if (int_en != interrupt->interrupt_enabled) {
                if (interrupt->interrupt_enabled) {
                    enable_rx_interrupt(queue_id);
                } else {
                    disable_rx_interrupt(queue_id);
                }
            }
        }
    }

    // If the driver runs in irq_wait mode, it should take care full
    // responsibility of IRQ handling (i.e., IRQ masking and unmasking are not
    // done manually by the application). Thus, we have to unmask the receive
    // IRQ here to guarantee a swift notification about newly incoming packets.
    if (interrupt_wait && interrupt->interrupt_enabled)
        Igc_device::ack_recv_irq(queue_id);

    return buf_index;
}
```

Listing A.8: tx_init() function adapted from igb.cc [L4R25a]

```
void Igc_device::init_tx(void) {
    // FIXME: Enable mq support for this type of NIC
    for (uint16_t i = 0; i < num_tx_queues; i++) {
        struct igc_tx_queue* queue = ((struct igc_tx_queue*) tx_queues) + i;
        ixl_debug("initializing tx queue %d", i);
```

```

        // setup descriptor ring, see section 7.1.9
        uint32_t ring_size_bytes = NUM_TX_QUEUE_ENTRIES * sizeof(struct igc_tx_desc);
        struct dma_memory mem = memory_allocate_dma(*this, ring_size_bytes);
        memset(mem.virt, -1, ring_size_bytes);

        // tell the device where it can write to (its iova, so DMA addrs)
        set_reg32(baddr[0], IGC_TDBAL, (uint32_t) (mem.phy & 0xFFFFFFFFull));
        set_reg32(baddr[0], IGC_TDBAH, (uint32_t) (mem.phy >> 32));
        set_reg32(baddr[0], IGC_TDLEN, ring_size_bytes);
        ixl_debug("tx ring %d phy addr:  0x%012llX", i, mem.phy);
        ixl_debug("tx ring %d virt addr: 0x%012lX", i, (uintptr_t) mem.virt);

        // Init TX queue to empty
        set_reg32(baddr[0], IGC_TDH, 0);
        set_reg32(baddr[0], IGC_TDT, 0);

        // Keep a reference to mem in the queue, otherwise the object will go
        // out of scope, leading to the revocation of the backing capability
        queue->descr_mem   = mem;

        // private data for the driver, 0-initialized
        queue->num_entries = NUM_TX_QUEUE_ENTRIES;
        queue->descriptors = (struct igc_tx_desc*) mem.virt;
    }
}
```

Listing A.9: start_tx_queue() function adapted from igb.cc [L4R25a]

```
/* Kicks of the TX part by configuring the TCTL register accordingly        */
void Igc_device::start_tx_queue(int queue_id) {
    (void) queue_id;

```

```
    // Enable queue 0
    // FIXME: As an optimization, we could also think about enabling bursted
    //        write back of finished send descriptors here.
    set_flags32(baddr[0], IGC_TXDCTL0, IGC_TXDCTL_EN);

    uint32_t tctl = get_reg32(baddr[0], IGC_TCTL);

    // Clear collision threshold bitmask
    tctl &= ~IGC_TCTL_CT;

    // Set collision threshold default values (16, as demanded by IEEE).
    tctl |= IGC_COLLISION_THRESHOLD << IGC_CT_SHIFT;

    // Enable TX queue, enable padding of short packets
    set_reg32(baddr[0], IGC_TCTL, tctl | IGC_TCTL_EN | IGC_TCTL_PSP);
}
```

Listing A.10: tx_batch() function adapted from igb.cc [L4R25a]

```
uint32_t Igc_device::tx_batch(uint16_t queue_id, struct pkt_buf* bufs[], uint32_t num_bufs) {
    struct igc_tx_queue* queue = ((struct igc_tx_queue*) tx_queues) + queue_id;
    // the descriptor is explained in section 3.3.2
    // we just use a struct copy & pasted from intel, but it basically has two formats (hence a union):
    // 1. the write-back format which is written by the NIC once sending it is finished this is used in step 1
    // 2. the read format which is read by the NIC and written by us, this is used in step 2

    uint16_t clean_index = queue->clean_index; // next descriptor to clean up

    // step 1: clean up descriptors that were sent out by the hardware and return them to the mempool
    // start by reading step 2 which is done first for each packet
    // cleaning up must be done in batches for performance reasons, so this is unfortunately somewhat complicated
```

```
13      while (true) {
14          // figure out how many descriptors can be cleaned up
15
16          // tx_index is always ahead of clean (invariant of
               our queue)
17          int32_t cleanable = queue->tx_index - clean_index;
18          if (cleanable < 0) {
19              // handle wrap-around
20              cleanable = queue->num_entries + cleanable;
21          }
22          if (cleanable < TX_CLEAN_BATCH) {
23              break;
24          }
25          // calculcate the index of the last transcriptor in
               the clean batch
26          // we can't check all descriptors for performance
               reasons
27          int32_t cleanup_to = clean_index + TX_CLEAN_BATCH - 1;
28          if (cleanup_to >= queue->num_entries) {
29              cleanup_to -= queue->num_entries;
30          }
31          volatile struct igc_tx_desc* txd = queue->descriptors
               + cleanup_to;
32          // hardware sets this flag as soon as it's sent out,
               we can give back all bufs in the batch back to the
               mempool
33          // comment from linux driver abouth this register: /*
               if DD is not set pending work has not been
               completed */
34          if (txd->upper.data & IGC_TXD_STAT_DD) {
35              int32_t i = clean_index;
36              while (true) {
37                  struct pkt_buf* buf = (struct pkt_buf *)
                       queue->virtual_addresses[i];
38                  pkt_buf_free(buf);
39                  if (i == cleanup_to) {
40                      break;
41                  }
42                  i = wrap_ring(i, queue->num_entries);
43              }
44              // next descriptor to be cleaned up is one after
                   the one we just cleaned
```

```
            clean_index = wrap_ring(cleanup_to, queue->num_entries);
        } else {
            // clean the whole batch or nothing; yes, this leaves some packets in
            // the queue forever if you stop transmitting, but that's not a real concern
            break;
        }
    }
    queue->clean_index = clean_index;

    // step 2: send out as many of our packets as possible
    uint32_t sent;
    for (sent = 0; sent < num_bufs; sent++) {
        uint32_t next_index = wrap_ring(queue->tx_index, queue->num_entries);
        // we are full if the next index is the one we are trying to reclaim
        if (clean_index == next_index) {
            break;
        }
        struct pkt_buf* buf = bufs[sent];
        // remember virtual address to clean it up later
        queue->virtual_addresses[queue->tx_index] = (void*)buf;
        volatile struct igc_tx_desc* txd = queue->descriptors + queue->tx_index;
        queue->tx_index = next_index;
        // NIC reads from here
        txd->buffer_addr  = buf->buf_addr_phy + offsetof(struct pkt_buf, data);

        // Reset descriptor command before sending (otherwise NIC won't emit any data!)
        txd->lower.data          = 0;
        txd->lower.flags.length  = buf->size;
        txd->upper.fields.status = 0;


        // always the same flags: one buffer (EOP), CRC offload, report status
```

```
        txd->lower.data |= IGC_TXD_CMD_EOP | IGC_TXD_CMD_IFCS | IGC_TXD_CMD_RS;

        // no fancy offloading stuff - only the total payload length
        // implement offloading flags here:
        //      * ip checksum offloading is trivial: just set the offset
        //      * tcp/udp checksum offloading is more annoying,
        //        you have to precalculate the pseudo-header checksum
        // TODO: Implement TCP / UDP offloading here...
    }

    // send out by advancing tail, i.e., pass control of the bufs to the nic
    // this seems like a textbook case for a release memory order,
    // but Intel's driver doesn't even use a compiler barrier here
    set_reg32(baddr[0], IGC_TDT, queue->tx_index);
    return sent;
}
```

Listing A.11: enable_rx_interrupt() function adapted from igb.cc [L4R25a]

```
// Enables a receive interrupt of the NIC.

   */
void Igc_device::enable_rx_interrupt(uint16_t qid) {
    // Get MSI-X vector allocated for the respective RX queue
    uint32_t msi_vec = interrupts.queues[qid].msi_vec;
    // Current content of IVAR register
    uint32_t ivar;

    // Allocate an IRQ vector via the IVAR reg, see also sections
    // 7.3.2 and 8.8.15 of the I350 programmers manual
    ivar = get_reg32(baddr[0], IGC_IVAR0 + 4 * (qid / 2));

    if ((qid % 2) == 0) {
```

```
        // Restrict queue ID to 5 bit in length and set valid bit
        // for the new entry in the IVAR reg
        ivar    = ivar | (msi_vec & 0x0000001f) | 0x00000080;

        // Set new IVAR value
        set_reg32(baddr[0], IGC_IVAR0 + 4 * (qid / 2), ivar);
    }
    else {
        // Restrict queue ID to 5 bit in length and set valid bit
        // for the new entry in the IVAR reg
        uint32_t vec = (msi_vec & 0x0000001f) << 16;
        ivar         = ivar | vec | 0x00800000;

        // Set new IVAR value
        set_reg32(baddr[0], IGC_IVAR0 + 4 * (qid / 2), ivar);
    }

    // Limit the ITR to prevent IRQ storms
    set_reg32(baddr[0], IGC_EITR + 4 * msi_vec,
              (interrupts.itr_rate & 0x00001fff) << 2);

    // No auto clear, following an interrupt, software might read the EICR
    // register to check for the interrupt causes.
    set_flags32(baddr[0], IGC_EIAC, 1 << msi_vec);

    // Set the auto mask in the EIAM register according to the preferred mode of
    // operation.
    if (interrupts.mode == interrupt_mode::Notify)
        // In Notify mode we prefer auto-masking the interrupts.
        set_flags32(baddr[0], IGC_EIAM, 1 << msi_vec);
    else if (interrupts.mode == interrupt_mode::Wait)
        // In Wait mode we prefer not auto-masking the interrupts.
        clear_flags32(baddr[0], IGC_EIAM, 1 << msi_vec);

    // Enable the receive interrupt cause
    set_reg32(baddr[0], IGC_EIMS, 1 << msi_vec);
```

```
}
```

Listing A.12: init_msix() function adapted from igb.cc [L4R25a]

```
void Igc_device::init_msix(void) {
    // based on igb setup_interrupt()

    interrupts.queues = (struct interrupt_queue*) malloc(num_rx_queues * sizeof(struct interrupt_queue));

    // FIXME: We should rely on MSI-X / MSI only for Igb devices, also these
    //        NIC have more than one interrupt available
    //
    // Determine type of interrupt available at the device. We will only go
    // with MSI-X in non-SR-IOV mode.
    if (pcidev_supports_msix(pci_dev)) {
        uint32_t bir;               // BAR location of MSI-X table
        uint32_t table_offs;        // Offset of MSI-X table in BAR
        uint32_t table_size;        // Size of MSI-X table

        ixl_info("Using MSI-X interrupts...");
        interrupts.interrupt_type = IXL_IRQ_MSIX;
        setup_msix(pci_dev);

        pcidev_get_msix_info(pci_dev, &bir, &table_offs, &table_size);
        // Check whether the requested BAR is already mapped
        if (baddr[bir] == NULL) {
            ixl_debug("Mapping in BAR%u for accessing MSI-X table.", bir);
            baddr[bir] = pci_map_bar(pci_dev, bir);
        }

        // For now we will do a 1:1 mapping between RX queue number and the
        // MSI-X vector allocated for the respective queue
        for (unsigned int rq = 0; rq < num_rx_queues; rq++) {
            // L4 representation of the MSI vector. We need to add flags to
```

```
31              // use L4's API correctly...
32              uint32_t          msi_vec_l4;
33              l4_icu_msi_info_t msi_info;
34
35              // This is what took me three hours to realize:
                   In the L4Re API,
36              // the same interface is used to handle legacy
                   IRQs and MSIs.
37              // Apparently, this is why it is mandatory to add
                   a special MSI
38              // flag to the actual MSI vector ID when calling
                   L4Re functions
39              // that should do something w.r.t. to said MSI...
40              msi_vec_l4 = rq | L4::Icu::F_msi;
41
42              create_and_bind_irq(msi_vec_l4,
                   &interrupts.queues[rq].irq, interrupts.vicu);
43
44              // Get the MSI info
45              uint64_t source = pci_dev.dev_handle() |
                   L4vbus::Icu::Src_dev_handle;
46
47              L4Re::chksys(
48                interrupts.vicu->msi_info(
49                  msi_vec_l4, source, &msi_info
50                ),
51                "Failed to retrieve MSI info."
52              );
53
54              ixl_debug("MSI info: vector = 0x%x addr = %llx,
                   data = %x", rq, msi_info.msi_addr,
                   msi_info.msi_data);
55
56              // PCI-enable of MSI-X
57              pcidev_enable_msix(rq, msi_info, baddr[bir],
                   table_offs, table_size);
58
59              L4Re::chksys(
60                l4_ipc_error(
61                  interrupts.vicu->unmask(msi_vec_l4),
62                  l4_utcb()
63                ),
```

```
64                "Failed to unmask interrupt"
65            );
66
67            ixl_debug("MSI-X vector allocated for RX queue %u
                 is %u", rq, rq);
68
69            interrupts.queues[rq].moving_avg.length = 0;
70            interrupts.queues[rq].moving_avg.index  = 0;
71            interrupts.queues[rq].msi_vec           = rq;
72            interrupts.queues[rq].interval =
                 INTERRUPT_INITIAL_INTERVAL;
73
74            ixl_debug("Attached to MSI-X %u", rq);
75        }
76    }
77    else {
78        // Disable IRQs completely if MSI-X is not present
79        interrupts.interrupt_type = IXL_IRQ_LEGACY;
80
81        ixl_warn("Device does not support MSIs. Disabling
             interrupts...");
82        interrupts.mode = interrupt_mode::Disable;
83        return;
84    }
85 }
```

Appendix B

Test Utilities and Results

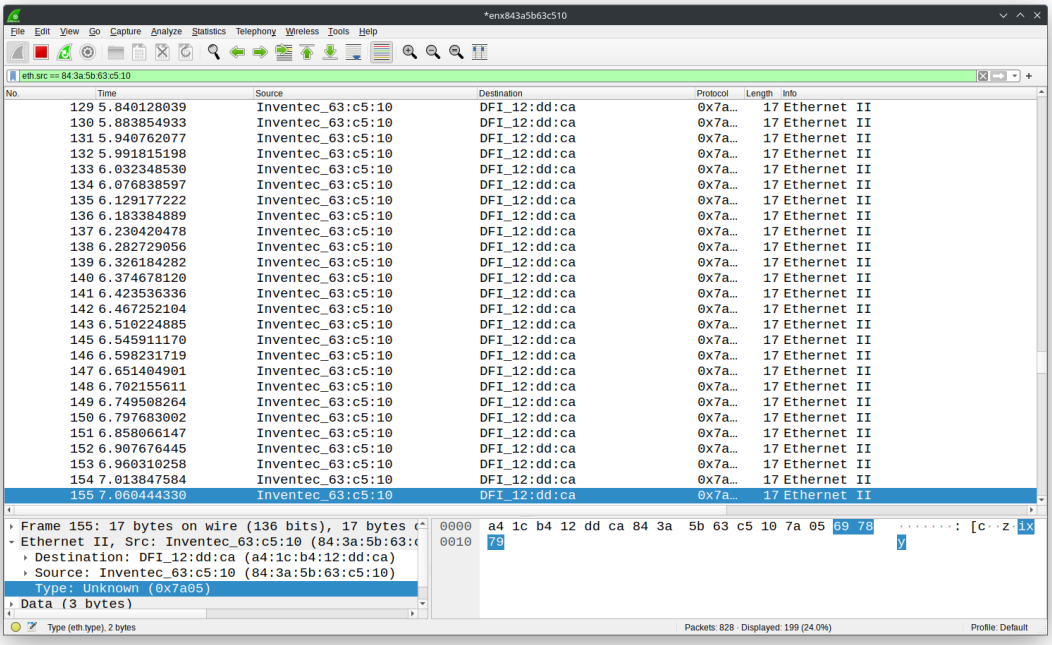


Figure B.1: Test 1: Wireshark output on the transmitting PC

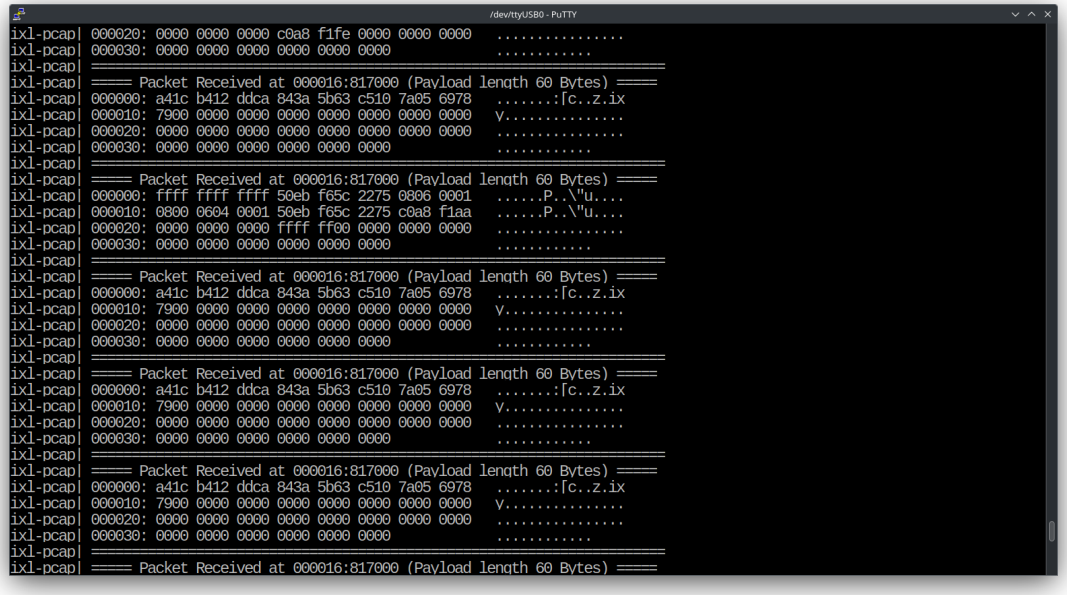


Figure B.2: Test 1: Console output of receiving PC

Listing B.1: Python Utility to send raw Ethernet frames

```python
from socket import *
import struct
import argparse # Import argparse for command-line arguments

def sendeth(src, dst, eth_type, payload, interface="eno1"):
  """
  Sends a raw Ethernet frame.

  Args:
    src (bytes): 6-byte source MAC address.
    dst (bytes): 6-byte destination MAC address.
    eth_type (int): 16-bit EtherType value (e.g., 0x0800 for IPv4).
    payload (bytes): The data to send in the frame's payload.
    interface (str): The network interface to send the frame on.

  Returns:
    int: The number of bytes sent.
  """
  # Use the struct module to pack the EtherType (16-bit integer) into 2 bytes.
  eth_type_bytes = struct.pack('!H', eth_type)

```

```
22    # Create the raw frame: Destination MAC + Source MAC +
        EtherType + Payload
23    frame = dst + src + eth_type_bytes + payload
24
25    # Create a raw socket (AF_PACKET is Linux-specific)
26    s = socket(AF_PACKET, SOCK_RAW)
27
28    # Bind the socket to the specified interface and protocol
        (0 means all protocols)
29    s.bind((interface, 0))
30
31    # Send the raw frame
32    return s.send(frame)
33
34 # --- Helper function to convert a string MAC address (e.g.,
     '3C:EC:EF:D6:29:B4') to bytes ---
35 def mac_str_to_bytes(mac_str):
36     """Converts a colon-separated MAC address string to a
         6-byte bytes object."""
37     try:
38         # Split the string by ':' and convert each hex part
             to an integer
39         # Then convert the list of integers back to a bytes
             object
40         return bytes.fromhex(mac_str.replace(':', ''))
41     except ValueError:
42         raise argparse.ArgumentTypeError(f"Invalid MAC
             address format: {mac_str}. Use XX:XX:XX:XX:XX:XX
             format.")
43
44 if __name__ == "__main__":
45     # example usage sudo python3 send_eth_frames.py --src
         3C:EC:EF:D6:29:B4 --dst 84:3A:5B:63:C5:10 --iface eno1
         --payload "Test Frame 123"
46
47     # 1. Setup Argument Parsing
48     parser = argparse.ArgumentParser(description="Send a raw
         Ethernet frame with specified MAC addresses.")
49
50     parser.add_argument('--src', required=True,
         type=mac_str_to_bytes,
```

```
                        help='Source MAC address (e.g., 3C:EC:EF:D6:29:B4)')
    parser.add_argument('--dst', required=True, type=mac_str_to_bytes,
                        help='Destination MAC address (e.g., 84:3A:5B:63:C5:10)')
    parser.add_argument('--iface', default='eno1',
                        help='Network interface to use (default: eno1)')
    parser.add_argument('--payload', default='hello world',
                        help='The string payload to send (default: "hello world")')

    args = parser.parse_args()

    # 2. Convert string payload to bytes
    data_payload = args.payload.encode('utf-8')

    # 3. Define EtherType (example value)
    eth_type_int = 0x7A05 # Arbitrary/Experimental EtherType

    # 4. Call the sendeth function with user-specified arguments
    try:
        bytes_sent = sendeth(
            src=args.src,
            dst=args.dst,
            eth_type=eth_type_int,
            payload=data_payload,
            interface=args.iface
        )
        print(f"Sent {bytes_sent}-byte Ethernet packet on {args.iface} from {args.src.hex(':')} to {args.dst.hex(':')}")
    except PermissionError:
        print("\nError: Permission denied. You must run this script with root/superuser privileges (e.g., using 'sudo').")
    except Exception as e:
        print(f"\nAn error occurred: {e}")
```

Listing B.2: Hardcoded UDP message for testing transmit functionality

```
static const uint8_t pkt_data[] = {
    0x84, 0x3A, 0x5B, 0x63, 0xC5, 0x10, // dst MAC 84:3a:5b:63:c5:10
    0x3C, 0xEC, 0xEF, 0xD4, 0x37, 0x61, // src MAC 3c:ec:ef:d4:37:61
    0x08, 0x00,                         // ether type: IPv4
    0x45, 0x00,                         // Version, IHL, TOS
    (PKT_SIZE - 14) >> 8,               // ip len excluding ethernet, high byte
    (PKT_SIZE - 14) & 0xFF,             // ip len exlucding ethernet, low byte
    0x00, 0x00, 0x00, 0x00,             // id, flags, fragmentation
    0x40, 0x11, 0x00, 0x00,             // TTL (64), protocol (UDP), checksum
    0xC0, 0xA8, 0xF1, 0x96,             // src ip (192.168.241.150)
    0xC0, 0xA8, 0xF1, 0x9B,             // dst ip (192.168.241.155)
    0x1F, 0x90, 0x1F, 0x90,             // src and dst ports (8080 -> 8080)
    (PKT_SIZE - 20 - 14) >> 8,          // udp len excluding ip & ethernet, high byte
    (PKT_SIZE - 20 - 14) & 0xFF,        // udp len exlucding ip & ethernet, low byte
    0x00, 0x00,                         // udp checksum, optional
    'i', 'x', 'y'                       // payload
    // rest of the payload is zero-filled because mempools guarantee empty bufs
};
```

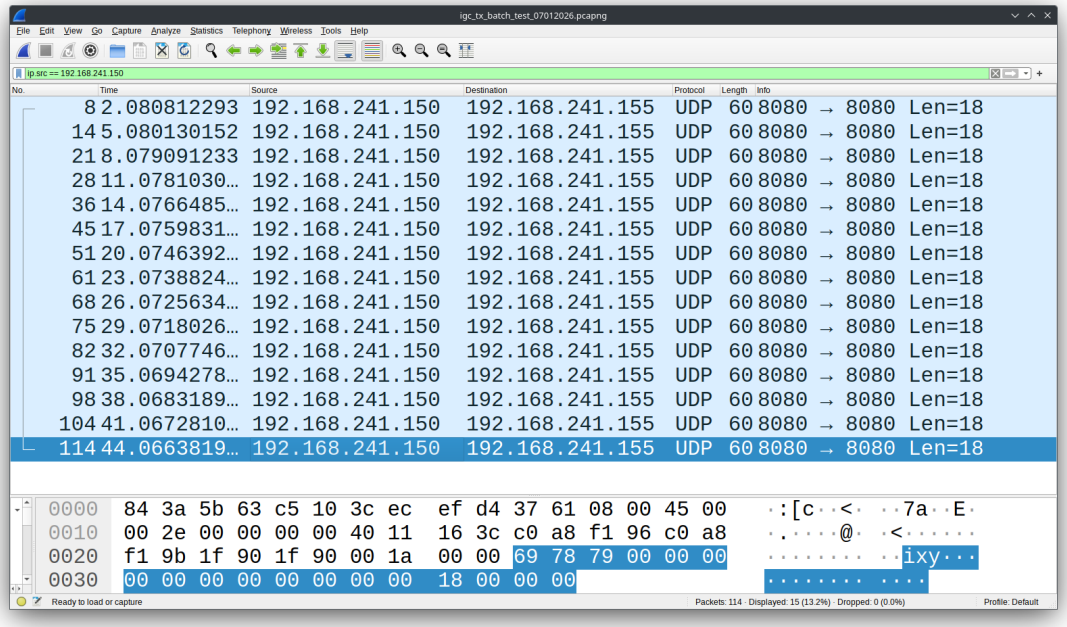


Figure B.3: Test 2: Wireshark output on the receiver PC

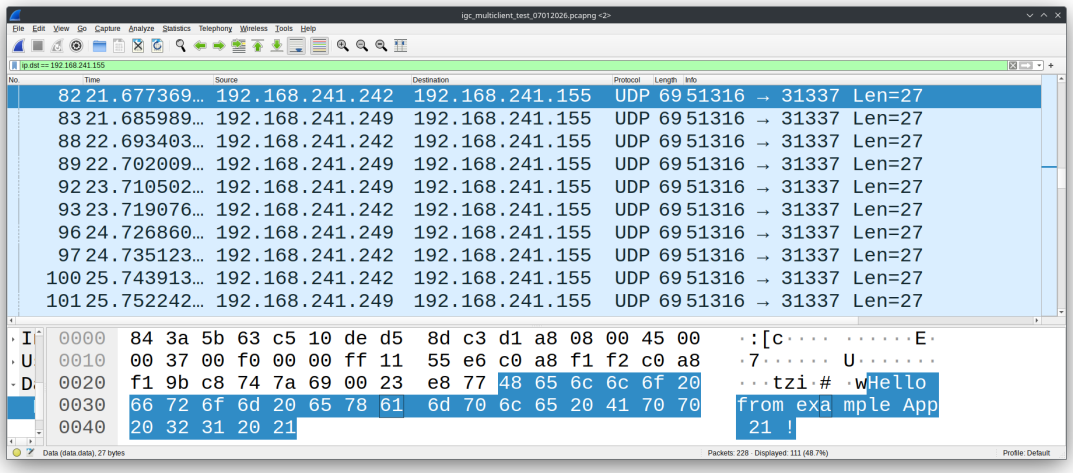


Figure B.4: Test 3: Wireshark output showing packet of UDP Sender 1

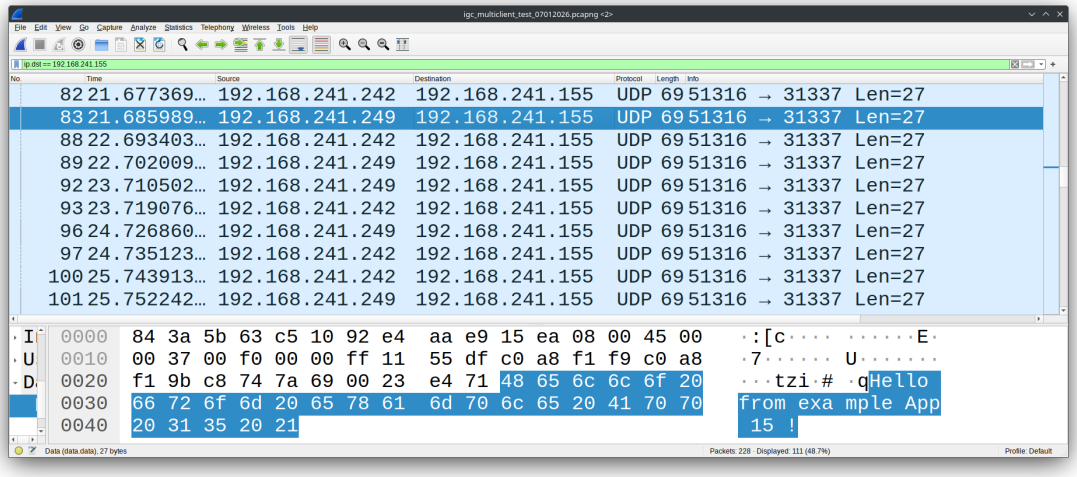


Figure B.5: Test 3: Wireshark output showing packet of UDP Sender 2

```
------------------------------------------------------------
Client connecting to 192.168.241.247, TCP port 68
TCP window size:  416 KByte (WARNING: requested 10.0 MByte)
------------------------------------------------------------
[  1] local 192.168.241.155 port 37178 connected with 192.168.241.247 port 68 (icwnd/mss/irtt=5/512/763)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-10.0465 sec  56.9 MBytes  47.5 Mbits/sec
------------------------------------------------------------
Client connecting to 192.168.241.247, TCP port 68
TCP window size:  416 KByte (WARNING: requested 10.0 MByte)
------------------------------------------------------------
[  1] local 192.168.241.155 port 54462 connected with 192.168.241.247 port 68 (icwnd/mss/irtt=5/512/401)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-10.0464 sec  56.8 MBytes  47.4 Mbits/sec
------------------------------------------------------------
Client connecting to 192.168.241.247, TCP port 68
TCP window size:  416 KByte (WARNING: requested 10.0 MByte)
------------------------------------------------------------
[  1] local 192.168.241.155 port 39108 connected with 192.168.241.247 port 68 (icwnd/mss/irtt=0/512/0)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-10.0482 sec  55.6 MBytes  46.4 Mbits/sec
```

Figure B.6: Test 4: Performance results of the ixl igc device driver

List of Figures

2.1 Overview of modern computer systems adapted from [Bau23, pp. 43–44] . . 4

2.2 Paging Example adapted from [SG23, p. 383] 5

2.3 PCI System Example [SA99, p. 546] 7

2.4 The L4 family tree (simplified). The x-axis is a timeline in calendar years. [HE16] 14

2.5 L4Re architecture [Lac19, p. 6] 15

2.6 Paging Hierarchy in the L4Re [Döb12b, 1:10:57] 19

2.7 I/O Resource Management by the I/O server [L4R25c, p. 63] 22

2.8 Final DDE architecture [Hel01, p. 27] 27

2.9 Simplified Example of Genode Architecture [Gen25] 28

2.10 dde_kit Architecture [Gen25] 29

4.1 Overview of NIC and CPU Connection on the Ecrin 42

4.2 I225-IT NIC architecture and planned device driver features adapted from [Rut] 43

4.3 Software Architecture 44

5.1 Initial Tx Ring State 64

5.2 State of Tx Ring after enqueueing 5 packets 65

5.3 State of Tx Ring after sending 4 packets 66

5.4 State of Tx Ring after 3 packets are cleaned 67

B.1 Test 1: Wireshark output on the transmitting PC xxxvii

B.2 Test 1: Console output of receiving PC xxxviii

B.3 Test 2: Wireshark output on the receiver PC xlii

B.4 Test 3: Wireshark output showing packet of UDP Sender 1 xlii

B.5 Test 3: Wireshark output showing packet of UDP Sender 2 xliii

B.6 Test 4: Performance results of the ixl igc device driver xliii

List of Tables

3.1 Compatibility Overview 33
3.2 LOC comparison for network drivers between ixl and Linux [L4R25a][Tor25] 38
3.3 Summary of Evaluation Assessments for Genode and ixl Frameworks 40

5.1 igc device driver essential functions found in igc_main.c [Tor25] 50
5.2 Essential Functions of an ixl device driver [L4R25a] 51
5.3 Match between Linux and Ixl Device Driver Interface 53
5.4 Virtual and conventional functions of an ixl device driver 54

7.1 Mapping of common network driver tasks between Linux and ixl 81

Listings

5.1 pci_driver struct igc_main.c Linux kernel 48

5.2 Snippet from the file igc_main.c from the Linux kernel 48

5.3 Showcasing igb and igc macro limitation 59

A.1 Adapted case statement from function ixl_init() from file device.cc xv

A.2 Adapted igb constructor from file igb.h xvi

A.3 Adapted igb reset_and_init function from file igb.cc xviii

A.4 init_link function adapted from igb constructor xx

A.5 init_rx function adapted from igb.cc xx

A.6 start_rx_queue() function adapted from igb.cc xxii

A.7 rx_batch() function adapted from igb.cc xxiii

A.8 tx_init() function adapted from igb.cc xxvii

A.9 start_tx_queue() function adapted from igb.cc xxviii

A.10 tx_batch() function adapted from igb.cc xxix

A.11 enable_rx_interrupt() function adapted from igb.cc xxxii

A.12 init_msix() function adapted from igb.cc xxxiv

B.1 Python Utility to send raw Ethernet frames xxxviii

B.2 Hardcoded UDP message configured in ixl-pktgen xli

Bibliography

[ABY11] Nadav Amit, Muli Ben-Yehuda, and Ben-Ami Yassour. "IOMMU: Strategies for Mitigating the IOTLB Bottleneck". In: *Computer Architecture*. Ed. by Ana Lucia Varbanescu, Anca Molnos, and Rob Van Nieuwpoort. Vol. 6161. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 256–274. ISBN: 978-3-642-24321-9 978-3-642-24322-6. DOI: 10.1007/978-3-642-24322-6_22. URL: http://link.springer.com/10.1007/978-3-642-24322-6_22 (visited on 10/10/2025).

[Aik+06] Mark Aiken et al. "Deconstructing Process Isolation". In: *Proceedings of the 2006 Workshop on Memory System Performance and Correctness*. MSPC '06. New York, NY, USA: Association for Computing Machinery, Oct. 22, 2006, pp. 1–10. ISBN: 978-1-59593-578-6. DOI: 10.1145/1178597.1178599. URL: https://dl.acm.org/doi/10.1145/1178597.1178599 (visited on 10/09/2025).

[ARR24] Timos Ampelikiotis, Alvise Rigo, and Daniel Raho. "Technical Overview & Performance Evaluation of Virtio-loopback". In: *2024 International Conference Automatics and Informatics (ICAI)*. 2024 International Conference Automatics and Informatics (ICAI). Oct. 2024, pp. 164–169. DOI: 10.1109/ICAI63388.2024.10851664. URL: https://ieeexplore.ieee.org/document/10851664/ (visited on 10/17/2025).

[BAS03] Ravi Budruk, Don Anderson, and Ed Solari. *PCI Express System Architecture*. Pearson Education, Sept. 2003. ISBN: 978-0-321-15630-3.

[Bau23] Christian Baun. *Operating Systems / Betriebssysteme*. Ed. by Christian Baun. Wiesbaden: Springer Fachmedien, 2023. ISBN: 978-3-658-42230-1. DOI: 10.1007/978-3-658-42230-1_4. URL: https://doi.org/10.1007/978-3-658-42230-1_4 (visited on 09/24/2025).

[BC05] Daniel Bovet and Marco Cesati. *Understanding The Linux Kernel*. Oreilly & Associates Inc, Sept. 2005. ISBN: 978-0-596-00565-8.

[Bel22] Nahom Aseged Belay. "Securing the Boot Process of Embedded Linux Systems". MA thesis. Trondheim, Norway: Norwegian University of Science and Technology, June 2022.

[Ben+07] Muli Ben-Yehuda et al. "The Price of Safety: Evaluating IOMMU Performance". In: *Proceedings of the Linux Symposium*. Vol. one. Ottawa, Canada, June 30, 2007.

[Ben05] Christian Benvenuti. *Understanding Linux Network Internals*. O'Reilly Media, Inc., Nov. 2005. ISBN: 978-0-596-00255-8.

[Ben18] Jiri Benc. "The Network Packet's Diary: A Kernel Journey". Devconf.Cz. 2018. URL: `https://www.youtube.com/watch?v=6Fl1rsxk4JQ` (visited on 12/07/2025).

[Boo09] Bootlin. *Kernel – Network Device Driver Programming*. Bootlin, 2009. URL: `https://bootlin.com/doc/legacy/network-drivers/network-drivers-lab.pdf` (visited on 09/12/2025).

[Bud+12] Ravi Budruk et al. *PCI Express Technology 3.0*. Mindshare Press, Sept. 2012. ISBN: 978-0-9770878-6-0.

[Che22] Maxime Chevallier. *Multi-Queue Improvements in Linux Kernel Ethernet Driver Mvneta*. Mar. 17, 2022. URL: `https://bootlin.com/blog/multi-queue-improvements-in-linux-kernel-ethernet-mvneta/` (visited on 11/25/2025).

[CRK05] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman. *Linux Device Drivers, 3rd Edition*. O'Reilly Media, Inc., Jan. 2005. ISBN: 978-0-596-00590-0.

[Döb12a] Björn Döbel. "Introduction to Microkernels" (Technische Universität Dresden). July 18, 2012. URL: `https://www.youtube.com/watch?v=SyOcmVamsmI` (visited on 10/14/2025).

[Döb12b] Björn Döbel. "Memory, IPC, and L4Re" (Technische Universität Dresden). July 20, 2012. URL: `https://www.youtube.com/watch?v=1sAtnvsltIk`.

[Dun] Adam Dunkels. *lwIP - A Lightweight TCP/IP Stack*. URL: `https://savannah.nongnu.org/projects/lwip/` (visited on 12/01/2025).

[EC24] Ata Elahi and Alex Cushman. *Computer Networks: Data Communications, Internet and Security*. Cham: Springer International Publishing, 2024. ISBN: 978-3-031-42017-7 978-3-031-42018-4. DOI: `10.1007/978-3-031-42018-4`. URL: `https://link.springer.com/10.1007/978-3-031-42018-4` (visited on 12/02/2025).

[ecr25] ecrin Systems. *nanoONYX User Manual*. unpublished, Aug. 22, 2025. URL: `https://ecrin.com/en/onyx-rugged-pc/ultra-small-low-power-modular-rugged-computer`.

[Emb23] EmbeTronicX. *Linux Device Driver Tutorial – Part 1 | Introduction*. July 3, 2023. URL: `https://embetronicx.com/tutorials/linux/device-drivers/linux-device-driver-part-1-introduction/` (visited on 09/04/2025).

[Emm+18] Paul Emmerich et al. "User Space Network Drivers". In: *Proceedings of the 2018 Applied Networking Research Workshop*. ANRW '18. New York, NY, USA: Association for Computing Machinery, July 16, 2018, pp. 91–93. ISBN: 978-1-4503-5585-8. DOI: 10.1145/3232755.3232767. URL: https://dl.acm.org/doi/10.1145/3232755.3232767 (visited on 09/15/2025).

[Emm+19] Paul Emmerich et al. "User Space Network Drivers". In: *Technical University of Munich* (2019). URL: https://www.net.in.tum.de/fileadmin/bibtex/publications/papers/ixy-writing-user-space-network-drivers.pdf.

[Emm25] Paul Emmerich. *Emmericp/Ixy*. Sept. 12, 2025. URL: https://github.com/emmericp/ixy (visited on 09/15/2025).

[Fes11] Norman Feske. *Relationship of Genode, L4Re, Fiasco.OC, NOVA (Was Re: Genode vs. L4Re)*. E-mail. Wed May 18 10:50:50 CEST 2011. URL: https://lists.genode.org/pipermail/users/2011-May/000466.html (visited on 10/18/2025).

[Fes23] Norman Feske. "Genode Operating System Framework Platforms 23.05". In: (May 2023).

[Fes25] Norman Feske. "Genode Operating System Framework Foundations". In: (June 6, 2025).

[FHB25] Norman Feske, Christian Helmuth, and Alexander Böttcher. *Genode - Github*. Aug. 28, 2025. URL: https://github.com/genodelabs/genode/ (visited on 10/27/2025).

[Gen25] Genode Labs GmbH. *Genode - Genode Operating System Framework*. 2025. URL: https://genode.org/ (visited on 08/05/2025).

[HE16] Gernot Heiser and Kevin Elphinstone. "L4 Microkernels: The Lessons from 20 Years of Research and Deployment". In: *ACM Trans. Comput. Syst.* 34.1 (Apr. 6, 2016), 1:1–1:29. ISSN: 0734-2071. DOI: 10.1145/2893177. URL: https://dl.acm.org/doi/10.1145/2893177 (visited on 08/01/2025).

[Hei+07] Gernot Heiser et al. "Towards Trustworthy Computing Systems: Taking Microkernels to the next Level". In: *ACM SIGOPS Operating Systems Review* 41.4 (July 2007), pp. 3–11. ISSN: 0163-5980. DOI: 10.1145/1278901.1278904. URL: https://dl.acm.org/doi/10.1145/1278901.1278904 (visited on 08/07/2025).

[Hel01] Christian Helmuth. "Generische Portierung von Linux-Gerätetreibern Auf Die DROPS-Architektur". Diplomarbeit. TU Dresden, Fakultät Informatik, Institut für Systemarchitektur, Professur Betriebssysteme, July 2001.

[Her+19] Tom Herbert et al. "IETF 105 Technology Deep Dive: How Network Interface Cards (NICs) Work Today". In: July 23, 2019. URL: https://www.youtube.com/watch?v=wHM7RVk3-yk (visited on 09/04/2025).

[Hoh22] Michael Hohmuth. *L4Re Operating System Framework - Kernkonzept*. May 10, 2022. URL: https://www.kernkonzept.com (visited on 08/27/2025).

[Hua23] Huawei Technologies Co., Ltd. *Data Communications and Network Technologies*. Singapore: Springer Nature, 2023. ISBN: 978-981-19-3028-7 978-981-19-3029-4. DOI: 10.1007/978-981-19-3029-4. URL: https://link.springer.com/10.1007/978-981-19-3029-4 (visited on 12/02/2025).

[ICN08] Hiroo Ishikawa, Alexandre Courbot, and Tatsuo Nakajima. "A Framework for Self-Healing Device Drivers". In: *2008 Second IEEE International Conference on Self-Adaptive and Self-Organizing Systems*. 2008 Second IEEE International Conference on Self-Adaptive and Self-Organizing Systems. Oct. 2008, pp. 277–286. DOI: 10.1109/SASO.2008.43. URL: https://ieeexplore.ieee.org/document/4663431 (visited on 08/01/2025).

[Int] © Intel Corporation. *Intel® Core™ I7-1185GRE Processor (12M Cache, up to 4.40 GHz) - Product Specifications*. Intel. URL: https://www.intel.com/content/www/us/en/products/sku/208082/intel-core-i71185gre-processor-12m-cache-up-to-4-40-ghz/specifications.html (visited on 09/23/2025).

[Int09] © Intel Corporation. "8254x Family of Gigabit Ethernet Controllers Software Developer's Manual". In: (Mar. 2009).

[Int20] © Intel Corporation. *Intel® Ethernet Controller I225 Software User Manual*. DocumentPub, Aug. 3, 2020. URL: https://dokumen.pub/intel-foxville-i225-25-gbps-ethernet-controller-software-user-manual-13nbsped.html.

[Int21] © Intel Corporation. *Intel® Ethernet Controller I210 Datasheet*. Jan. 2021. URL: https://cdrdv2.intel.com/v1/dl/getContent/333016.

[Int24] © Intel Corporation. *Intel® Ethernet Controller I225/ I226 – Product Brief*. Intel. Apr. 15, 2024. URL: https://cdrdv2.intel.com/v1/dl/getContent/621753?fileName=621753-Intel%C2%AE+Ethernet+Controller+I225-I226-Product+Brief-20240415.pdf (visited on 11/08/2025).

[Int25] © Intel Corporation. *Supported Operating Systems for Intel® Ethernet Controllers (LOM)*. Intel. July 31, 2025. URL: https://www.intel.com/content/www/us/en/support/articles/000055236.html (visited on 09/14/2025).

[KVM] KVM. *10G NIC Performance: VFIO vs Virtio - KVM*. URL: https://linux-kvm.org/page/10G_NIC_performance:_VFIO_vs_virtio (visited on 10/17/2025).

[L4R25a] L4Re Operating System Framework. *Ixl - A Collection of Userspace Network Drivers for L4Re*. Sept. 10, 2025. URL: https://github.com/L4Re/ixl (visited on 09/14/2025).

[L4R25b] L4Re project members and individual contributors. *Kernkonzept/Fiasco*. Kernkonzept, Sept. 26, 2025. URL: https://github.com/kernkonzept/fiasco (visited on 09/29/2025).

[L4R25c] L4Re project members and individual contributors. *L4Re Operating System Framework - Developer Documentation*. Sept. 28, 2025. URL: https://l4re.org/doc/l4re-doc.pdf (visited on 03/10/2025).

[L4R25d] L4Re project members and individual contributors. *L4Re Operating System Framework Documentation*. Aug. 12, 2025. URL: https://l4re.org (visited on 08/27/2025).

[L4R25e] L4Re project members and individual contributors. *L4Re/Lwip*. Sept. 27, 2025. URL: https://github.com/L4Re/lwip (visited on 01/06/2026).

[L4R25f] L4Re project members and individual contributors. *Networking with Curl — L4Re Operating System Framework Documentation*. Aug. 31, 2025. URL: https://l4re.org/use-cases/net-swt-curl/index.html (visited on 09/14/2025).

[Lac19] Adam Lackorzynski. "The L4Re Microkernel". June 2019. URL: https://os.inf.tu-dresden.de/Studium/MkK/SS2019/06_l4re.pdf.

[LAN22] LAN/MAN Standards Committee. "IEEE Standard for Ethernet". In: *IEEE Std 802.3-2022 (Revision of IEEE Std 802.3-2018)* (July 2022), pp. 1–7025. DOI: 10.1109/IEEESTD.2022.9844436. URL: https://ieeexplore.ieee.org/document/9844436 (visited on 01/09/2026).

[Lev84] Henry M. Levy. *Capability-Based Computer Systems*. Bedford, Mass: Digital Press, 1984. ISBN: 978-0-932376-22-0.

[Lie93] Jochen Liedtke. "Improving IPC by Kernel Design". In: *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles*. SOSP '93. New York, NY, USA: Association for Computing Machinery, Dec. 1, 1993, pp. 175–188. ISBN: 978-0-89791-632-5. DOI: 10.1145/168619.168633. URL: https://dl.acm.org/doi/10.1145/168619.168633 (visited on 09/15/2025).

[Lie95] J. Liedtke. "On Micro-Kernel Construction". In: *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles*. SOSP '95. New York, NY, USA: Association for Computing Machinery, Dec. 3, 1995, pp. 237–250. ISBN: 978-0-89791-715-5. DOI: 10.1145/224056.224075. URL: https://dl.acm.org/doi/10.1145/224056.224075 (visited on 08/05/2025).

[Lie96] J. Liedtke. "µ-Kernels Must And Can Be Small". In: *Proceedings of the Fifth International Workshop on Object-Orientation in Operation Systems*. The Fifth International Workshop on Object-Orientation in Operation Systems. Oct. 1996, pp. 152–155. DOI: 10.1109/IWOOOS.1996.557909. URL: https://ieeexplore.ieee.org/document/557909 (visited on 08/08/2025).

[Lin] Linux Foundation. *DPDK: Data Plane Development Kit*. URL: https://core.dpdk.org/supported/nics/intel/ (visited on 12/18/2025).

[LL] Gunnar Lessmann and Ong Boon Leong. *Implementing Real-Time System Using Intel Time-Sensitive Networking Capable Ethernet Controller on Linux Operating System*. URL: https://builders.intel.com/docs/networkbuilders/implementing-real-time-system-using-intel-time-sensitive-networking-capable-ethernet-controller-on-linux-operating-system-1709536025.pdf.

[LW09] Adam Lackorzynski and Alexander Warg. "Taming Subsystems: Capabilities as Universal Resource Access Control in L4". In: *Proceedings of the Second Workshop on Isolation and Integration in Embedded Systems*. IIES '09. New York, NY, USA: Association for Computing Machinery, Mar. 31, 2009, pp. 25–30. ISBN: 978-1-60558-464-5. DOI: 10.1145/1519130.1519135. URL: https://dl.acm.org/doi/10.1145/1519130.1519135 (visited on 08/03/2025).

[Mic] Microsoft. *PCI Driver Programming Guide - Windows Drivers*. URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/pci/ (visited on 10/12/2025).

[MLD18] Pedro Mejia-Alvarez, Luis Eduardo Leyva-del-Foyo, and Arnaldo Diaz-Ramirez. *Interrupt Handling Schemes in Operating Systems*. SpringerBriefs in Computer Science. Cham: Springer International Publishing, 2018. ISBN: 978-3-319-94492-0 978-3-319-94493-7. DOI: 10.1007/978-3-319-94493-7. URL: http://link.springer.com/10.1007/978-3-319-94493-7 (visited on 10/18/2025).

[MYS03] Mark S Miller, Ka-Ping Yee, and Jonathan Shapiro. "Capability Myths Demolished". In: (2003).

[Nem10] Evi Nemeth, ed. *Unix and Linux System Administration Handbook*. 4. ed. Upper Saddle River, NJ Munich: Prentice Hall, 2010. 1279 pp. ISBN: 978-0-13-148005-6.

[Nov25] Bojan Novković. "Improving Monolithic Operating System Kernel Security and Robustness Through Kernel Subsystem Isolation". PhD thesis. Zagreb, Croatia: University of Zagreb, 2025.

[OJ01] MATTIAS O'Nils and AXEL Jantsch. "Device Driver and DMA Controller Synthesis from HW /SW Communication Protocol Specifications". In: *Design Automation for Embedded Systems* 6.2 (Apr. 1, 2001), pp. 177–205. ISSN: 1572-8080. DOI: 10.1023/A:1011246731756. URL: https://doi.org/10.1023/A:1011246731756 (visited on 09/11/2025).

[Ope] Open Source Contributors. *DPDK/Dpdk: Data Plane Development Kit*. URL: https://github.com/DPDK/dpdk (visited on 12/11/2025).

[Pal22] Jonah Palmer. *Introduction to VirtIO*. May 24, 2022. URL: https://blogs.oracle.com/linux/post/introduction-to-VirtIO (visited on 10/16/2025).

[Roc04] Benjamin Roch. "Monolithic Kernel vs. Microkernel". In: (2004).

[Rus+15] Rusty Russell et al. *Virtual I/O Device (VIRTIO) Version 1.0*. 2015.

[Rus08] Rusty Russell. “Virtio: Towards a de-Facto Standard for Virtual I/O Devices”. In: *ACM SIGOPS Operating Systems Review* 42.5 (July 2008), pp. 95–103. ISSN: 0163-5980. DOI: 10.1145/1400097.1400108. URL: https://dl.acm.org/doi/10.1145/1400097.1400108 (visited on 10/16/2025).

[Rut] Rutronik. *INTEL® ETHERNET CONTROLLER I225*. URL: https://www.rutronik.com/fileadmin/images/Future_Markets/Advanced_Robotics/Fact_Sheets/Intel_robotics-fs_Ethernet.pdf.

[SA99] Tom Shanley and Don Anderson. *PCI System Architecture (4th Ed.)* USA: Addison-Wesley Longman Publishing Co., Inc., 1999. 787 pp. ISBN: 978-0-201-30974-4.

[SG23] Frank Slomka and Michael Glaß. *Grundlagen der Rechnerarchitektur: Von der Schaltung zum Prozessor*. Wiesbaden: Springer Fachmedien, 2023. ISBN: 978-3-658-36658-2 978-3-658-36659-9. DOI: 10.1007/978-3-658-36659-9. URL: https://link.springer.com/10.1007/978-3-658-36659-9 (visited on 09/23/2025).

[SGG12] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts, 9th Edition*. Wiley, Dec. 11, 2012. 944 pp. ISBN: 978-1-118-55963-5. Google Books: 9VMcAAAAQBAJ.

[sob21] sobkulir. *Intel NIC Drivers Confusion*. r/osdev. Nov. 22, 2021. URL: https://www.reddit.com/r/osdev/comments/qzyhvf/intel_nic_drivers_confusion/ (visited on 12/08/2025).

[Sta18] William Stallings. *Operating Systems: Internals and Design Principles*. Ninth edition, global edition. Harlow, Essex: Pearson Education Limited, 2018. 1 p. ISBN: 978-0-13-467095-9 978-1-292-21430-6.

[Sta96] René Stange. “Systematische Übertragung von Gerätetreibern von Einem Monolithischen Betriebssystem Auf Eine Mikrokernbasierte Architektur”. Diplomarbeit. Technische Universität Dresden, Fakultät Informatik, Institut für Betriebssysteme, Datenbanken und Rechnernetze, Lehrstuhl Betriebssysteme, May 1996.

[TB15] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. 4. ed. Boston: Prentice Hall, 2015. 1101 pp. ISBN: 978-0-13-359162-0 978-1-292-06142-9.

[The25] ©The kernel development community. *NAPI — The Linux Kernel Documentation*. Dec. 14, 2025. URL: https://docs.kernel.org/networking/napi.html#id3 (visited on 12/06/2025).

[Tor25] Linus Torvalds. *Linux Kernel - Github*. Sept. 10, 2025. URL: https://github.com/torvalds/linux (visited on 09/10/2025).

[Tsi18] Michael S. Tsirkin. “Virtio as a Universal Communication Format”. Linux Plumbers Conference 2018 (Vancouver). Nov. 13, 2018. URL: https://www.youtube.com/watch?v=QNUX3AVBaG0.

[TU 05] TU Dresden. *DROPS - Software*. Feb. 24, 2005. URL: https://os.inf.tu-dresden.de/drops/software.html (visited on 10/26/2025).

[Url23] Christoph Urlacher. "Interrupt Handling Using the X86 APIC". BA thesis. Düsseldorf, Germany: Heinrich Heine University Düsseldorf, Mar. 8, 2023. URL: https://coconucos.cs.uni-duesseldorf.de/lehre/abschlussarbeiten/BA-Urlacher-23.pdf.

[VA11] Durga Vemuri and Wasim Al-Hamdani. "Measures to Improve Security in a Microkernel Operating System". In: *Proceedings of the 2011 Information Security Curriculum Development Conference*. InfoSecCD '11: Information Security Curriculum Development Conference. Kennesaw Georgia: ACM, Sept. 30, 2011, pp. 25–33. ISBN: 978-1-4503-0812-0. DOI: 10.1145/2047456.2047460. URL: https://dl.acm.org/doi/10.1145/2047456.2047460 (visited on 10/05/2025).

[WDC12] Wenji Wu, Phil Demar, and Matt Crawford. "A Transport-Friendly NIC for Multicore/Multiprocessor Systems". In: *IEEE Transactions on Parallel and Distributed Systems* 23.4 (Apr. 2012), pp. 607–615. ISSN: 1558-2183. DOI: 10.1109/TPDS.2011.195. URL: https://ieeexplore.ieee.org/document/5953595/ (visited on 11/06/2025).

[Wu+15] Jianliang Wu et al. "PaddyFrog: Systematically Detecting Confused Deputy Vulnerability in Android Applications". In: *Security and Communication Networks* 8.13 (Sept. 10, 2015), pp. 2338–2349. ISSN: 19390114. DOI: 10.1002/sec.1179. URL: https://onlinelibrary.wiley.com/doi/10.1002/sec.1179 (visited on 10/05/2025).